

**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP.HỒ CHÍ
MINH**

KHOA ĐIỆN - ĐIỆN TỬ

BỘ MÔN KỸ THUẬT MÁY TÍNH – VIỄN THÔNG



HCMUTE

ĐỒ ÁN TỐT NGHIỆP

Đề tài

**TRIỂN KHAI THIẾT KẾ VÀ THI CÔNG HỆ THỐNG
CẢNH BÁO VA CHẠM TRÊN PHƯƠNG TIỆN ĐIỀU KHIỂN**

Sinh viên: **NGUYỄN ĐÌNH THỐNG**

MSSV: 22119236

LƯU PHÚC KHANG

MSSV: 22119188

TP. HỒ CHÍ MINH – 6/2026

**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP.HỒ CHÍ
MINH**

KHOA ĐIỆN - ĐIỆN TỬ

BỘ MÔN KỸ THUẬT MÁY TÍNH – VIỄN THÔNG



HCMUTE

ĐỒ ÁN TỐT NGHIỆP

Đề tài

**TRIỂN KHAI THIẾT KẾ VÀ THI CÔNG HỆ THỐNG
CẢNH BÁO VA CHẠM TRÊN PHƯƠNG TIỆN ĐIỀU KHIỂN**

Sinh viên: NGUYỄN ĐÌNH THỐNG

MSSV: 22119236

LƯU PHÚC KHANG

MSSV: 22119188

Hướng dẫn: ThS. NGUYỄN NGÔ LÂM

TP. HỒ CHÍ MINH – 6/2026

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP

(Dành cho giảng viên hướng dẫn)

Đề tài: TRIỂN KHAI THIẾT KẾ VÀ THỰC HIỆN THỐNG
CÁNH BẢO VÀ CHẠM TRÊN PHƯỜNG TIỀN ĐIỀU KHIỂN

Sinh viên: + Nguyễn Đình Thông
+ Lưu Phúc Khang

MSSV: 22119236

MSSV: 22119188

Hướng dẫn:

Nhận xét bao gồm các nội dung sau đây:

1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và thực tiễn:
Đặt vấn đề rõ ràng, mục tiêu cụ thể; đề tài có tính mới, cấp thiết; đề tài có khả năng ứng dụng, tính sáng tạo.

2. Phương pháp thực hiện/ phân tích/ thiết kế:

Phương pháp hợp lý và tin cậy dựa trên cơ sở lý thuyết; có phân tích và đánh giá phù hợp; có tính mới và tính sáng tạo.

3. Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:

Phù hợp với mục tiêu đề tài; phân tích và đánh giá/ kiểm thử thiết kế hợp lý; có tính sáng tạo/ kiểm định chặt chẽ và đảm bảo độ tin cậy.

4. Kết luận và đề xuất:

Kết luận phù hợp với cách đặt vấn đề, đề xuất mang tính cải tiến và thực tiễn; kết luận có đóng góp mới mẻ, đề xuất sáng tạo và thuyết phục.

5. Hình thức trình bày và bố cục báo cáo:

Văn phong nhất quán, bố cục hợp lý, cấu trúc rõ ràng, đúng định dạng mẫu; có tính hấp dẫn, thể hiện năng lực tốt, văn bản trau chuốt.

6. Kỹ năng chuyên nghiệp và tính sáng tạo:

Thể hiện các kỹ năng giao tiếp, kỹ năng làm việc nhóm, và các kỹ năng chuyên nghiệp khác trong việc thực hiện đề tài.

7. Tài liệu trích dẫn

Tính trung thực trong việc trích dẫn tài liệu tham khảo; tính phù hợp của các tài liệu trích dẫn; trích dẫn theo đúng chỉ dẫn APA.

8. Đánh giá về sự trùng lặp của đề tài

Cần khẳng định đề tài có trùng lặp hay không? Nếu có, đề nghị ghi rõ mức độ, tên đề tài, nơi công bố, năm công bố của đề tài đã công bố.

9. Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa*

10. Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên

Đề nghị của giảng viên hướng dẫn

Ghi rõ: "Báo cáo đạt/ không đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép/ không được phép bảo vệ khóa luận tốt nghiệp"

Báo cáo đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép bảo vệ khóa luận tốt nghiệp

TP. HCM, ngày 15 tháng 6 năm 2026

Người nhận xét
(Ký và ghi rõ họ tên)

Nguyễn Văn Khoa

* Giảng viên hướng dẫn có thể ghi tiếp vào mặt sau

NHIỆM VỤ KHÓA LUẬN TỐT NGHIỆP

 TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH Ho Chi Minh City University of Technology and Engineering	CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM <u>Độc lập – Tự do – Hạnh phúc</u> TP. Hồ Chí Minh, ngày 23 tháng 6 năm 2026
--	--

Họ và tên sinh viên: Nguyễn Đình Thống – Lưu Phúc Khang

Mã số sinh viên: 22119236 – 22119188

Chuyên ngành: Công nghệ kỹ thuật máy tính

Lớp: 221192A

Giảng viên hướng dẫn: Th.S Nguyễn Ngô Lâm

Ngày nhận đề tài: 11/4/2026

Ngày nộp đề tài: 26/6/2026

- Tên đề tài:** Triển khai thiết kế và thi công hệ thống cảnh báo va chạm trên phương tiện điều khiển
- Các số liệu, tài liệu ban đầu:**
 - Kiến thức cơ bản về hệ thống máy tính, thiết bị điện tử từ các môn học Mạch điện, Điện tử cơ bản, Vi xử lý, Hệ điều hành thời gian thực...
 - Lý thuyết về lập trình phân cứng và cá giao thức giao tiếp có dây và không dây.
 - Tài liệu datasheet, user manual của các linh kiện điện tử, các máy tính nhúng được sử dụng trong đồ án.
 - Lập trình sử dụng ngôn ngữ C/C++, Python, Bash Script trên Windows, Linux
- Nội dung thực hiện đề tài:**
 - Thực hiện nghiên cứu các giao thức truyền dẫn dữ liệu.
 - Phân tích nguyên lý hoạt động, cấu tạo của các vi điều khiển, các module tương tác với môi trường được sử dụng trong đề tài.
 - Phát triển thuật toán, logic quản lý vận hành trên phương tiện
 - Thiết kế, xây dựng và kiểm tra phần cứng.
 - Thực hiện báo cáo.
- Sản phẩm**
 - Mô hình xe điều khiển (Sử dụng Raspberry pi 3B, STM32F407G-DISC, khung xe).
 - Quyển báo cáo đề tài.

GIẢNG VIÊN HƯỚNG DẪN

LỜI CẢM ƠN

Trước hết, chúng tôi xin trân trọng gửi lời cảm ơn trân thành đến Ban lãnh đạo Trường Đại học Công nghệ Kỹ thuật Thành phố Hồ Chí Minh cùng toàn thể cán bộ, các Thầy, Cô cùng bạn bè và gia đình đã hỗ trợ và giúp đỡ chúng tôi trong suốt quá trình học tập và thực hiện khóa luận tốt nghiệp. Nhờ sự nhiệt tình tạo điều kiện thuận lợi của mọi người mà chúng tôi có thể hoàn thành được đề tài đồ án tốt nghiệp này.

Chúng tôi xin gửi lời biết ơn sâu sắc đến các quý Thầy, Cô trong Khoa Điện - Điện tử. Nhận được sự giảng dạy, cung cấp kiến thức và hướng dẫn nhiệt tình của quý Thầy, Cô đã giúp chúng tôi nắm bắt được kiến thức nền tảng vững chắc để có thể thực hiện hóa được đồ án này. Ngoài ra, chúng tôi xin gửi lời cảm ơn chân thành nhất đến giảng viên hướng dẫn chúng tôi trong đề tài, Th.S Nguyễn Ngô Lâm, người đã tận tâm hướng dẫn, đề xuất hướng đi và tạo điều kiện thuận lợi, tận tình góp ý để chúng tôi xác định được các kiến trúc, phương pháp phù hợp để hoàn thành tốt các nhiệm vụ, yêu cầu được giao.

Mặt khác, chúng tôi cũng xin được gửi lời cảm ơn đến gia đình, người thân, bạn bè đã luôn bên cạnh và giúp đỡ chúng tôi trong suốt quá trình học tập và thực hiện đồ án. Chúng tôi cũng xin cảm ơn các tác giả, các nghiên cứu về lý thuyết, ứng dụng của liên quan đến đề tài mà chúng tôi thực hiện. Nhờ những nghiên cứu ấy, chúng tôi có được nguồn tài liệu tham khảo hữu ích trong quá trình nghiên cứu và hoàn thiện đề tài thuận lợi.

Liên quan đến đề tài được thực hiện trong đồ án tốt nghiệp này, do giới hạn về thời gian cũng như trình độ, kiến thức và kinh nghiệm của các cá nhân trong nhóm mà báo cáo của đề tài này không khỏi tồn tại những thiếu sót, chúng tôi mong nhận được sự góp ý, đánh giá quý báu của quý Thầy, Cô để báo cáo của chúng tôi được hoàn thiện hơn.

Trân trọng.

Nhóm thực hiện

TÓM TẮT

Trong các hệ thống hỗ trợ lái nâng cao trên ô tô hiện nay, chức năng phát hiện vật cản và cảnh báo va chạm được xem là một trong những tính năng quan trọng nhằm hỗ trợ người lái xử lý kịp thời các tình huống nguy hiểm. Xuất phát từ ý tưởng mô phỏng lại một phần nguyên lý hoạt động của hệ thống này ở quy mô mô hình, đề tài “Triển khai thiết kế và thi công hệ thống cảnh báo va chạm trên phương tiện điều khiển” được thực hiện nhằm mục đích mô phỏng lại hành vi của xây dựng một phương tiện có khả năng nhận biết vật cản xung quanh, xử lý dữ liệu đo được và đưa ra cảnh báo khi có nguy cơ va chạm.

Hệ thống sử dụng Raspberry Pi làm khối xử lý trung tâm, đảm nhiệm vai trò điều phối hoạt động, xử lý dữ liệu ở mức hệ thống và xây dựng giao diện hiển thị - điều khiển thông qua Web. Bên cạnh đó, vi điều khiển STM32F407 được dùng để xử lý các tác vụ liên quan đến cảm biến và điều khiển phần cứng. Module giao tiếp CAN MCP2515 được tích hợp nhằm hỗ trợ truyền dữ liệu giữa các khối xử lý, trong khi module GPS được sử dụng để cung cấp thông tin vị trí cho phương tiện. Phần chuyển động của mô hình được thực hiện thông qua các động cơ DC kết hợp với mạch điều khiển động cơ.

Sau quá trình thiết kế và thi công, hệ thống triển khai đã có thể đo khoảng cách đến vật cản, hiển thị trạng thái cảnh báo và hỗ trợ điều khiển chuyển động của phương tiện ở mức thử nghiệm, hoàn thành được cơ bản mục tiêu ban đầu đã đề ra. Bên cạnh đó thì việc triển khai vẫn còn gặp khá nhiều thách thức nằm ở việc tối ưu hóa cả phần mềm, việc đồng giao thức điều khiển cũng như thiết bị phần cứng thử nghiệm chưa hoạt động hiệu quả.

Nhóm nghiên cứu sẽ tiếp tục phát triển và mở rộng thêm các tính năng dành cho đề tài, để đảm bảo kết quả mô phỏng sát với thực tế nhất, tối ưu được những hạn chế hiện tại, mở rộng phạm vi linh kiện của thiết bị với nhiều hơn những ECU điều khiển và kiểm soát.

MỤC LỤC	
DANH MỤC BẢNG.....	VII
DANH MỤC HÌNH ẢNH.....	VIII
DANH MỤC TỪ VIẾT TẮT.....	X
CHƯƠNG 1 TỔNG QUAN ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Tính cấp thiết của đề tài	1
1.3 Mục tiêu của đề tài	2
1.4 Đối tượng nghiên cứu.....	2
1.5 Phạm vi nghiên cứu.....	3
1.6 Phương pháp nghiên cứu.....	3
1.7 Bố cục đồ án	4
CHƯƠNG 2 CƠ SỞ LÝ THUYẾT.....	5
2.1 TỔNG QUAN CƠ SỞ LÝ THUYẾT	5
2.2 MỘT SỐ LINH KIỆN VÀ MODULE TRONG HỆ THỐNG.....	5
2.2.1 Vi điều khiển quản lý và xử lý cảm biến STM32F407G-DISC1	5
2.2.2 Máy tính nhúng Raspberry Pi 3 Model B	9
2.2.3 Cảm biến khoảng cách hồng ngoại LIDAR07-100W-B.....	11
2.2.4 Cảm biến khoảng cách siêu âm HC-SR04.....	15
2.2.5 Động cơ DC	17
2.2.6 Mạch cầu H điều khiển động cơ TB6612FNG	18
2.7 ~ 5,5V.....	19
2.2.7 Mạch điều khiển giao tiếp CAN TJA1050	21
2.3 CÁC CHUẨN GIAO TIẾP VÀ GIAO THỨC TRUYỀN DỮ LIỆU.....	23

2.3.1	UART (Universal Asynchronous Receiver/Transmitter)	23
2.3.2	SPI (Serial Peripheral Interface)	24
2.3.3	I2C (Inter-Integrated Circuit).....	25
2.3.4	CAN (Controller Area Network)	26
2.3.5	Giao thức WebSockets.....	27
2.3.6	WI-FI (Wireless Fidelity)	28
2.4	TỔNG QUAN VỀ CÁC CÔNG CỤ HỖ TRỢ XÂY DỰNG VÀ LẬP TRÌNH CHO HỆ THỐNG.....	29
2.4.1	Môi trường lập trình nhúng (IDE - Integrated Development Environment).....	29
2.4.2	Phần mềm mô phỏng và thiết kế mạch điện tử.....	31
2.4.3	Phần mềm thiết kế cơ khí 3D (CAD).....	32
2.5	TỔNG QUAN VỀ THUẬT TOÁN ĐIỀU KHIỂN PHƯƠNG TIỆN	32
CHƯƠNG 3 THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG.....		34
3.1	YÊU CẦU VÀ SƠ ĐỒ KHỐI HỆ THỐNG	34
3.1.1	Yêu cầu của hệ thống.....	34
3.1.2	Sơ đồ khối hệ thống và các chức năng của các khối trong hệ thống ..	35
3.1.3	Hoạt động của hệ thống	37
3.2	THIẾT KẾ HỆ THỐNG.....	38
3.2.1	Khối cảm biến khoảng cách phía trước	38
3.2.2	Khối cảm biến khoảng cách phía trái, phía phải, phía sau	44
3.2.3	Khối quản lý và xử lý cảm biến	47
3.2.4	Khối xử lý trung tâm (Gateway).....	51
3.2.5	Khối hiển thị - điều khiển	55

3.2.6	Khối điều khiển động cơ.....	58
3.2.7	Khối nguồn.....	62
3.2.8	Sơ đồ nguyên lý toàn hệ thống	67
CHƯƠNG 4 THIẾT KẾ HỆ THỐNG PHẦN MỀM.....		69
4.1	Tổng quan về kiến trúc phần mềm hệ thống.....	69
4.1.1	Sơ đồ kiến trúc phần mềm hệ thống	69
4.2	Lập trình phần mềm trên Raspberry Pi 3B.....	70
4.3	Lập trình phần mềm trên STM32F407G-DISC	73
4.4	Lập trình giao tiếp CAN giữa Raspberry Pi và STM32.....	76
CHƯƠNG 5 KẾT QUẢ VÀ ĐÁNH GIÁ HỆ THỐNG		79
5.1	KẾT QUẢ THI CÔNG HỆ THỐNG.....	79
5.1.1	Kết quả xây dựng phần cứng	79
5.1.2	Kết quả xây dựng giao diện người dùng.....	80
5.2	ĐÁNH GIÁ ĐỘ ỔN ĐỊNH CỦA HỆ THỐNG THEO THỜI GIAN THỰC	83
CHƯƠNG 6 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....		84
6.1	Kết luận chung quá trình xây dựng đề tài	84
6.2	Hạn chế của đề tài	85
6.3	Định hướng phát triển hệ thống	85
PHỤ LỤC.....		87

DANH MỤC BẢNG

Bảng 2.1: Thông số kỹ thuật của STM32F407G-DISC1 Development Boards.....	6
Bảng 2.2: Thông số kỹ thuật cơ bản của Raspberry Pi 3 Model B.....	9
Bảng 2.3: Thông số kỹ thuật của cảm biến đo khoảng cách bằng hồng ngoại LIDAR07-100W-B	12
Bảng 2.4: Các chân kết nối của Lidar07-100W-B	13
Bảng 2.5: Thông số kỹ thuật HC-SR04	16
Bảng 2.6: Các chân kết nối của HC-SR04	17
Bảng 2.7: Thông số kỹ thuật của động cơ DC	18
Bảng 2.8: Kết nối chân LN298N	20
Bảng 2.9: Thông số kỹ thuật Mạch cầu H điều khiển động cơ TJA1050.....	22
Bảng 2.10: Kết nối chân của TJA1050	23

DANH MỤC HÌNH ẢNH

Hình 2.1: STM32F407G-DISC1 Development Boards.....	6
Hình 2.2: Sơ đồ mạch STM32F407G-DISC1	8
Hình 2.3: Raspberry Pi 3 Model B.....	9
Hình 2.4: Sơ đồ mạch Raspberry Pi 3B	11
Hình 2.5: Cảm biến đo khoảng cách bằng hồng ngoại LIDAR07-100W-B.....	11
Hình 2.6: Sơ đồ chân giao tiếp của LIDAR07-100W-B.....	13
Hình 2.7: Cảm biến khoảng cách HC-SR04	16
Hình 2.8: Sơ đồ chân giao tiếp của HC-SR04	17
Hình 2.9: Động cơ DC	18
Hình 2.10: Mạch cầu H điều khiển động cơ TB6612FNG	19
Hình 2.11: Sơ đồ nguyên lý mạch cầu H điều khiển động cơ TB6612FNG	20
Hình 2.12: Sơ đồ chân giao tiếp của mạch cầu H TB6612FNG.....	20
Hình 2.13: Mạch điều khiển giao tiếp CAN TJA1050	21
Hình 2.14: Sơ đồ nguyên lý của TJA1050.....	22
Hình 2.15: Sơ đồ chân giao tiếp của mạch giao tiếp CAN TJA1050	23
Hình 2.16: Chuẩn giao tiếp UART	24
Hình 2.17: Chuẩn giao tiếp SPI	25
Hình 2.18: Chuẩn giao tiếp I2C	26
Hình 2.19: Chuẩn giao tiếp CAN.....	27
Hình 2.20: Mô tả giao thức WebSocket.....	28
Hình 2.21: Minh họa việc giao tiếp không dây thông qua Wifi	29
Hình 2.22: Trình soạn thảo văn bản Visual Studio Code	30
Hình 2.23: Giao diện công cụ cấu hình vi điều khiển STM32CubeMX.....	31
Hình 3.1: Sơ đồ khối hệ thống	36
Hình 3.2: Sơ đồ nguyên lý khối cảm biến khoảng cách phía trước	40
Hình 3.3: Sơ đồ xử lý luồng dữ liệu gửi từ STM32F407G-DISC xuống Lidar07-100W-B	42

Hình 3.4: Sơ đồ luồng xử lý dữ liệu từ Lidar07-100W-B lên STM32F407G-DISC	43
Hình 3.5: Sơ đồ nguyên lý khối cảm biến khoảng cách phía trái, phải, sau	46
Hình 3.6: Sơ đồ nguyên lý khối quản lý và xử lý cảm biến.....	49
Hình 3.7: Luồng xử lý thuật toán của khối quản lý và xử lý cảm biến.....	51
Hình 3.8: Sơ đồ nguyên lý khối xử lý trung tâm	54
Hình 3.9: Sơ đồ nguyên lý khối điều khiển động cơ	61
Hình 3.10: Sơ đồ nguyên lý khối nguồn	64
Hình 3.11: Sơ đồ kết nối 2 cell pin với mạch bảo vệ BMS	65
Hình 3.12: Sơ đồ nguyên lý của toàn hệ thống	68
Hình 4.1: Tổng quan hệ thống phần mềm trên Raspberry Pi 3B	71
Hình 4.2: Lưu đồ thuật toán xử lý điều khiển người dùng	72
Hình 4.3: Lưu đồ giải thuật điều khiển TB6612FNG	73
Hình 4.4: Lưu đồ giải thuật Task Lidar	74
Hình 4.5: Lưu đồ giả thuật Task US	75
Hình 4.6: Lưu đồ giải thuật giao tiếp CAN trên STM323	76
Hình 4.7: Lưu đồ giải thuật giao tiếp CAN trên Raspberry	77
Hình 5.1: Kết quả thi công phần cứng	80
Hình 5.2: Giao diện người dùng trên điện thoại di động	81
Hình 5.3: Giao diện người dùng hiển thị CAN bị ngắt.....	82
Hình 5.4: Giao diện người dùng hiển thị khoảng cách phía sau vượt ngưỡng	82

DANH MỤC TỪ VIẾT TẮT

Viết tắt	Tên đầy đủ	Ý nghĩa
ADAS	Advanced Driver Assistance Systems	Hệ thống hỗ trợ lái xe nâng cao (cảnh báo va chạm, an toàn chủ động)
CWS	Collision Warning System	Hệ thống cảnh báo va chạm
TTC	Time To Collision	Thời gian dự đoán sẽ xảy ra va chạm
ECU	Electronic Control Unit	Bộ điều khiển điện tử trong hệ thống
Tele-operation	-	Điều khiển phương tiện từ xa
Gateway	-	Khối trung gian xử lý và giao tiếp toàn hệ thống
IoT	Internet of Things	Mạng các thiết bị kết nối Internet
MCU	Microcontroller Unit	Vi điều khiển
STM32	STMicroelectronics 32-bit MCU	Dòng vi điều khiển ARM Cortex
ARM	Advanced RISC Machine	Kiến trúc CPU
GPIO	General Purpose Input/Output	Chân vào/ra đa dụng
CPU	Central Processing Unit	Bộ xử lý trung tâm
RAM	Random Access Memory	Bộ nhớ truy cập ngẫu nhiên
Flash	-	Bộ nhớ lưu trữ chương trình
ADC	Analog to Digital Converter	Chuyển đổi analog → digital
DAC	Digital to Analog Converter	Chuyển đổi digital → analog
PWM	Pulse Width Modulation	Điều chế độ rộng xung
MEMS	Micro Electro Mechanical Systems	Cảm biến vi cơ (ví dụ accelerometer)
CAN	Controller Area Network	Mạng truyền thông trong ô tô
UART	Universal Asynchronous Receiver/Transmitter	Giao tiếp nối tiếp không đồng bộ
SPI	Serial Peripheral Interface	Giao tiếp nối tiếp đồng bộ tốc độ cao
I2C	Inter-Integrated Circuit	Giao tiếp 2 dây
Wi-Fi	Wireless Fidelity	Mạng không dây

TCP	Transmission Control Protocol	Giao thức truyền tải
UDP	User Datagram Protocol	Giao thức truyền nhanh, không đảm bảo
WebSocket	-	Giao tiếp realtime 2 chiều qua TCP
LAN	Local Area Network	Mạng cục bộ
TX	Transmit	Truyền dữ liệu
RX	Receive	Nhận dữ liệu
MOSI	Master Out Slave In	Dữ liệu Master → Slave (SPI)
MISO	Master In Slave Out	Dữ liệu Slave → Master
SCK	Serial Clock	Xung clock (SPI/I2C)
SDA	Serial Data	Dữ liệu I2C
SCL	Serial Clock	Clock I2C
CS/SS	Chip Select / Slave Select	Chọn thiết bị SPI
ACK	Acknowledge	Xác nhận
NACK	Not Acknowledge	Không xác nhận
CRC	Cyclic Redundancy Check	Kiểm tra lỗi dữ liệu
LIDAR	Light Detection And Ranging	Cảm biến đo khoảng cách bằng laser
ToF	Time of Flight	Đo khoảng cách theo thời gian bay
HC-SR04	-	Cảm biến siêu âm đo khoảng cách
VCC	Voltage Common Collector	Nguồn cấp dương
GND	Ground	Mass (0V)
IDE	Integrated Development Environment	Môi trường phát triển
VS Code	Visual Studio Code	Trình soạn thảo code
HAL	Hardware Abstraction Layer	Lớp trừu tượng phần cứng
RTOS	Real-Time Operating System	Hệ điều hành thời gian thực
UI	User Interface	Giao diện người dùng
RT	Real-time	Thời gian thực
DC	Direct Current	Dòng điện một chiều
BMS	Battery Management System	Quản lý pin

CHƯƠNG 1 TỔNG QUAN ĐỀ TÀI

1.1 Đặt vấn đề

Trong bối cảnh các ngành công nghiệp kỹ thuật nói chung và nền công nghiệp ô tô nói riêng đang phát triển vô cùng thần tốc. Các cuộc chạy đua công nghệ diễn ra vô cùng quyết liệt, các tính năng, ứng dụng mới ngày càng nhiều. Song song với đó, khi công nghệ đã và đang phát triển vượt bậc thì các nhà sản xuất, người tiêu dùng càng quan tâm hơn về sự bảo mật, an toàn. Nền công nghiệp ô tô hiện tại đang dần chuyển mình mạnh mẽ sang các dòng xe thông minh và tự hành, để có thể đi cùng điều đó thì vấn đề an toàn giao thông và hỗ trợ người lái cũng trở thành ưu tiên hàng đầu. Hệ thống hỗ trợ người lái nâng cao (ADAS - Advanced Driver Assistance Systems), đặc biệt là tính năng cảnh báo va chạm, đóng vai trò then chốt trong việc giảm thiểu tai nạn.

Để các hệ thống này hoạt động ổn định và có độ trễ thấp nhất trong môi trường nhiều nhiễu điện từ như ô tô, chuẩn giao tiếp CAN (Controller Area Network) được sử dụng như một tiêu chuẩn công nghiệp bắt buộc. Hơn nữa, với sự phát triển của IoT và viễn thông, khái niệm vận hành xe từ xa (Tele-operation) đang trở thành một xu hướng nghiên cứu mới, cho phép giám sát và can thiệp vào quá trình di chuyển của xe thông qua trạm điều khiển mặt đất.

Xuất phát từ nhu cầu thực tiễn đó, việc nghiên cứu, thiết kế và thi công một mô hình xe cảnh báo va chạm sử dụng mạng CAN, kết hợp với trạm giám sát từ xa là một thách thức kỹ thuật bao hàm nhiều lĩnh vực: cơ khí, điện tử nhúng và phần mềm. Do đó, đề tài "Triển khai thiết kế và thi công hệ thống cảnh báo va chạm trên phương tiện điều khiển" được nhóm nghiên cứu lựa chọn để thực hiện.

1.2 Tính cấp thiết của đề tài

Tai nạn giao thông do không giữ khoảng cách an toàn và hạn chế tầm nhìn luôn là vấn đề nhức nhối. Tại các môi trường khắc nghiệt hoặc khu vực hạn chế con người

tiếp cận, nhu cầu sử dụng các phương tiện được điều khiển từ xa có khả năng tự động nhận diện vật cản là cực kỳ lớn.

Trong thực tế kỹ thuật, để các cảm biến có thể gửi dữ liệu khẩn cấp về bộ xử lý trung tâm mà không bị nhiễu sóng hoặc trễ tín hiệu, mạng giao tiếp CAN Bus là một giải pháp cấp thiết và bắt buộc trong chuẩn công nghiệp ô tô. Do đó, việc nghiên cứu ứng dụng giao thức CAN kết hợp với công nghệ điều khiển thời gian thực qua giao thức WebSockets không chỉ giải quyết bài toán mô phỏng đồ án mà còn bám sát các yêu cầu thực tiễn khắt khe của ngành công nghiệp tự động hóa phương tiện hiện nay.

1.3 Mục tiêu của đề tài

Đề tài hướng tới việc thiết kế và hoàn thiện một mô hình xe điều khiển từ xa, tích hợp các tính năng an toàn tự động với các mục tiêu cụ thể:

- Thiết kế khung gầm xe nhằm đáp ứng cấu trúc phân tầng vi mạch và cơ cấu lái thực tế.
- Xây dựng hệ thống thu thập dữ liệu khoảng cách đa hướng bằng các cảm biến đo khoảng cách, xử lý trên vi điều khiển STM32.
- Thiết lập mạng truyền thông CAN Bus để truyền tải các gói dữ liệu cảnh báo va chạm giữa các node điều khiển.
- Xây dựng trạm điều khiển mặt đất bằng Raspberry Pi, sử dụng giao thức WebSockets để hiển thị thông số và điều khiển xe từ xa.
- Lập trình cơ chế an toàn (Fail-safe), cho phép hệ thống trên xe tự động phanh (ghì dè lệnh điều khiển từ xa) khi phát hiện nguy cơ va chạm khẩn cấp.

1.4 Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài bao gồm các thành phần phần cứng và phần mềm cốt lõi:

- Về phần cứng: Kiến trúc vi điều khiển lõi ARM Cortex-M3 (STM32F407G-DISC), máy tính nhúng Raspberry Pi 3B, IC giao tiếp CAN (MCP2515, TJA1050), cảm biến siêu âm HC-SR04 và cấu trúc mạch cầu H điều khiển động cơ (TB6612FNG).
- Về phần mềm: Ngôn ngữ lập trình C/C++ dùng để viết firmware cho khối vi điều khiển nhúng. Môi trường Linux trên Raspberry Pi để cấu hình máy chủ WebSockets, xử lý giao diện hiển thị đồ họa.

1.5 Phạm vi nghiên cứu

- Mô hình vật lý: Xe được thiết kế ở tỷ lệ thu nhỏ (1:10), tốc độ giới hạn phù hợp với việc thử nghiệm trong không gian hẹp.
- Thuật toán cảnh báo: Hệ thống xử lý dữ liệu cảnh báo va chạm ở mức cơ bản (tính toán dựa trên ngưỡng khoảng cách ((Time-to-Collision tĩnh), chưa áp dụng các mô hình học sâu (Deep Learning) hay xử lý ảnh phức tạp (Computer Vision) để nhận diện đối tượng.
- Truyền thông: Mạng CAN giới hạn ở chuẩn CAN 2.0B tốc độ cao. Giao tiếp điều khiển từ xa phụ thuộc vào vùng phủ sóng của mạng Wi-Fi cục bộ được cấp phát cho hệ thống WebSockets.

1.6 Phương pháp nghiên cứu

Đề tài áp dụng các phương pháp nghiên cứu kết hợp giữa lý thuyết và thực nghiệm:

- Nghiên cứu lý thuyết: Tổng hợp các tài liệu kỹ thuật của các vi mạch, các giao thức truyền thông (CAN Bus, WebSockets, UART, I2C). Tham khảo cấu trúc cơ khí khung gầm điều khiển.
- Phương pháp mô phỏng: Sử dụng phần mềm Proteus để mô phỏng sơ đồ nguyên lý (Schematic) và kiểm tra logic hoạt động của các ngoại vi vi điều khiển trước khi thi công mạch thực tế. Sử dụng phần mềm thiết kế 3D để tạo bộ khung vỏ của hệ thống.

- Phương pháp kiểm định: Lắp ráp phần cứng, tiến hành nạp code (Firmware). Thực hiện đo đạc, thu thập các giá trị của cảm biến trong các kịch bản môi trường có vật cản. Đánh giá độ trễ của lệnh điều khiển từ xa qua giao diện Raspberry Pi và hiệu chỉnh hệ số vòng lặp điều khiển cho phù hợp.

1.7 Bố cục đồ án

Đồ án được trình bày với cấu trúc gồm các chương như sau:

- Chương 1: Tổng quan về đề tài. Trình bày lý do chọn đề tài, mục tiêu, đối tượng, phạm vi và phương pháp thực hiện.
- Chương 2: Cơ sở lý thuyết. Cung cấp nền tảng kiến thức về hệ thống ADAS, mạng giao thức CAN Bus, cấu trúc giao tiếp WebSockets và tổng quan về các linh kiện phần cứng được sử dụng.
- Chương 3: Thiết kế và thi công hệ thống. Trình bày chi tiết sơ đồ khối, quá trình thiết kế phần mềm, cấu trúc sơ đồ nguyên lý (Schematic) mạch điện và bản vẽ của xe mô hình.
- Chương 4: Thiết kế phần mềm. Mô tả chi tiết thuật toán điều khiển của vi điều khiển STM32, quy trình gửi tin nhắn cảnh báo và cấu trúc luồng điều khiển của máy chủ Raspberry Pi.
- Chương 5: Kết quả thực nghiệm và Đánh giá. Trình bày các hình ảnh thi công thực tế, số liệu đo đạc, độ trễ và đánh giá mức độ hoàn thành so với mục tiêu đề ra ban đầu. Đưa ra hướng phát triển trong tương lai.

CHƯƠNG 2 CƠ SỞ LÝ THUYẾT

2.1 TỔNG QUAN CƠ SỞ LÝ THUYẾT

Hệ thống điều khiển giám sát từ xa tích hợp cảnh báo va chạm là sự kết hợp giữa hai lĩnh vực cốt lõi trong kỹ thuật ô tô hiện đại: Hệ thống Hỗ trợ Người lái Nâng cao (ADAS) và công nghệ Viễn thông (Tele-operation).

- **Hệ thống cảnh báo va chạm (Collision Warning System - CWS):** Thuộc nhóm an toàn chủ động (Active Safety) trong hệ thống ADAS. Nguyên lý cơ bản dựa trên việc liên tục thu thập dữ liệu về khoảng cách không gian xung quanh xe thông qua các cảm biến đo lường (như siêu âm, Radar, Lidar). Hệ thống vi xử lý sẽ liên tục tính toán thông số thời gian dự kiến xảy ra va chạm (TTC - Time To Collision). Nếu khoảng cách an toàn bị vi phạm, hệ thống sẽ đưa ra các tín hiệu cảnh báo hoặc tự động kích hoạt cơ chế phanh khẩn cấp để tránh va chạm.
- **Tele-operation (Vận hành từ xa):** Là khái niệm điều khiển và giám sát phương tiện không người lái thông qua một trạm kiểm soát mặt đất (Ground Control Station). Khác với điều khiển vô tuyến truyền thống (RC), Tele-operation đòi hỏi một kênh truyền dữ liệu song công (Full-duplex), cho phép người vận hành không chỉ gửi lệnh điều khiển mượt mà (Steering, Throttle) mà còn nhận lại được các dữ liệu Telemetry theo thời gian thực (vị trí GPS, trạng thái cảm biến, tình trạng kết nối mạng).

2.2 MỘT SỐ LINH KIỆN VÀ MODULE TRONG HỆ THỐNG

2.2.1 Vi điều khiển quản lý và xử lý cảm biến STM32F407G-DISC1

STM32F407G-DISC1 là bộ vi xử lý nhúng 32-bit dựa trên kiến trúc lõi ARM Cortex-4, có xung nhịp hoạt động tối đa 168MHz. Trong hệ thống phân tán của xe, STM32 đóng vai trò là ECU (trung tâm điều khiển. Với khả năng xử lý ngắt thời gian

thực mạnh mẽ, bộ đếm thời gian độ phân giải cao, và đặc biệt là bộ điều khiển phần cứng CAN 2.0B được tích hợp sẵn, STM32 đảm bảo việc đọc dữ liệu từ các cảm biến siêu âm và truyền lên mạng CAN mà không gặp hiện tượng trễ nhịp.



Hình 2.1: STM32F407G-DISC1 Development Boards

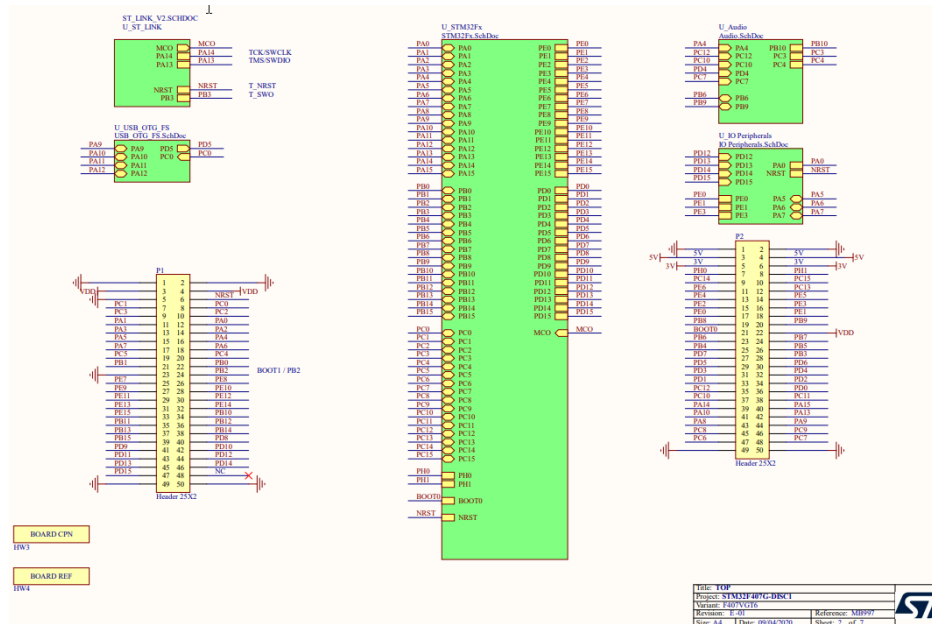
- Thông số kĩ thuật của STM32F407G-DISC1 Development Boards:

Thông số kĩ thuật của STM32F407G-DISC1 Development Boards được cung cấp trong bảng 2.1:

Bảng 2.1: Thông số kĩ thuật của STM32F407G-DISC1 Development Boards

STT	Thành phần	Đặc điểm
1	Bộ xử lý (CPU)	Board sử dụng vi điều khiển STM32F407VGT6 có lõi xử lý ARM Cortex-M4 32-bit hoạt động ở xung nhịp tối đa đạt 168MHz
2	Bộ nhớ	Board được trang bị bộ nhớ lớn gồm: - 1MB Flash memory - 192KB SRAM

3	Bộ nạp và gỡ lỗi (Debugger/Programmer)	Board hỗ trợ mạch nạp tích hợp ST-LINK/V2-A giúp nạp code và debug trực tiếp mà không thông qua mạch nạp ngoài
4	Cảm biến chuyển động	Board sử dụng cảm biến gia tốc MEMS 3 trục (3-axis accelerometer).
5	LEDs tích hợp	Mạch có tổng cộng 8 LEDs giúp hỗ trợ giao diện người dùng, trong đó: - 4 LEDs có thể lập trình được (LD3-Cam, LD4-Xanh lá, LD5-Đỏ, LD6-Xanh dương) - 1 LEDs nguồn (LD2-Đỏ) - 1 LEDs USB (LD1-Đỏ/Xanh lá) báo trạng thái giao tiếp của mạch nạp ST-Link - 2 LEDs USB OTG (LD7-Xanh lá và LD8-Đỏ)
6	Nút nhấn tích hợp	Mạch cũng có 2 nút nhấn vật lý: - Nút Reset (Đen): Dừng khi khởi động lại vi điều khiển - Nút User (Xanh dương): Nút nhấn cơ bản có thể lập trình theo ý muốn
7	GPIOs	Board hỗ trợ 82 PIN GPIOs có thể lập trình được, các PIN hầu hết đều được thiết kế chịu đựng 5V đầu vào.
8	Nguồn	Mạch cho phép cấp nguồn 5V trực tiếp từ cổng Mini-USB của ST-Link. Chuẩn mức logic: Phần lớn các GPIO cho phép hoạt động ở mức 5V



Hình 2.2: Sơ đồ mạch STM32F407G-DISC1

- GPIO và các giao tiếp hỗ trợ của STM32F407G-DISC1 Development Boards:

Vì điều khiển STM32F407VGT6 cung cấp số lượng I/O rất lớn, các chân này được kết nối với 2 dãy header ở 2 bên của board, từ đó cho ra 82 GPIO có thể lập trình được, mang lại khả năng mở rộng lớn cho hệ thống.

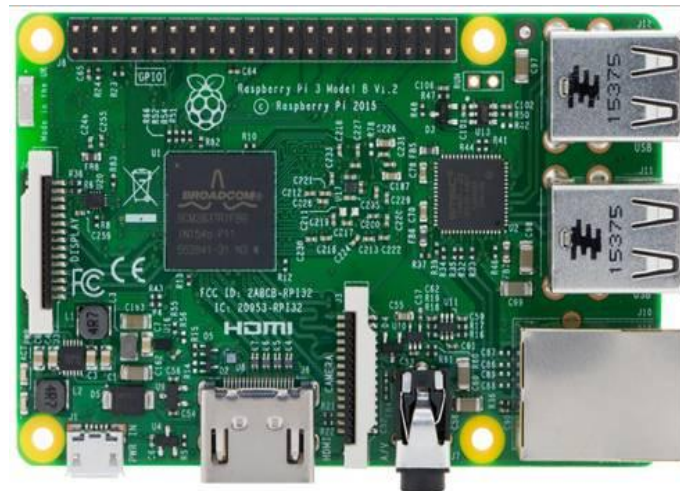
Các GPIOs được cấu hình để có thể thực hiện luân phiên các chức năng giao tiếp phân cứng như:

- I2C (Iner-Integrated Circuit): Đây là chuẩn giao tiếp đồng bộ chỉ sử dụng 2 dây kết nối SDA (dữ liệu) và SCL (clock). Mạch cung cấp tối đa 3 bộ tùy theo phiên bản của board.
- UART (Universal Asynchronous/Synchronous Receiver-Transmitter): Đây là giao tiếp bất đồng bộ sử dụng 2 dây TX và RX. Mạch được tích hợp 2 bộ UART tự cấu hình.
- SPI (Serial Peripheral Interface): Mạch tích hợp 3 bộ SPI giao tiếp đồng bộ sử dụng 4 dây kết nối MOSI-MISO-SCK-CS.

- 3 bộ chuyển đổi ADC 12-bit và 2 bộ chuyển đổi DAC 12-bit với tốc độ lấy mẫu đạt 2.4 MSPS
- 17 bộ định thời (Timers) 16-bit và 32-bit.

2.2.2 Máy tính nhúng Raspberry Pi 3 Model B

Raspberry Pi 3B là một hệ thống trên một vi mạch sở hữu vi xử lý kiến trúc ARMv8, chạy hệ điều hành Linux. Thay vì tham gia vào việc xử lý tín hiệu phần cứng ở mức thấp, Raspberry Pi đảm nhận các tác vụ tính toán mức cao, bao gồm: vận hành máy chủ truyền thông mạng, quản lý luồng dữ liệu điều khiển từ xa và hiển thị giao diện đồ họa giám sát hệ thống.



Hình 2.3: Raspberry Pi 3 Model B

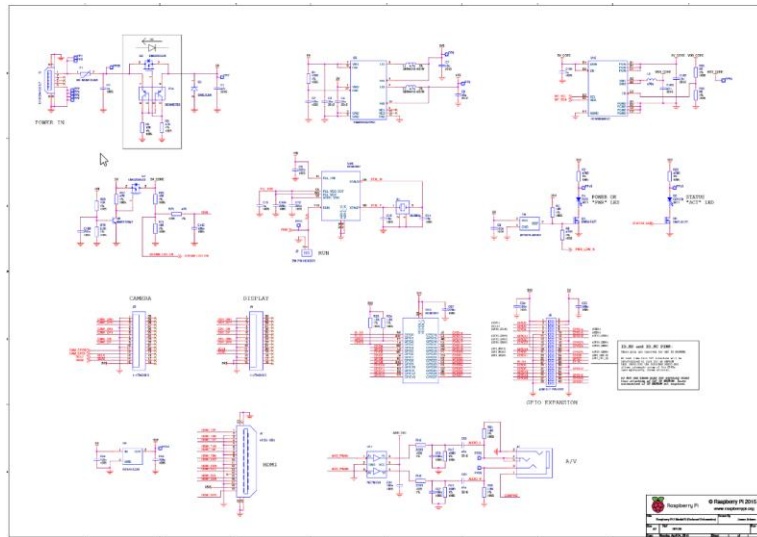
- Thông số kỹ thuật của Raspberry Pi 3 Model B:

Thông số kỹ thuật của Raspberry Pi 3 Model B được cung cấp trong bảng:

Bảng 2.2: Thông số kỹ thuật cơ bản của Raspberry Pi 3 Model B

STT	Thành phần	Đặc điểm
-----	------------	----------

1	Bộ xử lý (CPU)	Board xử dụng vi điều khiển Broadcom BCM2387 có lõi xử lý Quad-Core ARM Cortex-A53 32-bit hoạt động ở xung nhịp 1.2GHz, tích hợp xử lý Wireless LAN 802.11 b/g/n và Bluetooth 4.1
2	Bộ nhớ	Board được trang bị bộ nhớ RAM lớn 1GB LPDDR2, trong khi cung cấp khe cắm thẻ Micro SD cho phép boot các hệ điều hành và lưu trữ dữ liệu
3	GPU	Board hỗ trợ vi xử lý Dual Core VideoCore IV Multimedia Co-Processor
4	Kết nối	Board tích hợp 4 cổng USB 2.0
5	GPIOs	Board hỗ trợ 40 PIN GPIO, được hỗ trợ đầy đủ các giao thức như I2C, SPI, UART... trong đó có 27 GPIO có thể lập trình được.
6	Nguồn	Cấp nguồn qua cổng Micro USB với thông số 5V - 2.5A.



Hình 2.4: Sơ đồ mạch Raspberry Pi 3B

2.2.3 Cảm biến khoảng cách hồng ngoại LIDAR07-100W-B



Hình 2.5: Cảm biến đo khoảng cách bằng hồng ngoại LIDAR07-100W-B

- Thông số kĩ thuật của cảm biến đo khoảng cách bằng hồng ngoại LIDAR07-100W-B:

Thông số kỹ thuật của cảm biến đo khoảng cách bằng hồng ngoại LIDAR07-100W-B được cung cấp trong bảng 2.3 dưới đây:

Bảng 2.3: Thông số kỹ thuật của cảm biến đo khoảng cách bằng hồng ngoại LIDAR07-100W-B

STT	Thành phần	Đặc điểm
1	Điện áp cung cấp	DC 4.8V – 5.2V
2	Phạm vi đo	0.2m – 12m (trong nhà /phản xạ 90%)
3	Độ phân giải khoảng cách	1mm
4	Chế độ hoạt động	Chế độ liên tục, chế độ đơn
5	Giao tiếp	UART, I2C
6	Frame rate	Tối đa 500Hz
7	Nhiệt độ hoạt động	-10°C – 60°C

- Phương pháp đo đặc khoảng cách của LIDAR07-100W-B:

Cảm biến LIDAR07-100W-B hoạt động dựa trên nguyên lý quang học ToF (Time of Flight – Thời gian bay). Theo đó khoảng cách được tính toán dựa trên công thức:

$$d = \frac{c \times \Delta t}{2}$$

Trong đó:

d : Khoảng cách thực tế từ cảm biến đến bề mặt vật thể (m)

c : Vận tốc ánh sáng trong môi trường không khí (khoảng 3.10^8 m/s)

Δt : Tổng thời gian từ khoảnh khắc diot hồng ngoại (TX) phát ra xung ánh sáng cho đến khi diot thu (RX) nhận được tia phản xạ quay về.

- Sơ đồ chân kết nối của LIDAR07-100W-B:

Sơ đồ chân kết nối được trình bày trong ảnh và bảng dưới đây:



Hình 2.6: Sơ đồ chân giao tiếp của LIDAR07-100W-B

Bảng 2.4: Các chân kết nối của Lidar07-100W-B

STT	Thành phần	Đặc điểm
1	SCL	Chân cấp xung nhịp cho giao tiếp I2C.
2	SDA	Chân truyền/nhận dữ liệu cho giao tiếp I2C.
3	GND_LED	Chân mass (GND) riêng cho đèn LED hồng ngoại.
4	VCC_LED	Cấp nguồn riêng cho đèn LED hồng ngoại.
5	TX	Chân phát dữ liệu của UART.
6	RX	Chân phát dữ liệu của UART.
7	GND	Chân mass (GND) cho bo mạch xử lý chính.
8	VCC	Cấp nguồn cho bo mạch xử lý chính.

Trong hệ thống này, module LIDAR07-100W-B được kết nối với khối quản lý và xử lý cảm biến thông qua sử dụng chuẩn giao tiếp I2C. Theo đó, các chân VCC, VCC_LED, GND, GND_LED của lidar được kết nối chung lần lượt với VCC +5V nguồn và GND nguồn với STM32F407G-DISC1 Development board. Giao tiếp I2C

giữa board và cảm biến được thiết lập bằng cách kết nối chân SCL của lidar vào PA07 của board, trong khi SDA của lidar được kết nối với GPIO PB08.

- **Mô tả hoạt động giao tiếp giữa khối quản lý và xử lý cảm biến với khối cảm biến khoảng cách phía trước:**

Trong chế độ truyền sử dụng I2C, board đóng vai trò là Master trong khi cảm biến Lidar sẽ là Slave. Theo đó, giao tiếp giữa vi điều khiển và cảm biến hoạt động trình tự theo quá trình “hỏi-đáp” như sau:

1. **Tạo tín hiệu bắt đầu (Start condition):** STM32 tiến hành kéo SDA xuống mức thấp (LOW) trong khi SCL vẫn ở mức cao (HIGH) để báo hiệu cho thiết bị trên bus chuẩn bị nhận tín hiệu
2. **Gửi địa chỉ Slave và lệnh ghi (Write):** STM32 gửi địa chỉ I2C của Lidar (0x70) kèm theo bit Write (bit 0). Lidar khi nhận được tín hiệu sẽ trả về xung ACK.
3. **Chỉ định Thanh ghi (Register Address):** STM32 gửi tiếp địa chỉ của thanh ghi bên trong bộ nhớ LiDAR07 chứa dữ liệu khoảng cách (Distance Register). Lidar tiếp tục trả lời ACK.
4. **Tạo lại tín hiệu bắt đầu:** STM32 gửi lệnh stop và tiếp tục gửi lệnh start cho quá trình đọc dữ liệu
5. **Gửi địa chỉ Slave và lệnh đọc (Read):** Lần này, STM32 gửi địa chỉ của Lidar kèm theo bit Read (bit 1) để Lidar thực hiện gửi kết quả về.
6. **Đọc dữ liệu (Data Read):** Khi này Lidar bắt đầu gửi các bytes dữ liệu (Distance High Byte, Distance Low Byte, Signal Amplitude..) theo mỗi xung clock từ STM32. Sau mỗi byte nhận được, STM32 phải phải hỏi ACK để Lidar gửi byte tiếp theo
7. **Kết thúc (Stop condition):** Khi đã đọc đủ số byte, board phải hỏi bằng NACK để giải phóng bus và ngừng quá trình đọc dữ liệu.

Trong đó, data frame dữ liệu gửi về từ cảm biến bao gồm: 16-bit giá trị khoảng cách (Distance), 16-bit giá trị biên độ tín hiệu (Signal Amplitude), 16-bit giá trị ánh sáng nền (Ambient Light), 8-bit mã trạng thái (Status Code).

2.2.4 Cảm biến khoảng cách siêu âm HC-SR04

Cảm biến siêu âm hoạt động dựa trên nguyên lý Time-of-Flight (ToF). Module phát ra các chuỗi xung sóng âm ở tần số 40kHz. Khi gặp vật cản, sóng âm phản xạ lại và được module thu nhận. Khoảng cách d được vi điều khiển tính toán thông qua thời gian truyền tín hiệu t và vận tốc âm thanh trong không khí $v \simeq 340 \text{ m/s}$ theo công thức

$$d = \frac{v \times t}{2}$$

Trong đó:

d : Khoảng cách thực tế từ cảm biến đến bề mặt vật thể (m)

v : Vận tốc ánh sáng trong môi trường không khí (khoảng 340 m/s)

Δt : Tổng thời gian từ khoảnh khắc diot hồng ngoại (TX) phát ra xung ánh sáng cho đến khi diot thu (RX) nhận được tia phản xạ quay về.



Hình 2.7: Cảm biến khoảng cách HC-SR04

- **Thông số kĩ thuật của cảm biến đo khoảng cách bằng âm thanh HC-SR04:**

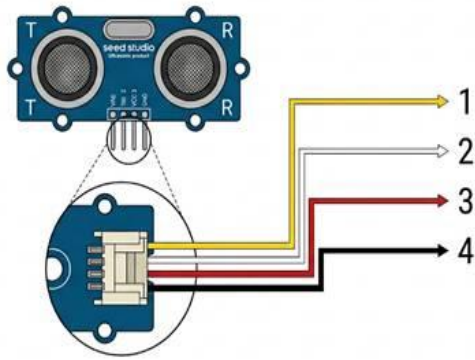
Thông số kĩ thuật của cảm biến đo khoảng cách bằng âm thanh HC-SR04 được cung cấp trong bảng dưới đây:

Bảng 2.5: Thông số kĩ thuật HC-SR04

STT	Thành phần	Đặc điểm
1	Nguồn cung cấp	3.2V – 5.3V
2	Tần số hoạt động	40kHz
3	Khoảng cách đo đạc	2cm - 350cm
4	Độ phân giải khoảng cách	1cm
5	Giao tiếp	Không sử dụng chuẩn giao tiếp

- **Sơ đồ chân kết nối của HC-SR04:**

Sơ đồ chân kết nối được trình bày trong ảnh và bảng dưới đây:



Hình 2.8: Sơ đồ chân giao tiếp của HC-SR04

Bảng 2.6: Các chân kết nối của HC-SR04

STT	Thành phần	Đặc điểm
1	SIG	Chân tín hiệu giao tiếp.
2	NC	Không kết nối
3	VCC	Cấp nguồn cho module
4	GND	Nối mass module với hệ thống.

2.2.5 Động cơ DC

Động cơ một chiều (DC) là một loại động cơ cơ bản có khả năng biến đổi dòng điện một chiều thành cơ năng dưới dạng chuyển động quay. Nhờ cấu tạo cơ bản, động cơ DC cho phép điều khiển tốc độ và chiều quay dễ dàng, được sử dụng rộng rãi trong các mô hình xe tự hành, robot di động hay các hệ thống truyền động kích thước vừa và nhỏ.



Hình 2.9: Động cơ DC

Trong đề tài này, động cơ DC được sử dụng làm cơ cấu mô phỏng chuyển động của phương tiện. Tốc độ và chiều quay của động cơ được điều khiển bởi vi điều khiển đảm nhận driver điều khiển động cơ thông qua mạch cầu H TB6612FNG.

- Thông số kĩ thuật của động cơ DC:

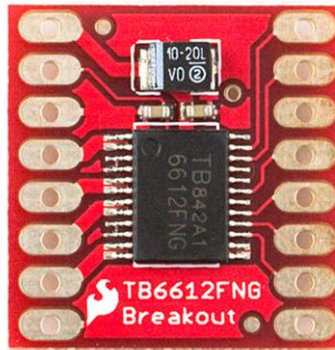
Thông số kĩ thuật của động cơ DC được cung cấp trong bảng dưới đây:

Bảng 2.7: Thông số kĩ thuật của động cơ DC

STT	Thành phần	Đặc điểm
1	Nguồn cung cấp	3.2V – 5.3V
2	Mô men xoắn	1 – 5 kg·cm
3	Chiều quay	Cho phép xoay theo 2 chiều
4	Phương pháp điều khiển	PWM

2.2.6 Mạch cầu H điều khiển động cơ TB6612FNG

Mạch cầu H điều khiển động cơ TB6612FNG sử dụng IC điều khiển động cơ tích hợp hai mạch cầu H (Dual Full-Bridge Driver), cho phép điều khiển đồng thời hai động cơ DC hoặc một động cơ bước hai pha.



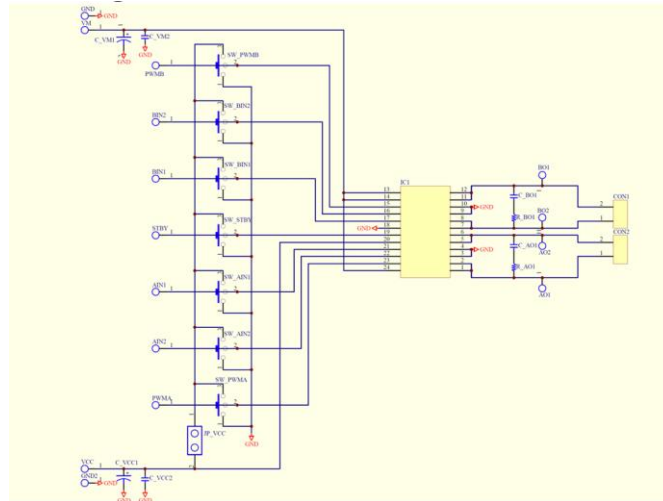
Hình 2.10: Mạch cầu H điều khiển động cơ TB6612FNG

Trong đề tài, mạch cầu H TB6612FNG được sử dụng đóng vai trò làm trung gian điều khiển giữa động cơ DC và vi điều khiển động cơ Raspberry Pi 3B.

- **Thông số kỹ thuật của TB6612FNG:**

Thông số kỹ thuật Mạch cầu H điều khiển động cơ TB6612FNG được cung cấp trong bảng dưới đây:

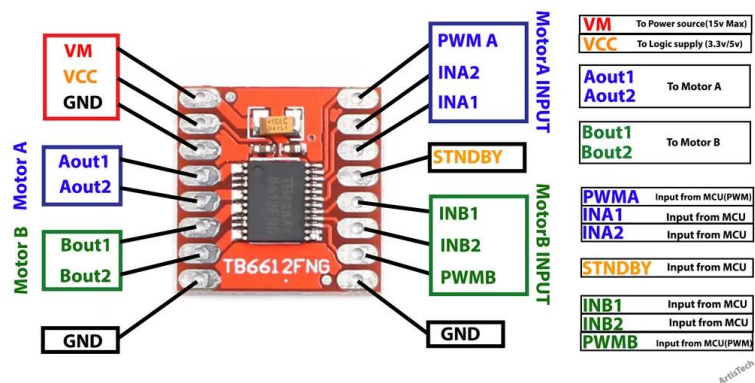
STT	Thành phần	Đặc điểm
1	Nguồn cung cấp	+5V/+12V
2	Điện áp động cơ	5 – 15V
3	Điện áp tín hiệu logic	2.7 ~ 5,5V
4	Dòng tối đa mỗi kênh	1.2A
5	Số động cơ hỗ trợ tối đa	2 động cơ
6	Điều khiển tốc độ	PWM (tối đa 100kHz)
7	Bảo vệ diode hồi tiếp	Có



Hình 2.11: Sơ đồ nguyên lý mạch cầu H điều khiển động cơ TB6612FNG

- **Sơ đồ chân kết nối của TB6612FNG:**

Sơ đồ chân kết nối được trình bày trong ảnh và bảng dưới đây:



Hình 2.12: Sơ đồ chân giao tiếp của mạch cầu H TB6612FNG

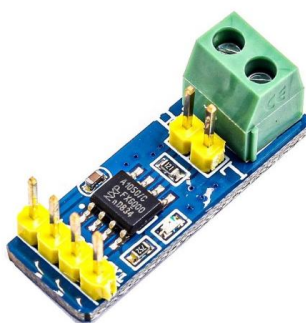
Bảng 2.8: Kết nối chân LN298N

STT	Thành phần	Đặc điểm
1	VM	Nguồn cấp cho motor (tối đa 15V)
2	VCC	Nguồn cấp cho mạch logic (tối đa 5.5v)

3	Ain1	Điều khiển chiều động cơ A
4	Ain2	
5	Bin1	Điều khiển chiều động cơ B
6	Bin2	
7	PWM A	PWM điều khiển động cơ A
8	PWM B	PWM điều khiển động cơ B
9	Aout1, Aout2	Nối động cơ A
10	Bout1, Bout2	Nối động cơ B
11	GND	Nguồn 0VDC

2.2.7 Mạch điều khiển giao tiếp CAN TJA1050

TJA1050 là IC thu phát sử dụng giao tiếp CAN được phát triển bởi nhà phân phối sản phẩm NXP. Mạch đóng vai trò chuyển đổi tín hiệu mức logic của bộ điều khiển CAN sang tín hiệu vi sai trên đường truyền CAN Bus và ngược lại.



Hình 2.13: Mạch điều khiển giao tiếp CAN TJA1050

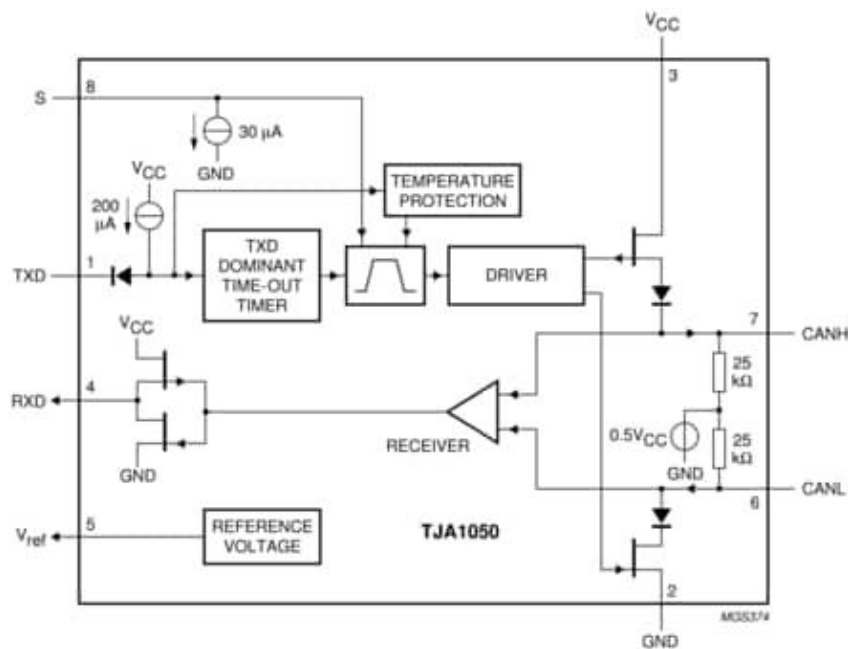
Bên trong hệ thống được thiết kế của đề tài này, TJA1050 được sử dụng để kết nối với máy tính Raspberry Pi 3B do thiết bị này không hỗ trợ CAN controller.

- **Thông số kỹ thuật của TJA1050:**

Thông số kỹ thuật Mạch cầu H điều khiển động cơ LN298N được cung cấp trong bảng dưới đây:

Bảng 2.9: Thông số kỹ thuật Mạch cầu H điều khiển động cơ TJA1050

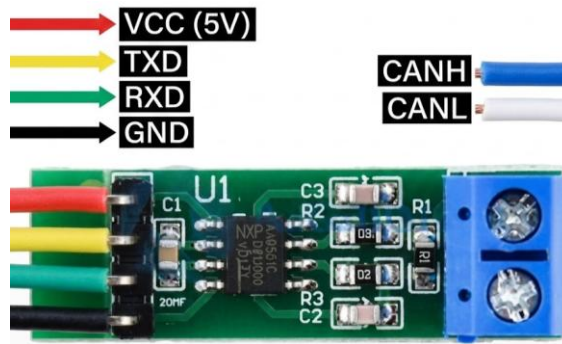
STT	Thành phần	Đặc điểm
1	Điện áp hoạt động	~5V
2	Tốc độ truyền tối đa	1Mbps
3	Kết nối	SPI - CAN



Hình 2.14: Sơ đồ nguyên lý của TJA1050

- **Sơ đồ chân kết nối của TJA1050:**

Sơ đồ chân kết nối được trình bày trong ảnh và bảng dưới đây:



Hình 2.15: Sơ đồ chân giao tiếp của mạch giao tiếp CAN TJA1050

Bảng 2.10: Kết nối chân của TJA1050

STT	Thành phần	Đặc điểm
1	VCC	5V
2	TXD	Chân nhận dữ liệu truyền từ vi điều khiển gửi đến module
3	RXD	Chân truyền dữ liệu nhận được từ CAN bus về vi điều khiển.
4	GND	Mass của mạch
5	CANH	Đường truyền tín hiệu CAN mức cao
6	CANL	Đường truyền tín hiệu CAN mức thấp

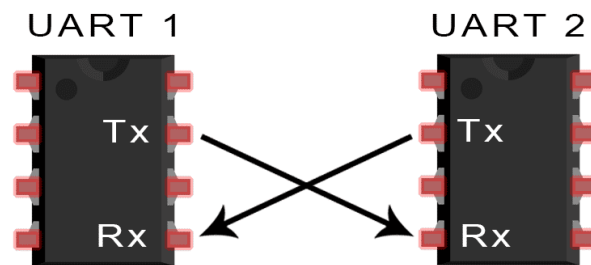
2.3 CÁC CHUẨN GIAO TIẾP VÀ GIAO THỨC TRUYỀN DỮ LIỆU

Để các module phần cứng và trạm điều khiển hoạt động đồng bộ (Asynchronous), hệ thống áp dụng nhiều chuẩn giao tiếp khác nhau tương ứng với từng tầng chức năng.

2.3.1 UART (Universal Asynchronous Receiver/Transmitter)

Chuẩn truyền thông nối tiếp không đồng bộ cơ bản, sử dụng hai dây TX (Transmit) và RX (Receive). Chuẩn giao tiếp này giúp hai thiết bị tham gia giao tiếp

mà không cần sử dụng chung một đường tín hiệu xung nhịp (Clock) để đồng bộ hóa dữ liệu.



Hình 2.16: Chuẩn giao tiếp UART

Do không sử dụng chung tín hiệu xung đồng bộ, các thiết bị khi giao tiếp cần thiết lập sẵn một tốc độ truyền dữ liệu gọi là Baudrate. Khi tham gia truyền tin, dữ liệu được đóng gói thành các khung (Frame). Một khung dữ liệu tiêu chuẩn có thể bao gồm: [1 Start bit][5 đến 9 Datas bits][1 Parity bit][1 đến 2 Stop bits].

Do đặc tính giao tiếp, chuẩn truyền thông này cho phép truyền tải dưới chế độ song công toàn phần (Full-duplex) để vừa truyền và nhận dữ liệu đồng thời. Nhưng tốc độ truyền tải của giao tiếp này thường tương đối thấp, khoảng cách truyền tin ngắn, dễ bị lệch xung nhịp và chỉ hỗ trợ giao tiếp điểm - điểm (Point to Point) nên phương thức này thường dùng để thực hiện giao tiếp giữa các thiết bị có khoảng cách gần nhau như máy vi tính kết nối với vi điều khiển, giữa các IC và ECUs...

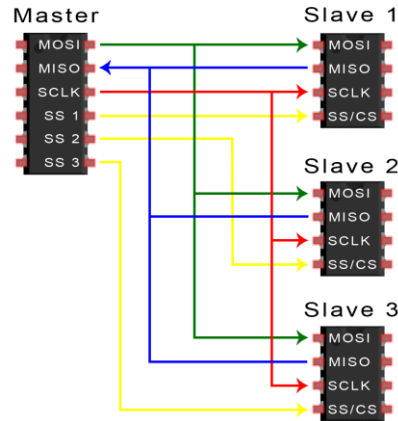
2.3.2 SPI (Serial Peripheral Interface)

Chuẩn giao tiếp SPI là giao thức truyền dữ liệu nối tiếp đồng bộ (Synchronous) hoạt động theo mô hình Master – Slave.

Về cấu trúc vật lý, chuẩn giao tiếp sử dụng cấu hình 4 dây tín hiệu tiêu chuẩn bao gồm:

- SCLK (Serial Clock): Tín hiệu xung nhịp đồng bộ do thiết bị làm Master tạo ra.
- MOSI (Master Out Slave In): Đường truyền dữ liệu từ Master đến Slave.

- MISO (Master In Slave Out): Đường truyền dữ liệu từ Slave đến Master.
- SS/CS (Slave Select/Chip Select): Đường tín hiệu quyết định thiết bị được chọn để giao tiếp



Hình 2.17: Chuẩn giao tiếp SPI

Khi Master muốn giao tiếp với một Slave cụ thể, Master sẽ thay đổi mức logic chân SS/CS tương ứng với Slave cần giao tiếp (thường kéo từ 1V xuống 0V). Khi này, Master tạo nhịp trên đường SCLK. Tại mỗi xung nhịp, một bit dữ liệu được truyền đi trên đường MOSI và ngược lại, Master nhận được một bit dữ liệu được gửi từ Slave.

Nhờ đặc tính truyền dữ liệu, SPI cũng cho phép truyền tải ở chế độ Full-Duplex nhưng có khả năng truyền tải ở tốc độ cao nên thường được sử dụng trong giao tiếp thẻ nhớ, Flash rời, các màn hình LCD/TFT...

2.3.3 I2C (Inter-Integrated Circuit)

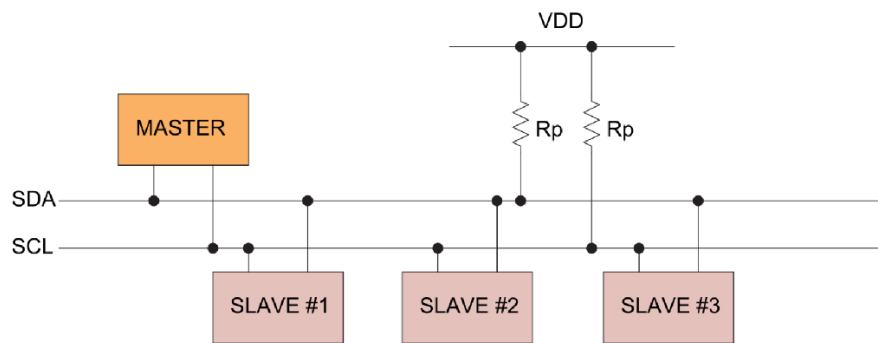
Giao thức I2C được phát triển bởi Philips Semiconductors với mục đích hỗ trợ giao tiếp giữa nhiều Master – nhiều Slave đồng bộ dựa trên cơ chế định địa chỉ bằng phần mềm.

Về cấu trúc vật lý, giao tiếp chỉ sử dụng duy nhất 2 dây tín hiệu 2 chiều:

- SDA (Serial Data): Đường truyền dữ liệu.

- SCL (Serial Clock): Đường truyền xung nhịp đồng bộ.

Khi hoạt động, Tất cả các thiết bị trên cùng Bus được định danh bằng một địa chỉ đặc biệt (7-bit hoặc 10-bit). Khi Master muốn giao tiếp, nó phát ra một trạng thái START trên bus kèm theo ID của Slave cùng với bit chỉ định hành động đọc/ghi (Read/Write hay R/W). Khi đó, Slave với ID trùng khớp sẽ phản hồi bằng một bit ACK và quá trình truyền nhận các byte dữ liệu được thực hiện. Khi kết thúc, Master sẽ gửi thông điệp STOP lên bus.



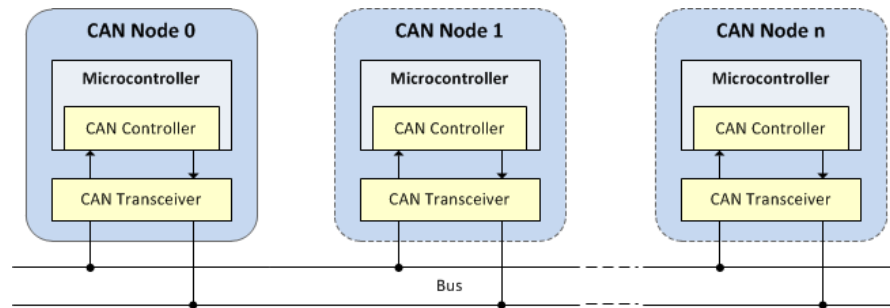
Hình 2.18: Chuẩn giao tiếp I2C

Dưới góc độ kỹ thuật, phương pháp truyền tin này chỉ cho phép truyền tải ở chế độ Bán song công (Haft-Duplex). Chuẩn giao tiếp này giúp master giao tiếp với tối đa 127 thiết bị chỉ với 2 chân và tốc độ truyền tải trung bình nên được ứng dụng rộng rãi trong các hệ thống cần đọc cảm biến môi trường, giữa các IC cấu hình EEPROM dung lượng nhỏ...

2.3.4 CAN (Controller Area Network)

CAN là một chuẩn giao tiếp mạng bất đồng bộ được phát triển bởi công ty Robert Bosch GmbH dành cho ngành công nghiệp ô tô nhằm mục đích thay thế hệ thống dây điện phức tạp tại thời điểm đó. Giao tiếp sử dụng một cặp dây xoắn vì sai bao gồm CAN_H (CAN High) và CAN_L (CAN_Low) để truyền dữ liệu giữa các bộ điều khiển.

Chuẩn giao tiếp CAN không sử dụng phương thức đánh địa chỉ phần cứng như I2C mà định nghĩa địa chỉ (ID) cho từng loại gói tin. Khi này các nodes trong mạng giao tiếp đều được nhận dữ liệu và chỉ cho phép các gói tin hợp lệ thông qua các lớp lọc (filters) được xây dựng trong quá trình phát triển sản phẩm.



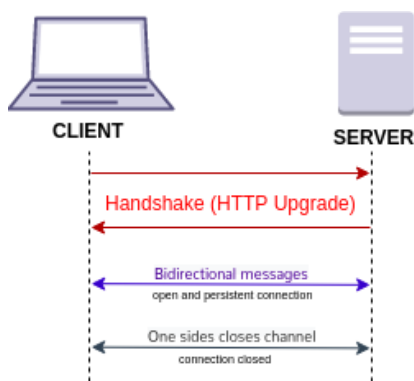
Hình 2.19: Chuẩn giao tiếp CAN

Về mặt kỹ thuật, giao thức CAN có khả năng chống nhiễu tốt nhờ sử dụng tín hiệu vi sao, khoảng cách truyền xa và được trang bị các cơ chế phát hiện và cô lập lỗi bằng phần cứng mạnh mẽ. Giao thức được ứng dụng mạnh mẽ trong mạng điều khiển trên xe ô tô (kết nối các ECUs, hệ thống phanh ABS...), hay các hệ thống điều khiển tự động trong nhà máy, xí nghiệp, các tòa nhà cao tầng...

2.3.5 Giao thức WebSockets

Để đáp ứng yêu cầu khắt khe về thời gian thực của tác vụ điều khiển xe từ xa, giao thức truyền thông như HTTP (vốn hoạt động theo cơ chế Request-Response một chiều và phải thiết lập lại header liên tục) là không khả thi. Mặt khác, WebSocket là một giao truyền tải dữ liệu phần mềm, cung cấp kênh giao tiếp song công toàn phần và liên tục trên một kết nối TCP duy nhất giữa Client và Server.

Về nguyên lý hoạt động, khi cần kết nối thì Client sẽ gửi một yêu cầu HTTP tiêu chuẩn lên Server để thiết lập Socket. Sau khi nhận được sự chấp nhận của Server, kết nối HTTP được nâng cấp thành WebSocket để loại bỏ hoàn toàn thủ tục Request – Response. Điều này giúp cả phía Server và Client có thể liên tục gửi tín hiệu không dây bất cứ lúc nào mà không cần tạo lại kết nối.

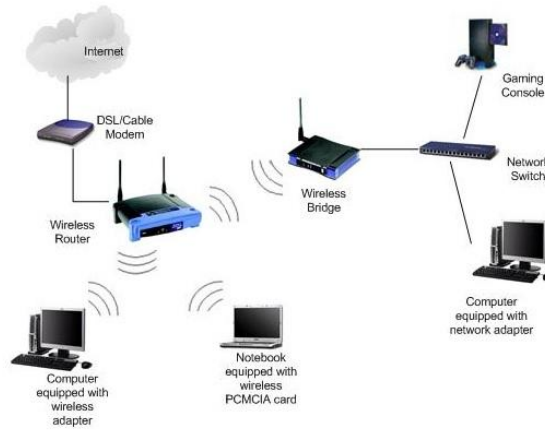


Hình 2.20: Mô tả giao thức WebSocket

Nhờ ưu thế độ trễ truyền tin cực thấp, băng thông mạng sử dụng cũng được tiết kiệm nhờ loại bỏ các header trong quá trình Request – Response mà WebSocket được ứng dụng trong các hệ thống yêu cầu cập nhật liên tục ở khoảng cách xa như trò chơi trực tuyến, các ứng dụng nhắn tin...

2.3.6 WI-FI (Wireless Fidelity)

Wifi là một họ các giao thức mạng không dây dựa trên tiêu chuẩn IEEE 802.11 để thiết lập các kết nối mạng cục bộ không dây (WLAN) và truy cập Internet bằng thông rộng. Trong đó, một thiết bị trung tâm (Access point) chuyển đổi dữ liệu số từ các mạng có dây thành sóng vô tuyến và phát sóng. Các thiết bị đầu cuối như máy tính, smartphone, các vi điều khiển sẽ nhận tín hiệu thông qua anten để giải mã dữ liệu thông qua các giao thức bảo mật (WPA2/WPA3) để đảm bảo an toàn thông tin.



Hình 2.21: Minh họa việc giao tiếp không dây thông qua Wifi

Wifi ở thời điểm hiện tại được ứng dụng rất phổ biến toàn cầu, phương thức kết nối này được thiết lập trong mọi môi trường như gia đình với việc kết nối Internet với các thiết bị dân dụng, truyền tải video (Camera), các hệ thống thông minh gia dụng cho đến môi trường doanh nghiệp để kết nối các máy chủ, các thiết bị điều khiển, quản lý trong mạng nội bộ.

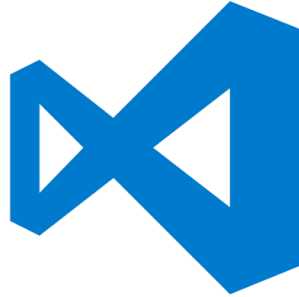
2.4 TỔNG QUAN VỀ CÁC CÔNG CỤ HỖ TRỢ XÂY DỰNG VÀ LẬP TRÌNH CHO HỆ THỐNG

Trong quy trình phát triển một hệ thống nhúng – IoT hiện đại, chu trình phát triển không chỉ giới hạn trong một lĩnh vực quy nhất. Đây thực chất là quy trình đòi hỏi sự phối hợp chặt chẽ giữa các lĩnh vực kỹ thuật trọng điểm: Cơ khí (Mechanical) – Điện tử (Electronics) – Phần mềm (Software). Trong đề tài này, các công cụ phổ biến trong các lĩnh vực được nhóm sử dụng để phác thảo, xây dựng hệ thống, đồng thời qua đó giúp làm rõ vai trò của các công cụ trong quy trình phát triển hệ thống kỹ thuật.

2.4.1 Môi trường lập trình nhúng (IDE - Integrated Development Environment)

Trong dự án này, nhóm sử dụng công cụ soạn thảo mã nguồn Visual Studio Code (VS Code) làm môi trường lập trình chính. Đây là trình soạn thảo văn bản do Microsoft phát triển, hỗ trợ đa nền tảng (Windows, Linux, macOS...). VS Code đóng

vai trò trung tâm, đảm nhiệm phần lớn các chức năng lập trình của toàn bộ hệ thống. Mặc dù không phải là một IDE tích hợp sẵn các công cụ chuyên dụng cho phần cứng, công cụ này lại sở hữu hệ sinh thái gói mở rộng (Extension) phong phú, hỗ trợ biến môi trường lập trình thành một nền tảng phù hợp cho việc phát triển hệ thống nhúng.



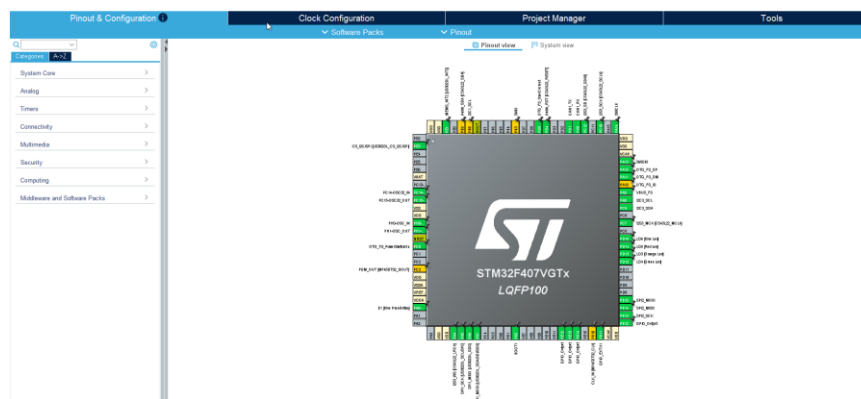
Hình 2.22: Trình soạn thảo văn bản Visual Studio Code

Các Extensions nổi bật được trang bị cho VS Code:

- **Cortex-Debug:** Đây là gói mở rộng chuyên dụng để gỡ lỗi (Debug) cho các dòng vi xử lý dựa trên lõi ARM Cortex. Tính năng này cho phép IDE tương thích với các mạch nạp/gỡ lỗi phổ biến như ST-LINK, J-LINK... Ngoài ra, tính năng đặt các điểm dừng (Break point) cho phép giám sát hoạt động của hệ thống lập trình theo các bước (stepping) và trạng thái thanh ghi để kịp thời phát hiện điểm cần gỡ lỗi.
- **Serial Monitor:** VS Code cũng cung cấp giao diện giao tiếp với phần cứng ngay trong không gian làm việc thông qua sử dụng cổng giao tiếp (Serial port). Điều này giúp người lập trình dễ dàng quan sát các input, output và trạng thái được phần cứng gửi đến.
- **Cmake Tools:** Gói tiện ích cho phép trình soạn thảo tự động quét và phát hiện các trình biên dịch có sẵn trong dự án và kiến trúc của dự án để thực hiện tự động hóa quá trình quản lý, cấu hình và biên dịch chương trình của người dùng sang mã máy cho các vi điều khiển

- **Git**(Version Management): Để quản lý dự án, VS Code cũng cung cấp cho người dùng công cụ quản lý mã nguồn Git để quản lý lịch sử thay đổi mã nguồn, khôi phục dữ liệu.... Điều này đặc biệt quan trọng giúp các kỹ sư có thể làm việc với dự án trong môi trường nhiều thành viên.

Ngoài ra, công cụ cấu hình phần cứng STM32CubeMX được cung cấp bởi nhà phát triển STMicroelectronics cũng được nhóm sử dụng trong dự án. Công cụ này được xây dựng để giúp các kỹ sư thiết lập các cấu hình vật lý của chip thông qua giao diện đồ họa trực quan thay vì tự tra cứu và lập trình thanh ghi thủ công.



Hình 2.23: Giao diện công cụ cấu hình vi điều khiển STM32CubeMX

Công cụ này đặc biệt quan trọng trong dự án bởi nó cung cấp người dùng mã nguồn giao tiếp với phần cứng được lập trình sẵn bởi nhà phát triển, người dùng chỉ cần sử dụng các APIs giao tiếp phần cứng để lập trình giao tiếp thay vì lập trình từng thanh ghi thủ công. Các file mã nguồn cũng được xây dựng theo một kiến trúc rõ ràng giúp người dùng có thể dễ dàng truy cập tra cứu và thay đổi các cấu hình thông qua công cụ truy xuất mã nguồn có sẵn trong VS Code.

2.4.2 Phần mềm mô phỏng và thiết kế mạch điện tử

Phần mềm mô phỏng mạch điện tử Proteus được sử dụng để xây dựng sơ đồ nguyên lý (Schematic). Công cụ này đặc biệt hữu ích để kiểm chứng logic thiết kế hệ thống nguồn (phân áp 3.3V/5V), kiểm tra dòng điện nuôi các module viễn thông khát dòng (như SIM900D), và giả lập các cổng COM ảo để test tập lệnh AT hoặc chuỗi dữ

liệu GPS trước khi nạp code xuống phần cứng vật lý. Việc tạo ra các khối linh kiện (Make Device) một cách chuyên nghiệp trong phần mềm này giúp chuẩn hóa bản vẽ báo cáo kỹ thuật.

2.4.3 Phần mềm thiết kế cơ khí 3D (CAD)

Phần mềm thiết kế cơ khí 3D (CAD) sử dụng các nền tảng như SolidWorks hoặc Fusion 360 để xây dựng bản vẽ kết cấu khung gầm. Các bản vẽ lắp ráp đảm bảo sự chính xác về kích thước vật lý khi khoan cắt các giá đỡ cho Lidar, Camera, cũng như mô phỏng cơ cấu lái hình thang Ackermann nhằm phản ánh đúng nhất động lực học của một chiếc xe ô tô thực tế. Tầng thiết kế này là tiền đề cho việc chế tạo bộ vỏ xe bằng phương pháp in 3D.

2.5 TỔNG QUAN VỀ THUẬT TOÁN ĐIỀU KHIỂN PHƯƠNG TIỆN

Thuật toán điều khiển phương tiện (AGV) được sử dụng đóng vai trò là bộ não chịu trách nhiệm tiếp nhận thông tin, xử lý logic và đưa ra các quyết định vận hành chính xác. Hệ thống điều khiển phương tiện hiện đại thường được thiết kế với cấu trúc phân lớp theo chức năng như sau:

- **Lớp thu thập và nhận thức môi trường (perception):** Hệ thống liên tục thu thập dữ liệu từ nhiều loại cảm biến khác nhau (Lidar, Radar,...) để số hóa các thông tin về môi trường xung quanh phương tiện nhằm tạo ra bản đồ thời gian thực về vị trí hiện tại của phương tiện so với vật cản xung quanh.
- **Lớp quyết định và an toàn chủ động (Decision & Collision Avoidance):** Dựa trên yêu cầu điều hướng đầu vào (từ người điều khiển hoặc thuật toán lập trình sẵn) kết hợp với thông tin nhận thức môi trường, hệ thống sẽ tiến hành đánh giá rủi ro. Các thuật toán phòng tránh va chạm đóng vai trò như một cơ chế an toàn độc lập.
- **Lớp điều khiển chấp hành (Actuation):** Khi một quyết định di chuyển được thông qua, nó cần được chuyển hóa thành các tác động cơ học. Các lệnh toán học được sử dụng để quy đổi thông tin điều khiển thành tín hiệu điện (như điều

ché độ rộng xung - PWM) để điều tiết vận tốc và hướng quay của cơ cấu chấp hành (động cơ), kết hợp với các cơ chế phanh điện từ giúp phương tiện tăng tốc và dừng lại mượt mà, chính xác.

- **Lớp bảo vệ dự phòng (Failsafe & Fault Tolerance):** Hệ thống luôn duy trì các bộ định thời (Watchdog Timer) để tự giám sát tính toàn vẹn của luồng giao tiếp dữ liệu. Nếu phát hiện sự cố mất tín hiệu điều khiển, độ trễ vượt ngưỡng an toàn hoặc trục trặc phần cứng, thuật toán sẽ ngay lập tức vô hiệu hóa hệ thống truyền động, đưa phương tiện về trạng thái an toàn nhất (Safe-state) nhằm ngăn ngừa tình trạng mất kiểm soát do quán tính.

CHƯƠNG 3 THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG

3.1 YÊU CẦU VÀ SƠ ĐỒ KHỐI HỆ THỐNG

3.1.1 Yêu cầu của hệ thống

Nhằm mục đích thực hiện mục tiêu xây dựng một hệ thống hỗ trợ đo khoảng cách vật thể và cảnh báo va chạm, đề tài “Thiết kế và triển khai thiết kế và thi công hệ thống cảnh báo va chạm trên phương tiện điều khiển” được thiết kế với các chức năng chính sau đây:

- **Cơ chế hoạt động**

Hệ thống được thiết kế để tự động giao tiếp và thu thập dữ liệu từ các cảm biến Lidar ở đầu thân xe và các cảm biến Ultrasonic ở các cánh trái, cánh phải và phía sau thân xe và gửi thông tin về cho người dùng. Đồng thời khi khoảng cách thu thập được từ các cảm biến đạt ngưỡng báo động (Ví dụ khi khoảng cách giữa xe và vật thể dưới 30 cm) thì hệ thống tự đưa ra quyết định phát tín hiệu cảnh báo và chủ động giảm tốc độ nhằm giữ khoảng cách an toàn. Mặt khác, hệ thống cũng cung cấp người dùng dịch vụ điều khiển phương tiện thông qua giao diện người dùng.

- **Thông báo và cảnh báo**

Khi phát hiện các tình huống nguy hiểm được thiết lập trong quá trình phát triển, hệ thống phải có một cơ chế truyền tải thông điệp cảnh báo đến người dùng bằng nhiều hình thức (hình ảnh, hành động...)

- **Giao diện người dùng**

Hệ thống phải có một giao diện điều khiển trực quan, dễ sử dụng, thao tác để người dùng chủ động trong việc điều khiển phương tiện cũng như nắm bắt được các thông tin trực quan về trạng thái của phương tiện, hệ thống nhằm thủ công đưa ra các lựa chọn phù hợp.

- **Lưu trữ dữ liệu**

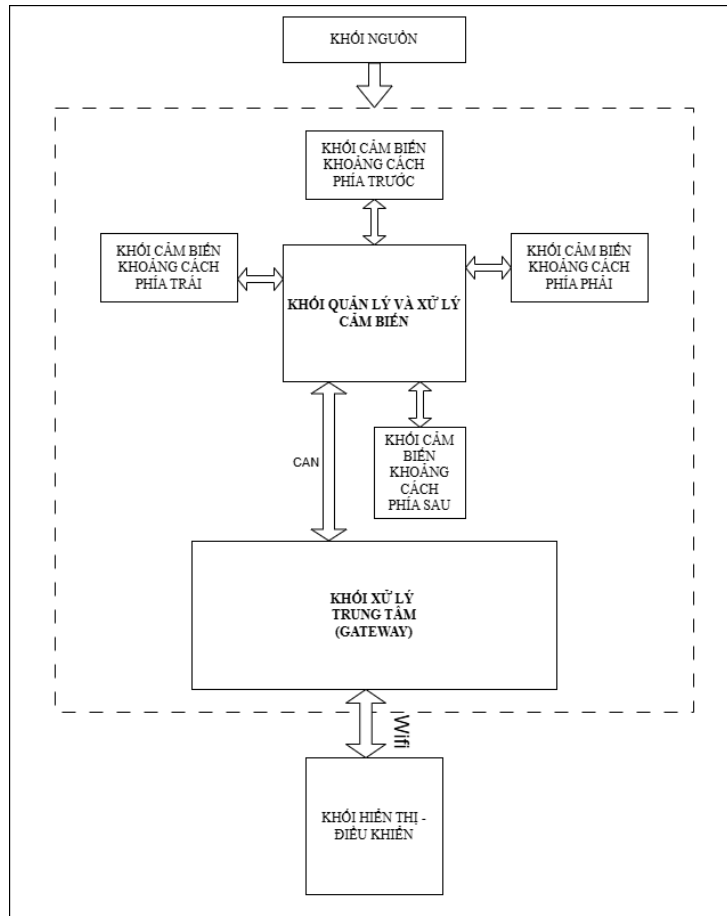
Hệ thống phải có khả năng lưu trữ các thông tin quan trọng về trạng thái của phương tiện điều khiển trong suốt quá trình hoạt động. Điều này đảm bảo hệ thống được quản lý chặt chẽ, đồng thời giúp người kỹ sư kịp thời can thiệp và sửa chữa trong các trường hợp phát hiện lỗi tiềm ẩn hoặc cần cập nhật các tính năng hỗ trợ người dùng.

Sau cùng, hệ thống trong đề tài được thiết kế nhằm hướng tới các mục tiêu:

- Hệ thống phải đảm bảo được sự ổn định với độ chính xác cao trong thời gian thực nhằm đảm bảo an toàn cho xe (yếu tố tiên quyết của bài toán về cảnh báo va chạm).
- Hệ thống không được thiết kế để thay thế hoàn toàn sự điều khiển của người dùng phương tiện mà chỉ nhằm mục đích trợ giúp báo cáo các thông tin quan trọng về khoảng cách và hỗ trợ điều khiển phương tiện trong trường hợp có thể xảy ra va chạm.

3.1.2 Sơ đồ khối hệ thống và các chức năng của các khối trong hệ thống

Hệ thống được thiết kế xoay quanh các chức năng cơ bản của các khối trong hệ thống được mô tả trong hình 3.1 dưới đây:



Hình 3.1: Sơ đồ khối hệ thống

Theo đó, các khối trong hệ thống được thiết kế với các chức năng cơ bản:

- **Khối quản lý và xử lý cảm biến:** Đây là khối chính đóng vai trò quản lý, giao tiếp, thập và xử lý các tín hiệu thu thập được từ các sensors và thực hiện tính toán, đưa ra quyết định tùy theo các điều kiện được thiết lập từ trước. Đồng thời khối cũng thực hiện chức năng gửi dữ liệu thu thập được cho người dùng thông qua màn hình TFT... sử dụng các giao thức I2C, UART và SPI. Đây là trung tâm điều phối toàn bộ hoạt động quan sát môi trường thực tế.
- **Khối cảm biến khoảng cách phía trước:** Đây thực tế là một hệ thống Lidar được gắn trước mũi trước phương tiện nhằm mục đích đo đạc và

gửi tín hiệu khoảng cách với độ trễ cực thấp kèm theo khoảng cách đảm bảo thời gian thực cho khối quản lý và xử lý cảm biến.

- **Khối cảm biến khoảng cách bên trái:** Là hệ thống sử dụng cảm biến ultra sonic gắn phía bên trái phương tiện nhằm mục đích đo đạc và gửi tín hiệu đến khối quản lý và xử lý cảm biến.
- **Khối cảm biến khoảng cách bên phải:** Là hệ thống sử dụng cảm biến ultra sonic gắn phía bên phải phương tiện nhằm mục đích đo đạc và gửi tín hiệu đến khối quản lý và xử lý cảm biến.
- **Khối cảm biến khoảng cách phía sau:** Là hệ thống sử dụng cảm biến ultra sonic gắn phía sau phương tiện nhằm mục đích đo đạc và gửi tín hiệu đến khối quản lý và xử lý cảm biến.
- **Khối hiển thị điều khiển:** Hiển thị giao diện người dùng, xuất các thông tin thu thập được từ các khối cảm biến khoảng cách nhằm giúp người dùng theo dõi trực tiếp các thông tin từ hệ thống
- **Khối xử lý trung tâm:** Là cầu nối giao tiếp giữa khối quản lý và xử lý cảm biến và người dùng, đóng vai trò điều khiển khối hiển thị điều khiển, điều khiển phương tiện và gửi cảnh báo đến người dùng trong các điều kiện cụ thể.
- **Khối nguồn:** Khối này đảm nhận cung cấp ổn định năng lượng cho các hoạt động của vi điều khiển, các sensors...

3.1.3 Hoạt động của hệ thống

Hệ thống được xây dựng để hoạt động theo kiến trúc phân tán gồm hai tầng xử lý cốt lõi. Khi được cấp nguồn, tầng cấp thấp của hệ thống sử dụng vi điều khiển STM32F407G-DISC chịu trách nhiệm thu thập dữ liệu thời gian thực từ các cụm cảm biến đa hướng (Lidar và các Ultrasonic sensors khác) để liên tục thu thập và tính toán khoảng cách giữa chủ thể là phương tiện điều khiển với các vật cản ở môi trường xung quanh. Khi phát hiện vật cản trong vùng nguy hiểm, STM32 lập tức gửi tín hiệu cảnh báo đến tầng cao hơn sử dụng Raspberry Pi 3B để kích hoạt cơ chế phanh tự động

khẩn cấp (Fail-Safe) để đảm bảo phương tiện được an toàn. Mặt khác, ở tầng trên sử dụng Raspberry có trách nhiệm quản lý trạng thái của toàn bộ hệ thống, tiếp nhận thông tin thu thập được từ các ECUs tầng dưới và làm trung gian giao tiếp giữa người dùng và phương tiện điều khiển thông qua sử dụng Websocket để đồng bộ telemetry, cho phép người dùng giám sát trực tiếp và vận hành xe từ xa với độ trễ thấp.

3.2 THIẾT KẾ HỆ THỐNG

3.2.1 Khối cảm biến khoảng cách phía trước

Trong hệ thống, khối cảm biến khoảng cách phía trước được sử dụng để đo khoảng cách giữa phương tiện và vật cản ở phía trước. Khối này được thiết kế yêu cầu sử dụng thiết bị độ chính xác cao, tốc độ đo đặc, phải hồi nhanh và hỗ trợ đo khoảng cách lớn. Với nhóm thực hiện đề tài, lý do quan trọng nhất khi yêu cầu cấu hình cao từ phần cứng do theo chiều di chuyển của xe, sự chênh lệch tốc độ giữa phương tiện và vật cản có thể trở nên rất lớn (ví dụ khi 1 phương tiện di chuyển với vận tốc 30 km/h, 1 vật cản khác di chuyển ngược chiều với vận tốc 10 km/h, như vậy độ chênh lệch vận tốc tương đối giữa phương tiện và vật cản lên đến 40 km/h).

❖ **Đầu vào:** Khoảng cách thực tế từ phương tiện đến vật cản phía trước.

❖ **Đầu ra:** Tín hiệu số thể hiện khoảng cách gửi đến STM32 để xử lý.

▪ Các phương án thiết kế và lựa chọn linh kiện

Phương án 1: Sử dụng cảm biến siêu âm HC-SR04

- **Đặc điểm:** Cảm biến hoạt động dựa trên nguyên lý phát ra sóng âm và đo đặc khoảng cách từ thời gian phản hồi. Cảm biến này nổi bật bởi giá thành thấp, dễ lập trình – tích hợp với các vi điều khiển và phổ biến trong các project các nhân kích cỡ nhỏ đến vừa
- **Đánh giá:** Cảm biến tuy có giá thành rẻ, dễ tích hợp song khoảng cách đo đạt tối đa thấp (khoảng 4m) và dễ bị hấp thụ tín hiệu, đồng thời tốc độ cập nhật

cũng không nhanh nên không phù hợp với bài toán đo khoảng cách xa với tốc độ lớn với khối cảm biến khoảng cách phía trước.

Phương án 2: Sử dụng cảm biến Lidar RPLIDAR A1M8

- Đặc điểm: Cảm biến Lidar quét này cung cấp khả năng phát hiện vật cả trên phạm vi rộng, thu thập được thông tin khoảng cách ở nhiều hướng khác nhau với độ chính xác lớn.
- Đánh giá: Dù có hiệu suất hoạt động tốt, cảm biến lại yêu cầu tiêu thụ điện năng lớn tương đối cao cùng với yêu cầu bài toán xử lý dữ liệu lớn vượt quá yêu cầu tính năng được thiết kế của khối cảm biến khoảng cách phía trước. Ngoài ra, kích thước cảm biến lớn dẫn đến khó tích hợp vào khung của phương tiện di chuyển trong dự án này.

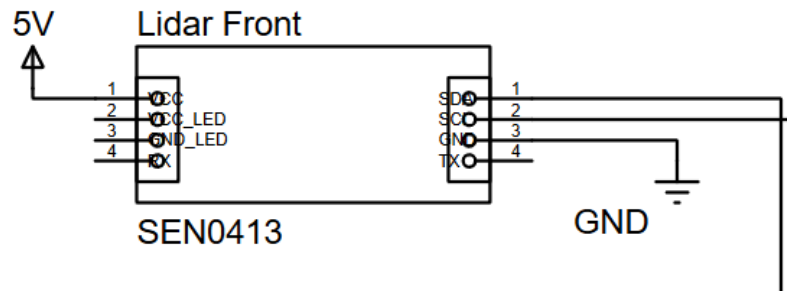
Phương án 3: Sử dụng cảm biến LIDAR07-100W-B

- Đặc điểm: Cảm biến sử dụng tia laser hồng ngoại phát xung tới vật thể và đo thời gian phản hồi để xác định khoảng cách.
- Đánh giá: Cảm biến này khắc phục được nhược điểm về khoảng cách đo của cảm biến siêu âm ở phương án 1, đồng thời vẫn đạt được độ chính xác cao và xung nhịp hoạt động lớn. Tuy nhiên, cảm biến không được hỗ trợ rộng rãi ở các dòng vi điều khiển nên yêu cầu bài toán tích hợp từ kỹ sư.

Phương án lựa chọn: Trong giới hạn đề tài, dựa trên các tiêu chí giá thành, đặc điểm, yêu cầu bài toán... nhóm thực hiện đề tài quyết định lựa chọn sử dụng cảm biến đo khoảng cách bằng hồng ngoại LIDAR07-100W-B. Nhờ khả năng đo đặc khoảng cách lớn với độ phản hồi thấp, thiết kế nhỏ gọn và giá thành hợp lý, Lidar07 được nhóm thực hiện đề tài đánh giá là sự lựa chọn phù hợp, tiết kiệm, đơn giản nhưng vẫn đảm bảo độ chính xác cao trong triển khai hệ thống.

▪ Thiết kế và nguyên lý hoạt động

KHOI CAM BIEN



Hình 3.2: Sơ đồ nguyên lý khối cảm biến khoảng cách phía trước

Khối cảm biến khoảng cách phía trước được thiết kế lấy trung tâm là cảm biến LIDAR07-100W-B. Trong đó, cảm biến này cung cấp dịch vụ giao tiếp thông qua hai chuẩn giao tiếp phổ thông là I2C và UART.

Trong sơ đồ nguyên lý khối cảm biến khoảng cách phía trước được đề cập phía trên, LIDAR07-100W-B được kết nối với khối quản lý và xử lý cảm biến thông qua header GPIO SCL và SDA của cảm biến đến header GPIO PA8(I2C_SCL) và PC9(I2C_SDA) của STM32F407G-DISC. Theo đó trong thiết kế logic được xây dựng bởi nhóm thực hiện đề tài, các thành viên trong nhóm quyết định sử dụng giao tiếp I2C để giao tiếp với cảm biến nhằm chủ động trong việc tiếp nhận tín hiệu theo chu kỳ thay vì để cảm biến tự động gửi tín hiệu liên tục đến STM dẫn đến khả năng tiềm ẩn lỗi mất tín hiệu, tín hiệu bị dịch trong quá trình đọc,...

▪ Cơ chế giao tiếp I2C giữa LIDAR07-100W-B và STM32F407G-DISC

Trong hệ thống, LIDAR07-100W-B và STM32F407G-DISC giao tiếp với nhau sử dụng chuẩn giao tiếp I2C. Theo đó, STM đóng vai trò là I2C Master chủ động trong việc tạo phiên truyền thông để yêu cầu dữ liệu khoảng cách, còn Lidar đóng vai trò I2C Slave phản hồi mỗi khi nhận được yêu cầu từ Master.

Đặc biệt, khung truyền dữ liệu nguyên bản của LIDAR07 được thiết kế cho chuẩn UART. Tuy nhiên, khi vận hành ở chế độ I2C (địa chỉ phần cứng 0x70), giao thức yêu cầu lược bỏ byte Header (0x00 hoặc 0xFA) trên đường truyền vật lý nhằm tối ưu hóa băng thông, đồng thời vi điều khiển host phải thực hiện tái tạo cấu trúc khung truyền bằng phần mềm để đảm bảo tính toàn vẹn của thuật toán kiểm tra lỗi (CRC).

a. Chiều truyền tín hiệu từ STM32F407G-DISC xuống LIDAR07-100W-B

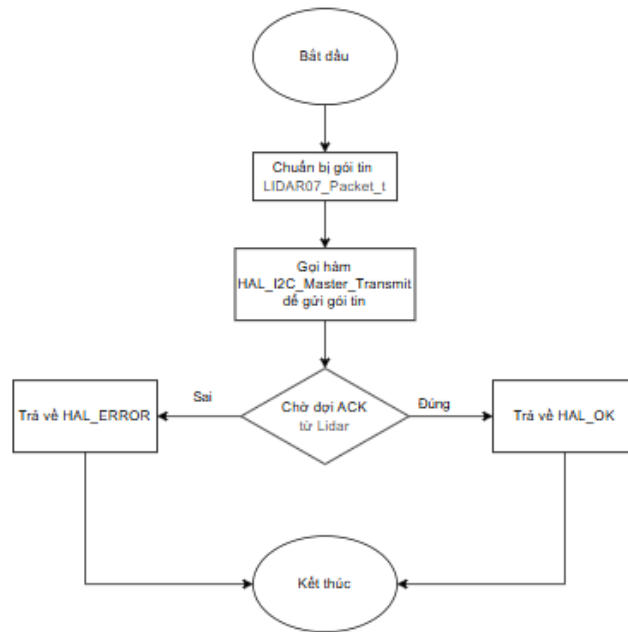
Ở chiều tạo phiên truyền thông từ STM32F407G-DISC, microcontroller này tiến hành gửi xuống Lidar gói dữ liệu khởi tạo chứa các thông tin yêu cầu kết quả đo đặc khoảng cách từ Lidar. Gói lệnh được gửi xuống có cấu trúc:

```
typedef struct {  
    uint8_t head;          /* 0xF5 for UART, 0x00 for I2C (not sent over bus) */  
    uint8_t command;  
    uint8_t data[4];  
    uint8_t checkData[4];  
} LIDAR07_Packet_t;
```

Trong đó:

- header = 0x00: Đánh dấu việc giao tiếp được thực hiện thông qua chuẩn I2C.
- command: Mã lệnh điều khiển Lidar (Ví dụ: 0x43 để đọc version, 0xE0 để yêu cầu đọc).
- data: Dữ liệu kèm theo mã lệnh điều khiển
- checkData: Sử dụng cơ chế kiểm tra lỗi CRC-32/MPEG-2 của toàn bộ 5 bytes command và data để đảm bảo tính toàn vẹn dữ liệu.

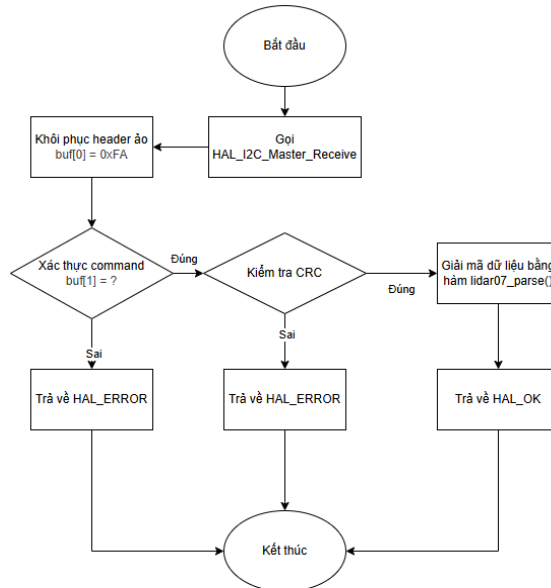
Cụ thể hơn, quá trình truyền gói tin được diễn ra như sau:



Hình 3.3: Sơ đồ xử lý luồng dữ liệu gửi từ STM32F407G-DISC xuống Lidar07-100W-B

b. Chiều nhận tín hiệu từ LIDAR07-100W-B về STM32F407G-DISC

Sau khi gửi xuống gói tín hiệu khởi tạo, Sensor tiến hành gửi lại gói tin về STM32 thông qua 2 loại: Gói tin phản hồi lệnh và Gói tin dữ liệu đo lường. Trong đó, quá trình nhận gói tin được diễn ra như sau:



Hình 3.4: Sơ đồ luồng xử lý dữ liệu từ Lidar07-100W-B lên STM32F407G-DISC

c. Các gói dữ liệu phản hồi

Tùy vào gói lệnh được gửi đi (Đọc dữ liệu hay đọc thông tin phần cứng) mà Lidar sẽ trả về cho STM32 chuỗi dữ liệu tương ứng. Cụ thể hơn:

- Gói phản hồi lệnh (Command Response Packet): Gói thông tin này được sử dụng khi STM32 gửi các lệnh cấu hình hoặc đọc trạng thái phần cứng cảm biến như phiên bản firmware, chế độ đo... Khi xử lý lệnh này, Lidar sẽ gửi lại một khung 11 bytes phản hồi xác thực việc nhận được lệnh thực thi gồm [0xFA][1 byte Echo][6 bytes Payload thông tin trả về theo lệnh][4 bytes CRC-32].
- Gói dữ liệu đo lường (Measurement Packet): Gói dữ liệu này được dùng để trả về kết quả cho STM32 thông tin khoảng cách đo đạc được. Khi dữ liệu có kích thước 24 bytes và được chia thành các nhóm gồm [0xFA][1 byte Echo][2 bytes Reserved][2 bytes giá trị khoảng cách đo được][2 bytes Reserved][2 bytes cường độ tín hiệu][10 bytes Reserved][4 bytes CRC].

d. Phần mềm/ Thuật toán sử dụng

Để xây dựng lớp phần mềm hỗ trợ giao tiếp giữa STM32F407G-DISC và LIDAR07-100W-B, nhóm thực hiện đề tài kế thừa chương trình giao tiếp giữa Lidar07-100W-B và dòng vi điều khiển ESP, thay đổi các APIs giao tiếp I2C, UART dành riêng cho dòng vi điều khiển ESP sang APIs đặc trưng của dòng SRM32 để xây dựng được một bộ thư viện giao tiếp mới. Bộ thư viện này đảm bảo được các chức năng cơ bản để giao tiếp với Lidar như cấu hình chế độ đọc, thông tin phiên bản, xử lý CRC, trích xuất thông tin gói tin...

3.2.2 Khối cảm biến khoảng cách phía trái, phía phải, phía sau

Khác với yêu cầu khắt khe đối với khối cảm biến khoảng cách phía trước. Ở các phía bên trái, bên phải của phương tiện, khi xét về độ chênh lệch của phương tiện với các vật thể theo chiều này thường thấp hơn rất nhiều (không xét đến trường hợp phương tiện di chuyển theo chiều ngang với chiều di chuyển của vật thể). Mặt khác, khi xét phía sau của phương tiện, việc sử dụng phần cứng cảm biến cao cấp cũng kém hấp dẫn khi đa số các phương tiện hiện nay không cung cấp khả năng di chuyển tốc độ cao theo hướng này.

❖ **Đầu vào:** Khoảng cách thực tế từ phương tiện đến vật cản phía trước.

❖ **Đầu ra:** Tín hiệu số thể hiện khoảng cách gửi đến STM32 để xử lý.

▪ Các phương án thiết kế và lựa chọn linh kiện

Phương án 1: Sử dụng cảm biến siêu âm HC-SR04

- **Đặc điểm:** Cảm biến hoạt động dựa trên nguyên lý phát ra sóng âm và đo đặc khoảng cách từ thời gian phản hồi. Cảm biến này nổi bật bởi giá thành thấp, dễ lập trình – tích hợp với các vi điều khiển và phổ biến trong các project các nhân kích cỡ nhỏ đến vừa
- **Đánh giá:** Cảm biến tuy có giá thành rẻ, dễ tích hợp song khoảng cách đo đạt tối đa thấp (khoảng 4m) và dễ bị hấp thụ tín hiệu, đồng thời tốc độ cập nhật cũng không nhanh nên không phù hợp với bài toán đo khoảng cách xa với tốc độ lớn với khối cảm biến khoảng cách phía trước.

Phương án 2: Sử dụng cảm biến siêu âm chống nước JSN-SR04T

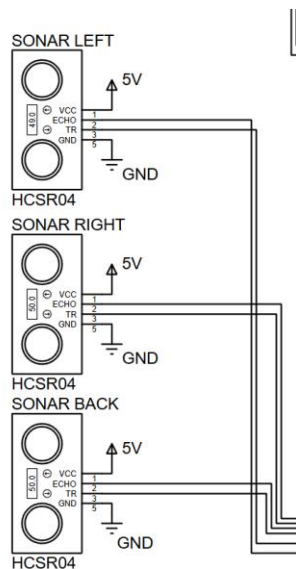
- Đặc điểm: Cảm biến JSN-SR04T có chung nguyên lý hoạt động với HC-SR04 nhưng được thiết kế với đầu dò chống nước, đặc biệt phù hợp với môi trường ngoài trời hoặc môi trường có độ ẩm cao
- Đánh giá: Cảm biến có giá thành rẻ, dễ dàng sử dụng nhưng yêu cầu khoảng cách đo tối thiểu tương đối lớn (khoảng 20 cm), độ chính xác thấp hơn HC-SR04 cũng như có tốc độ xử lý chậm hơn.

Phương án 3: Sử dụng cảm biến hồng ngoại ToF VL53L0X

- Đặc điểm: Cảm biến sử dụng tia laser hồng ngoại phát xung tới vật thể và đo thời gian phản hồi để xác định khoảng cách, có kích thước nhỏ gọn cho các dòng xe mô hình có kích thước nhỏ
- Đánh giá: cảm biến có độ chính xác rất cao cũng như tốc độ cập nhật nhanh nhưng giá thành tương đối cao hơn so với 2 phương án trên nếu xét về mặt yêu cầu chức năng trên giá thành

Phương án lựa chọn: Bởi những lý do trên đối chiếu với giá thành, hiệu suất, yêu cầu bài toán..., đối với cả ba khối cảm biến khoảng cách phía trái, phía phải và phía sau, nhóm thiết kế quyết định sử dụng cảm biến khoảng cách HC-SR04.

- **Thiết kế và nguyên lý hoạt động**



Hình 3.5: Sơ đồ nguyên lý khối cảm biến khoảng cách phía trái, phải, sau

Khối cảm biến khoảng cách ở các phía bên phải, bên trái và phía sau được thiết kế sử dụng các sensor HC-SR04. Trong đó, các cảm biến này không sử dụng các giao thức chuẩn phổ thông mà được thiết kế sử dụng giao tiếp 1 dây đặc biệt.

Trong sơ đồ nguyên lý khối cảm biến khoảng cách phía trái, phía phải, phía sau được đề cập, các HC-SR04 được nhóm thiết kế sử dụng chân tín hiệu (SIG) gắn với các header GPIO PE12, PE13, PE14 trên STM32F407G-DISC với yêu cầu các GPIO này trên STM32 phải đảm nhận cả 2 chức năng truyền/nhận tín hiệu. Giao thức giao tiếp khi được xây dựng bởi nhóm cũng phải đáp ứng được yêu cầu Khối quản lý và xử lý tín hiệu phải được chủ động trong vấn đề tiếp nhận tín hiệu để đảm bảo hiệu năng tính toán, tránh các cảm biến phải hoạt động ngay cả khi tín hiệu không được đọc dẫn đến lãng phí tài nguyên tính toán, năng lượng, đồng thời đảm bảo tính toàn vẹn của tín hiệu trong quá trình thu thập dữ liệu.

- **Cơ chế giao tiếp độc quyền giữa HC-SR04 và STM32F407G-DISC**

Khác với nhiều cảm biến phổ thông sử dụng các giao thức giao tiếp chuẩn như I2C, SPI..., HC-SR04 được nhà cung cấp thiết kế để chỉ sử dụng 1 đường truyền duy nhất để giao tiếp với các vi điều khiển, đồng thời cơ chế nhận/ gửi dữ liệu cũng được

thiết kế riêng cho module này. Để có thể giao tiếp với HC-SR04, nhóm tiến hành kế thừa nguồn xử lý thuật toán điều khiển của nhà cung cấp cho các dòng vi điều khiển ARDUINO, ESP... và xây dựng 1 lớp giao tiếp mới sử dụng các APIs giao tiếp phần cứng với STM32 để chỉnh sửa, thay thế các chức năng trong mã nguồn sao cho phù hợp với vi điều khiển STM32F407G-DISC.

3.2.3 Khối quản lý và xử lý cảm biến

Trong đề tài “Thiết kế và triển khai thiết kế và thi công hệ thống cảnh báo va chạm trên phương tiện điều khiển”, khối quản lý và xử lý hệ thống đóng vai trò quan trọng mật thiết trong hỗ trợ phương tiện quan sát môi trường. Với vai trò như vậy, khối yêu cầu một bộ tính toán mạnh mẽ nhằm đáp ứng điều khiển nhiều thiết bị với độ chính xác đáng tin cậy, độ trễ cực thấp hay thời gian thực mà không gây treo, sập hệ thống.

Trên thị trường hiện nay có rất nhiều các vi điều khiển hay máy tính nhúng phổ thông được phát hành như ESP, ARDUINO, RASPBERRY... hay các dòng được thiết kế cụ thể giải quyết bài toán Automotive của NXP hay RENESAS. Việc lựa chọn vi điều khiển phù hợp phải được cân nhắc thông qua các tiêu chí như nhu cầu sử dụng, giá thành sản phẩm, các tính năng hỗ trợ, công cụ được nhà phát triển cung cấp...

- **Các phương án thiết kế và lựa chọn linh kiện**

Phương án 1: Sử dụng máy tính nhúng Raspberry Pi 3 Model B

- **Đặc điểm:** Raspberry Pi 3B là một máy tính nhúng sử dụng bộ xử lý ứng dụng chạy hệ điều hành Linux. Máy tính này có hiệu năng xử lý rất cao đồng thời có hệ sinh thái người dùng rộng lớn, hỗ trợ giao diện điều khiển và kết nối không dây.
- **Đánh giá:** Mạch phù hợp với việc sử dụng là module cho khối quản lý và xử lý cảm biến, nhưng mạch tiêu thụ điện năng lớn, độ phức tạp trong xây dựng phần mềm quản lý cao đồng thời chi phí rất lớn.

Phương án 2: Sử dụng vi điều khiển ESP32

- Đặc điểm: ESP32 có khả năng tích hợp kết nối không dây như Wifi hay Bluetooth, có hệ sinh thái người dùng lớn giúp việc phát triển các hệ thống nhúng – IoT dễ dàng hơn.
- Đánh giá: Tuy có nhiều thư viện hỗ trợ, mạch có hiệu năng xử lý tương đối thấp so với nhu cầu bài toán của nhóm. Ngoài ra, mạch cũng chưa đáp ứng được sự ổn định trong hoạt động lâu dài.

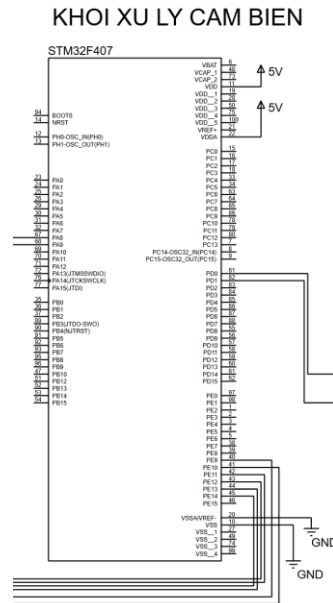
Phương án 3: Sử dụng STM32F407G-DISC

- Đặc điểm: STM32F407G-DISC là bộ kit phát triển sử dụng nhân ARM Cortex-M4 giúp xử lý tốt các thuật toán xử lý dữ liệu lớn nhờ khả năng tính toán mạnh mẽ, mạch cũng cung cấp các module giao tiếp phổ thông như I2C, SPI, CAN, UART,...
- Đánh giá: Tuy mạch có giá thành cao, yêu cầu phức tạp trong xây dựng và phát triển các sản phẩm nhưng STM32F407G-DISC có khả năng xử lý rất cao, có khả năng đáp ứng tính toán tốt theo thời gian thực.

Phương án lựa chọn: Với các vi điều khiển, máy tính nhúng của ESP hay ARDUINO thường có kích thước bộ nhớ thấp, tốc độ xử lý chậm và không hỗ trợ tốt xử lý đa nhiệm trong thời gian thực trong khi các dòng của RASPBERRY, NXP hay RENESAS tuy đáp ứng tốt các yêu cầu về bộ nhớ, lường thực thi và tốc độ xử lý đáng kinh ngạc lại có chi phí cao, chỉ phù hợp cho các nhà sản xuất, nhà phân phối giải pháp Automotive lớn sử dụng. Dựa trên các yêu cầu bài toán cần phải xử lý và yêu cầu hệ thống, nhóm thực hiện đề tài quyết định sử dụng dòng vi điều khiển STM32F407G-DISC1 Development Boards của STMicroelectronics làm trái tim của khối quản lý và xử lý cảm biến. Thiết bị được trang bị vi điều khiển ARM Cortex-M4 32-bit với tốc độ xung nhịp đạt đến 168MHz phù hợp cho các tác vụ điều khiển với tốc độ xử lý cao, theo đó là bộ nhớ 1MB Flash và 192KB RAM đáp ứng tốt cho việc sử dụng middleware RTOS như FreeRTOS và lưu trữ các mảng dữ liệu lớn. Bởi những ưu điểm trên, STM32F407G-DISC1 là lựa chọn phù hợp cho khối quản lý và xử lý

cảm biến, với các nhiệm vụ bao gồm: thu thập dữ liệu từ các cảm biến từ 4 phía của phương tiện, xử lý các tín hiệu thu thập được và gửi đến khối xử lý trung tâm, tự đưa ra các quyết định trong trường hợp cụ thể.

- **Thiết kế và nguyên lý hoạt động**



Hình 3.6: Sơ đồ nguyên lý khối quản lý và xử lý cảm biến

Khối quản lý và xử lý cảm biến được kết nối trực tiếp với các cảm biến đo khoảng cách ở các khối cảm biến khoảng cách, đồng thời làm trung gian vận hành của các “giác quan” hệ thống với khối xử lý trung tâm. Khối đảm nhận vai trò thu thập và xử lý dữ liệu, truyền dữ liệu sang khối xử lý trung tâm đến người dùng.

Tham chiếu hình: Các kết nối của khối quản lý và xử lý cảm biến được hiển thị trong sơ đồ nguyên lý đề cập phía trên, trong đó:

- Tiếp nguồn của khối được kết nối trực tiếp đến khối nguồn toàn bộ hệ thống.
- Đường GND được kết nối chung với toàn bộ hệ thống.
- Các GPIO I2C3 được kết nối đến Lidar07-100W-B để đồng bộ clock và tạo đường truyền dữ liệu

- Các GPIO được kết nối lần lượt cho các cảm biến Ultrasonic được gắn ở phía trái, phía phải và phía sau xe.

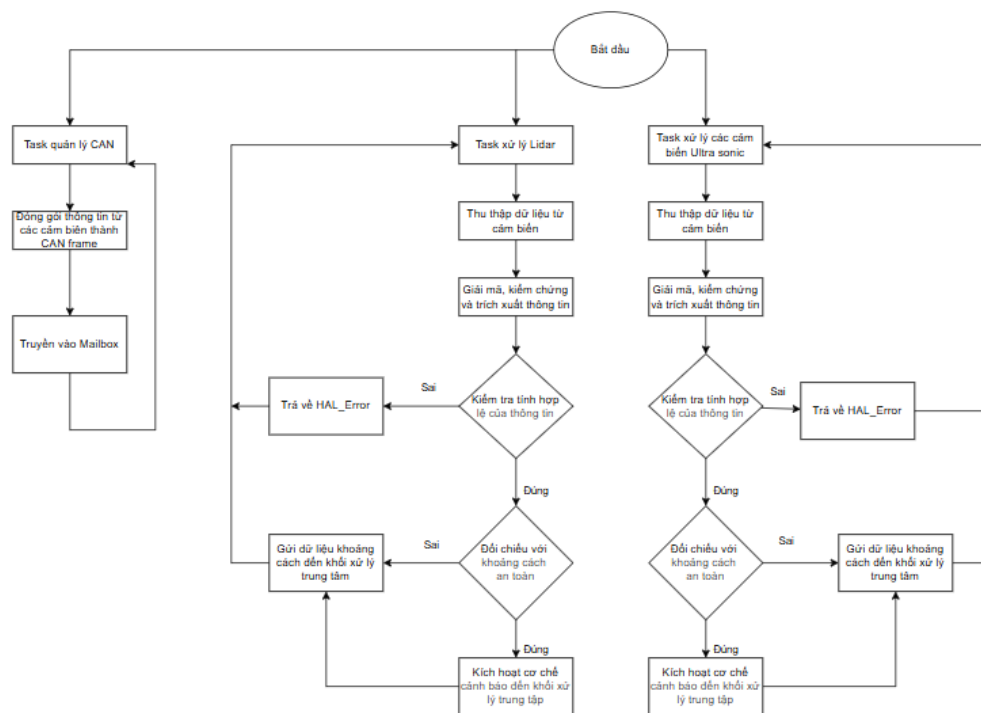
Nguyên lý hoạt động:

Khi được cấp nguồn, STM32F407G-DISC tiến hành boot khởi tạo các driver nội bộ như driver giao tiếp (I2C, CAN...), hệ điều hành RTOS và lập tức khởi tạo driver điều khiển các cảm biến được tạo bởi người lập trình.

Trong quá trình hệ thống hoạt động, STM32 này liên tục thu thập các tín hiệu từ các cảm biến khoảng cách ở các phía, tiến hành xử lý chuỗi dữ liệu thu thập được và cập nhật dữ liệu khoảng cách lên khối xử lý trung tâm thông qua chuẩn giao tiếp CAN.

STM32 cũng sử dụng 4 LEDs tích hợp để hiển thị trạng thái khoảng cách hiện tại trực tiếp thông qua thông điệp bằng tần số nhấp nháy tùy theo các mức khoảng cách được thiết lập từ trước, trong đó đèn nháy mỗi 1s khi khoảng cách hiện tại lớn hơn 1m, nhấp mỗi 0.5s khi khoảng cách trong khoảng 0.5m đến 1m và nhấp liên tục mỗi 0.2s khi khoảng cách dưới 0.5m.

- **Phần mềm/ Thuật toán sử dụng**



Hình 3.7: Luồng xử lý thuật toán của khối quản lý và xử lý cảm biến

STM32F407G-DISC sử dụng hệ điều hành thời gian thực FreeRTOS để xử lý toàn bộ các tác vụ trong hệ thống đã đề cập phía trên. Microcontroller được lập trình để điều phối xử lý các tín hiệu bằng ngôn ngữ C, kết hợp với các BSP được cung cấp bởi nhà phát triển STMicroelectronics để hỗ trợ giao tiếp với phần cứng gián tiếp thông qua lớp HAL. Cụ thể hơn, khi hệ thống hoạt động, STM32 tiến hành phân chia thành 2 tiến trình quản lý bao gồm tiến trình xử lý dữ liệu khoảng cách từ các cánh bên trái, bên phải, phía sau và tiến trình khác dành riêng cho việc xử lý dữ liệu khoảng cách phía trước. Sau khi các dữ liệu được thu thập và giải mã, chúng được STM32 lập tức chuyển đến khối xử lý trung tâm sử dụng chuẩn giao tiếp CAN. Các tiến trình liên tục lặp lại theo thời gian thực, đáp ứng các chức năng cảnh báo cơ bản của một hệ thống ADAS.

3.2.4 Khối xử lý trung tâm (Gateway)

Trong đề tài này, khối xử lý trung tâm đóng vai trò là bộ não của toàn bộ hệ thống, trong đó khối trực tiếp chịu trách nhiệm tiếp nhận dữ liệu từ khối quản lý cảm

biến để thực hiện tổng hợp thông tin, đưa ra quyết định ở mức hệ thống và đóng vai trò là cầu nối giữa phương tiện điều khiển và toàn bộ các hệ thống và thiết bị khác bên ngoài.

Chức năng chính: Trong hệ thống điều khiển được nhóm thiết kế, khối xử lý trung tâm đảm nhận các nhiệm vụ sau:

- Thu thập dữ liệu từ các ECUs (Trong trường hợp này là STM32F407G-DISC)
 - Hiển thị trạng thái hệ thống trên giao diện người dùng
 - Truyền tải dữ liệu điều khiển và cảm biến giữa phương tiện và các thiết bị theo dõi không dây.
 - Điều khiển và quản lý trạng thái của phương tiện.
- **Các phương án thiết kế và lựa chọn linh kiện**

Phương án 1: Sử dụng máy tính nhúng Raspberry Pi 3 Model B

- Đặc điểm: Raspberry Pi 3B là một máy tính nhúng sử dụng bộ xử lý ứng dụng chạy hệ điều hành Linux. Máy tính này có hiệu năng xử lý rất cao đồng thời có hệ sinh thái người dùng rộng lớn, hỗ trợ giao diện điều khiển và kết nối không dây.
- Đánh giá: Mạch phù hợp với việc sử dụng là module cho khối quản lý và xử lý cảm biến, nhưng mạch tiêu thụ điện năng lớn, độ phức tạp trong xây dựng phần mềm quản lý cao đồng thời chi phí rất lớn.

Phương án 2: Tích hợp hệ thống vào STM32F407G-DISC

- Đặc điểm: STM32F407G-DISC có khả năng đa nhiệm tốt với độ trễ thấp, giúp người lập trình có thể sử dụng thêm tài nguyên tính toán để xây dựng một hệ thống nhỏ giúp vận hành xe
- Đánh giá: Tuy có tài nguyên bộ nhớ và tốc độ xử lý cao, đồng thời việc tích hợp có thể làm giảm chi phí cũng như độ phức tạp giữa việc điều tiết tín hiệu

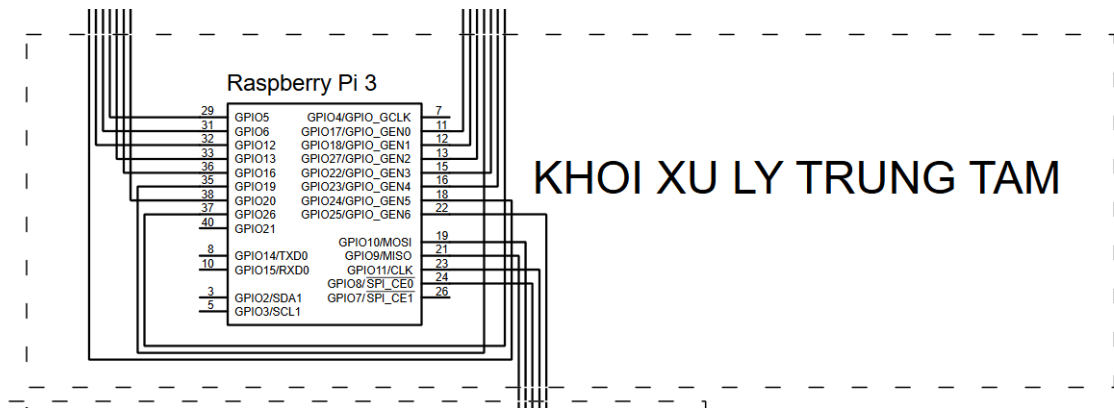
cảm biến và tín hiệu điều khiển, việc sử dụng STM32 trong trường hợp này gây khó khăn trong việc xây dựng hệ thống giao diện người dùng. Ngoài ra việc phải tích hợp thêm hệ thống điều khiển cũng có thể khiến tốc độ xử lý các tác vụ khác bị gián đoạn. Sau cùng, dựa trên nghiên cứu của nhóm, phương pháp tách các ECUs và Gateway thường được các nhà thiết kế xe sử dụng nhằm đảm bảo khả năng vận hành cũng như độ chính xác với các vi điều khiển trong thời gian hoạt động lâu dài, liên tục.

Phương án 3: Sử dụng Smartphone làm Gateway kết hợp ESP32 điều khiển

- **Đặc điểm:** Các thiết bị điện thoại di động thông minh được tích phần cứng mạnh mẽ cũng như cung cấp các dịch vụ cần thiết như màn hình, mạng không dây,... Người dùng có thể xây dựng một app linh hoạt để vừa điều khiển vừa quản lý trạng thái của phương tiện.
- **Đánh giá:** Việc sử dụng thiết bị di động có thể khiến hệ thống trở nên phức tạp, khó can thiệp vào các tầng thấp hơn của thiết bị di động. Việc sử dụng smartphone đòi hỏi kiến thức lập trình ứng dụng di động cũng như hệ thống nhúng phức tạp.

Phương án lựa chọn: Sau khi đối chiếu với yêu cầu của đề tài là một mô hình xe cảnh báo va chạm và cần đảm bảo tính thời gian thực, nhóm thực hiện đề tài quyết định lựa chọn sử dụng Raspberry Pi 3B cho Gateway của hệ thống. Phương án này đảm bảo sự cân bằng giữa khả năng xử lý mạng, tính ổn định của hệ thống truyền thông và sự thuận tiện trong việc phát triển giao diện giám sát, đáp ứng đúng các tiêu chí của hệ thống đã đặt ra.

▪ Thiết kế và nguyên lý hoạt động



Hình 3.8: Sơ đồ nguyên lý khối xử lý trung tâm

Khối xử lý trung tâm trong hệ thống được lựa chọn tập trung vào Raspberry Pi 3B, đóng vai trò là Gateway của toàn bộ hệ thống. Khối này đóng vai trò là bộ não cấp cao (High-level control unit) để đảm nhiệm các chức năng:

- Thu thập dữ liệu từ khối xử lý cảm biến thông qua mạng LAN để quyết định hành vi của phương tiện, đồng thời hiển thị thông tin cho người dùng thông qua giao diện người dùng.
- Xử lý, tổng hợp và quản lý trạng thái điều khiển chính của hệ thống.
- Gửi cảnh báo và chủ động xử lý trong tình huống phát hiện có thể xảy ra va chạm.

Tham chiếu hình: Các kết nối của khối quản lý và xử lý cảm biến được hiển thị trong sơ đồ nguyên lý đề cập phía trên, trong đó:

- Tiếp nguồn của khối được kết nối trực tiếp đến khối nguồn toàn bộ hệ thống.
- Đường GND được kết nối chung với toàn bộ hệ thống.
- Các GPIO I2C3 được kết nối đến TJA1050 thông qua sử dụng giao tiếp SPI, gián tiếp giao tiếp với khối quản lý và xử lý cảm biến thông qua mạng giao tiếp CAN.
- Sử dụng Socket và Web Interface để giao tiếp với người dùng, liên tục cập nhật thông tin khoảng cách và nhận tín hiệu điều khiển phương tiện.

Nguyên lý hoạt động:

Khi được cấp nguồn, Raspberry tiến hành boot hệ thống, khởi tạo các cấu hình phần cứng, phần mềm và hệ điều hành Linux.

Trong quá trình hoạt động, Raspberry liên tục giao tiếp với STM32 để thu thập các dữ liệu về khoảng cách từ khối quản lý và xử lý cảm biến, đồng thời cập nhật lên giao diện người dùng các thông tin này thông qua sử dụng WebSocket cùng với một số thông tin khác về trạng thái hoạt động hệ thống. Mặt khác, máy tính cũng tiếp nhận thông tin điều khiển từ phía người dùng để xử lý và đưa ra quyết định điều khiển động cơ của phương tiện

- **Phần mềm/ Thuật toán sử dụng**

Raspberry sử dụng Linux Kernel là hệ điều hành chịu trách nhiệm điều phối toàn bộ hoạt động của hệ thống. Khi được hệ điều hành này điều khiển, các bash script được xây dựng để hệ thống tự động khởi động và reboot hoạt động của mạng CAN giao tiếp với khối quản lý và xử lý cảm biến, các chương trình điều khiển động cơ và giao diện người dùng trên lớp User Space mà không cần đưa hệ thống xuống Kernrl Space, vốn yêu cầu nghiêm ngặt đối với hoạt động của các driver điều khiển và thường bị cấm truy cập trong các hệ điều hành sử dụng trên phương tiện tham gia giao thông để tránh tình trạng sập hệ thống không mong muốn.

Trong đề tài này, hệ Linux đảm nhận việc chia luồng các công việc đọc dữ liệu từ CAN bus, host giao diện người dùng và điều khiển động cơ để hệ thống hoạt động ổn định theo thời gian thực.

3.2.5 Khối hiển thị - điều khiển

Trong đề tài này, khối hiển thị - điều khiển được nhóm xây dựng với mục đích là giao diện tương tác trực tiếp giữa hệ thống cảnh báo va chạm với người sử dụng. Khối có nhiệm vụ cung cấp thông tin trạng thái hoạt động của phương tiện, hiển thị khoảng cách với vật cản, đồng thời cho phép người dùng thực hiện các thao tác điều khiển hướng di chuyển của phương tiện

- ❖ **Đầu vào:** Tín hiệu điều khiển phương tiện do người dùng thủ công nhập vào, các thông tin được nhập vào bao gồm tốc độ xoay của các motors.
- ❖ **Đầu ra:** Thông tin về trạng thái xe bao gồm khoảng cách từ các cảm biến phía trước, phía phải, phía trái, phía sau, trạng thái cảnh báo va chạm và tình trạng kết nối với hệ thống.

Hệ thống khi được thiết kế được nhóm xác định các yêu cầu sau:

- Hiện thị trực quan hướng xác định vật cản và mức độ nguy hiểm tương ứng với khoảng cách đo được.
- Có khả năng tương tác, cập nhật liên tục trong thời gian thực.
- Có giao diện đơn giản, dễ quan sát và sử dụng.
- Dễ tích hợp với bộ xử lý trung tâm.

Phương án 1: Sử dụng một bộ điều khiển từ xa.

- Đặc điểm: Với phương án này, nhóm có thể sử dụng một bộ điều khiển được thiết sẵn để điều khiển không dây với phương tiện
- Đánh giá: Module điều khiển được thiết kế sẵn, giá thành thấp và có thiết kế bắt mắt. Tuy nhiên, nhiều module điều khiển trên thị trường không được cung cấp thiết kế sản phẩm dẫn đến khó khăn trong việc tích hợp cơ chế điều khiển vô tuyến phù hợp với hệ thống. Mặt khác, module không có màn hình hiển thị hay các LEDs thông điệp khoảng cách dẫn đến module thiếu tính năng so với yêu cầu của nhóm

Phương án 2: Thiết kế một module gắn màn hình TFT kết hợp các phím điều khiển

- Đặc điểm: Ở phương án này, một module tự tạo có khả năng hiển thị thông tin khoảng cách đo được từ các cảm biến, trạng thái hoạt động của hệ thống và mức độ nguy hiểm. Người dùng đồng thời có thể sử dụng các nút nhấn/joystick để điều khiển di chuyển của hệ thống.

- Đánh giá: Module tùy biến có khả năng giao tiếp với người dùng ở mức độ đáp ứng cơ bản các yêu cầu, dễ dàng tích hợp thông qua các giao tiếp có dây/không dây. Tuy nhiên để thiết kế module cần sử dụng 1 microcontroller phù hợp dẫn đến tăng chi phí trong thiết kế và sản xuất. Đồng thời khiến hệ thống trở nên phức tạp.

Phương án 3: Sử dụng Smartphone và giao diện Web làm giao diện người dùng

- Đặc điểm: Điện thoại thông minh có phần cứng mạnh mẽ và được thiết kế sẵn các logic tương tác với màn hình, người dùng có thể truy cập Web để theo dõi trạng thái hệ thống theo thời gian thực.
- Đánh giá: Phương án này cho phép khả năng tùy chỉnh cao, có khả năng hiển thị màu sắc trực quan theo các mức cảnh báo và khoảng cách, nhiều tính năng cũng có thể được hiển thị cùng lúc. Ngoài ra, phương án này cho phép người dùng điều khiển ở nhiều vị trí khác nhau so với phương tiện, chỉ phụ thuộc vào mạng giao tiếp với hệ thống. Tuy nhiên, phương án này yêu cầu bài toán phát triển và triển khai Web trên một hệ thống để người dùng có thể truy cập, dẫn đến độ phức tạp phần mềm cao hơn.

Phương án lựa chọn: Sau khi đánh giá các phương án, nhóm quyết định sử dụng giao diện Web trên điện thoại thông minh sử dụng chính Raspberry Pi làm host điều phối. Lý do nhóm quyết định lựa chọn phương án này bao gồm khối xử lý trung tâm được tích hợp module phát Wifi giúp việc xây dựng giao diện hiển thị dễ dàng mà không cần thiết kế phần cứng bổ sung. Mặt khác, điện thoại thông minh là thiết bị phổ biến trên thị trường và gần như luôn có sẵn đối với người sử dụng, giúp giảm chi phí triển khai hệ thống.

▪ Thiết kế và nguyên lý hoạt động

Khối hiển thị - điều khiển sử dụng giao diện Web làm giao diện người dùng. Người dùng có thể kết nối với giao diện này thông qua kết nối đến Raspberry Pi thông qua mạng nội bộ và truy cập vào địa chỉ host của Raspberry.

Nguyên lý hoạt động: Mạng liên kết giữa khối hiển thị - điều khiển và khối xử lý trung tâm được xác định là mạng kết nối không dây Wifi, trong đó:

- Khi hệ thống được cấp nguồn, Raspberry Pi 3B khởi động module giao diện người dùng được thiết kế trước bởi nhóm.
- Trong suốt quá trình hoạt động, module liên tục thu thập dữ liệu cảm biến được truyền về từ STM32 lên UI của giao diện điều khiển, giúp người dùng quan sát trực quan khoảng cách với các vật cản ở các phía. Đồng thời, module cũng tiếp nhận tín hiệu điều khiển của người dùng trên giao diện web về để khối xử lý trung tâm xử lý và điều khiển di chuyển phương tiện.

▪ **Phần mềm/ Thuật toán sử dụng**

Khối sử dụng Raspberry Pi để host một giao diện người dùng sử dụng qua Web Interface sử dụng thư viện Flask trên nền ngôn ngữ lập trình Python. Hệ thống thiết lập một WebSocket để gửi các gói tin khoảng cách lên giao diện cũng như nhận các tín hiệu điều khiển của người dùng xuống khối điều khiển động cơ.

3.2.6 Khối điều khiển động cơ

Trong hệ thống được thiết kế, khối điều khiển động cơ quyết định hành vi và trạng thái di chuyển của phương tiện thực tế. Khối đảm nhận điều phối các động cơ bên trong xe cũng như điều khiển rẽ hướng để di chuyển phương tiện.

- ❖ **Đầu vào:** Tín hiệu điều khiển phương tiện do người dùng thủ công nhập vào, các thông tin được nhập vào bao gồm tốc độ xoay của các motors và được hệ thống điều khiển trung tâm xử lý trước khi khối điều khiển động cơ nhận được.
- ❖ **Đầu ra:** Các trạng thái hướng xoay và tốc độ của các động cơ trong việc điều khiển di chuyển của phương tiện.

Khối khi được thiết kế được nhóm xác định các yêu cầu sau:

- Có khả năng điều hướng phương tiện sang các vị trí khác tùy thuộc vào sự điều khiển của người dùng

- Có khả năng thay đổi tốc độ trong quá trình điều khiển
- Có cơ chế rẽ hướng dễ xây dựng và điều hành.
- Dễ tích hợp với bộ xử lý trung tâm.

Phương án 1: Sử dụng 1 Module TB6612FNG cho Cơ cấu lái Ô tô (1 Chạy + 1 Rẽ)

- Đặc điểm: Với phương án này, nhóm có thể sử dụng 2 động cơ cho việc điều khiển dẫn động cho phương tiện và rẽ hướng. Cụ thể hơn, một động cơ DC được kết nối và điều khiển thông qua module TB6612FNG để thay đổi tốc độ, hướng di chuyển của động cơ. Mặt khác, 1 động cơ servo có thể được sử dụng để làm xoay trục của 2 bánh trước hoặc sau thông qua cơ chế cơ khí đặc biệt của bánh rẽ để giúp phương tiện di chuyển
- Đánh giá: Phương án này hoàn hảo trong việc xây dựng 1 mô hình xe mô phỏng phương tiện tham gia giao thông trong thực tế. Tuy nhiên phương án cần được cung cấp một mô hình xe có hỗ trợ hệ thống đánh lái, điều này đòi hỏi người thực hiện dự án có kinh nghiệm trong xây dựng mô hình phần cứng, cũng như gia công cơ khí để hoàn thành sản phẩm phục vụ cho phương án.

Phương án 2: Sử dụng 1 Module TB6612FNG (Mắc song song theo cặp)

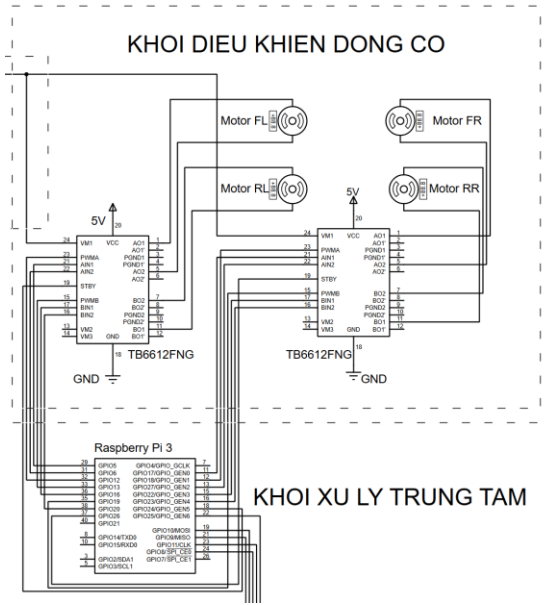
- Đặc điểm: Ở phương án này, hai động cơ bên trái và bên phải được đấu song song nhau và kết nối vào cùng một module TB6612FNG. Khi này xe sẽ xoay theo cơ chế lái trượt, trong đó bánh ở 2 bên trái, phải sẽ xoay theo chiều ngược nhau để thực hiện rẽ hướng
- Đánh giá: Phương án có chi phí rẻ, ít tốn không gian mạch điện đồng thời cho phép người dùng lập trình thay đổi hướng dễ dàng thông qua giao tiếp với chỉ 1 module TB6612FNG. Tuy nhiên module này có khả năng chịu dòng khá thấp nên việc sử dụng 1 module cho đồng thời 4 động cơ có thể dẫn đến quá tải hệ thống.

Phương án 3: Sử dụng 2 Module TB6612FNG (Mắc song song theo cặp)

- Đặc điểm: Phương án này kế thừa ý tưởng của phương án 2 nhưng việc điều khiển động cơ ở 2 phía được tách riêng cho 2 module TB6612FNG quản lý.
- Đánh giá: Phương án này khắc phục khả năng chịu dòng của phương án 2 nhưng yêu cầu người dùng quản lý tốt trong cơ chế điều khiển module, đồng thời phương án có khả năng chiếm dụng không gian nhiều hơn cho lượng GPIO cần để phục vụ phương án tăng gấp 2 lần.

Phương án lựa chọn: Sau khi đánh giá các phương án, nhóm quyết định sử dụng phương án 3 cho dự án. Lý do nhóm quyết định lựa chọn phương án này bao gồm hệ thống gặp điểm yếu trong khung phương tiện được xây dựng để lắp ráp các module được liệt kê phía trên. Đồng thời phương án 3 cũng giúp phương tiện có khả năng xoay hướng mà không cần tịnh tiến về một phía, đặc biệt phù hợp trong địa hình có không gian hẹp.

- **Thiết kế và nguyên lý hoạt động**



Hình 3.9: Sơ đồ nguyên lý khối điều khiển động cơ

Đi sâu vào cấu trúc điều phối, hệ thống trên áp dụng cấu hình phân bổ mỗi IC TB6612FNG quản lý một cụm hai bánh xe, khai thác tối đa 4 kênh điều khiển độc lập cho 4 động cơ. Cụ thể, IC TB6612FNG thứ nhất (phía bên trái) nhận tín hiệu từ Pi để điều khiển hai động cơ bên trái; trong đó Kênh A (các ngõ ra AO1, AO2) cấp dòng cho động cơ trước trái (Motor FL) và Kênh B (các ngõ ra BO1, BO2) cấp dòng cho động cơ sau trái (Motor RL). Tương tự, IC TB6612FNG thứ hai (phía bên phải) được cấp phát để quản lý hai động cơ bên phải, với Kênh A điều khiển động cơ trước phải (Motor FR) và Kênh B điều khiển động cơ sau phải (Motor RR).

Khi Raspberry Pi phát tín hiệu, các module TB6612FNG đóng vai trò khuếch đại dòng điện, chuyển đổi tín hiệu điều khiển công suất thấp thành dòng điện công suất cao để làm quay các động cơ theo đúng chiều và tốc độ được chỉ định. Nhờ kiến trúc sử dụng 2 module điều khiển 4 động cơ hoàn toàn biệt lập, hệ thống này khắc phục được tình trạng quá tải dòng điện so với việc mắc song song, đồng thời mang lại khả năng cơ động. Cấu trúc phần cứng này là nền tảng tiêu chuẩn để triển khai các thuật toán di chuyển phức tạp, từ cơ chế đánh lái trượt cho đến việc tích hợp bánh xe đa hướng giúp robot có thể tịnh tiến ngang hoặc xoay tròn tại chỗ.

3.2.7 Khối nguồn

Khối nguồn là một trong những phần quan trọng của hệ thống xe tự hành cảnh báo va chạm, vì đây là khối chịu trách nhiệm cấp và phân phối năng lượng cho toàn bộ các linh kiện trên xe. Trong quá trình hoạt động, các khối như Raspberry Pi, STM32F407, cảm biến, module truyền thông và mạch điều khiển động cơ đều cần được cấp nguồn ổn định. Nếu nguồn cấp không đủ dòng hoặc điện áp bị dao động quá nhiều, hệ thống có thể bị treo, cảm biến đọc sai giá trị hoặc động cơ hoạt động không ổn định. Vì vậy, khi thiết kế khối nguồn, nhóm cần lựa chọn phương án vừa đảm bảo yêu cầu kỹ thuật, vừa phù hợp với mô hình thực nghiệm của đề tài.

- ❖ **Đầu vào:** Nguồn năng lượng từ bộ 2 pin Lithium-ion 18650 mắc nối tiếp, cung cấp điện áp đầu vào cho toàn bộ hệ thống.
- ❖ **Đầu ra:** Mức điện áp 5V ổn định, được sử dụng để cấp cho các khối chức năng chính như khối xử lý trung tâm Raspberry Pi, khối xử lý cảm biến STM32F407, các cảm biến Lidar/Sonar, module giao tiếp CAN, hệ thống động cơ và mạch điều khiển động cơ.

Trong quá trình thiết kế và hoàn thiện hệ thống, nhóm sinh viên đã xác định được những tiêu chí mà khối nguồn cần đáp ứng như sau:

- Cung cấp điện áp, dòng điện ổn định cho toàn hệ thống
- Đáp ứng được dòng tải khi các linh kiện hoạt động đồng thời, đặc biệt là tại thời điểm động cơ khởi động hoặc thay đổi tốc độ.
- Dễ dàng tích hợp với các khối phần cứng khác cũng như thuận tiện cho việc bảo trì và thay thế.

Phương án 1: Sử dụng pin dự phòng (Power Bank)

- Đặc điểm: Nguồn năng lượng được cung cấp từ một bộ sạc dự phòng thương mại. Với khả năng cung cấp điện áp 5V thông qua cổng USB và được tích hợp sẵn mạch quản lý pin, bảo vệ quá dòng, quá áp và sạc pin.

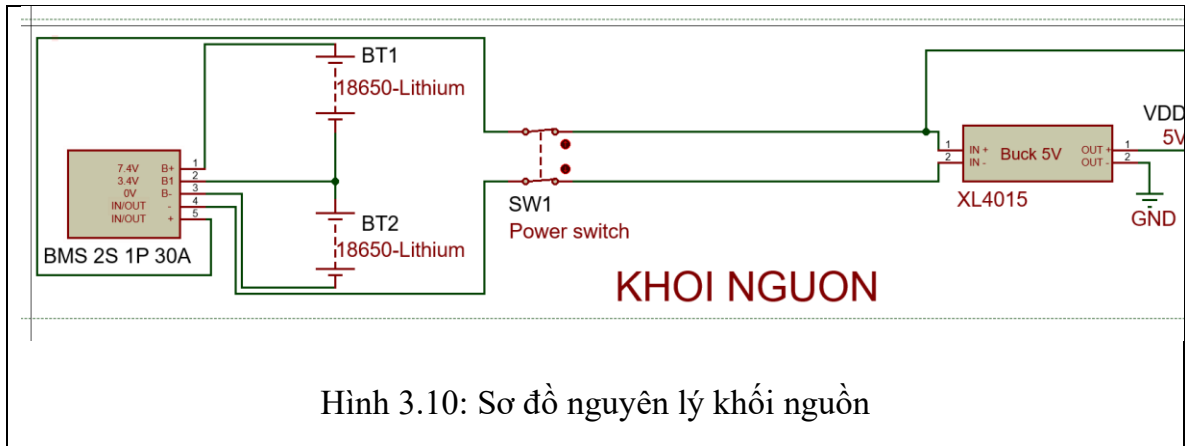
- Đánh giá: Phương án có ưu điểm là dễ sử dụng, độ an toàn cao và không yêu cầu thiết kế thêm mạch quản lý năng lượng. Tuy nhiên, công suất đầu ra của nhiều bộ sạc dự phòng bị giới hạn, gây khó khăn khi hệ thống hoạt động đồng thời với nhiều động cơ. Khả năng tùy biến và mở rộng của phương án này còn hạn chế.

Phương án 2: Sử dụng bộ pin Lithium-ion 2S kết hợp mạch hạ áp DC-DC

- Đặc điểm: Hệ thống sử dụng hai cell pin Lithium-ion 18650 mắc nối tiếp tạo thành bộ pin 2S với điện áp danh định khoảng 7,4V và điện áp tối đa 8,4V. Điện áp từ bộ pin được đưa qua mạch chuyển đổi DC-DC hạ áp để tạo ra mức điện áp 5V cung cấp cho Raspberry Pi, STM32, các cảm biến và các module ngoại vi.
- Đánh giá: Phương án có cấu trúc đơn giản, dễ triển khai và phù hợp với yêu cầu công suất của hệ thống. Điện áp đầu vào cao hơn điện áp hoạt động của các thiết bị giúp mạch hạ áp hoạt động hiệu quả và ổn định. Đồng thời, bộ pin 2S có kích thước nhỏ gọn, khối lượng thấp và dễ bố trí trên mô hình xe. Tuy nhiên, điện áp pin giảm dần trong quá trình sử dụng, do đó cần có cơ chế giám sát mức pin để tránh hiện tượng xả sâu làm ảnh hưởng đến tuổi thọ pin.

Phương án lựa chọn: Sau khi phân tích các phương án trên, nhóm quyết định lựa chọn phương án sử dụng bộ pin Lithium-ion 2S kết hợp mạch hạ áp DC-DC. Phương án này đáp ứng đầy đủ yêu cầu về công suất, điện áp và thời gian hoạt động của hệ thống trong khi vẫn đảm bảo tính đơn giản trong thiết kế và thi công. Điện áp đầu ra 5 V từ mạch hạ áp có thể được phân phối trực tiếp đến Raspberry Pi, STM32F407, các cảm biến và module giao tiếp CAN. Bên cạnh đó, việc sử dụng bộ pin 2S giúp giảm khối lượng và kích thước tổng thể của phương tiện, phù hợp với mục tiêu xây dựng mô hình thử nghiệm có chi phí hợp lý và khả năng triển khai thực tế cao.

▪ Thiết kế và nguyên lý hoạt động



Kết nối chính:

- Khối nguồn của hệ thống sử dụng hai cell pin Lithium-ion 18650 mắc nối tiếp để tạo thành bộ pin 2S. Mỗi cell pin Lithium-ion có điện áp danh định khoảng 3,7V và điện áp cực đại khi sạc đầy khoảng 4,2V. Do mắc nối tiếp nên có thể tính toán tổng điện áp của khối nguồn bằng công thức như sau:

$$V_{pack} = V_{BT1} + V_{BT2}$$

Trong đó:

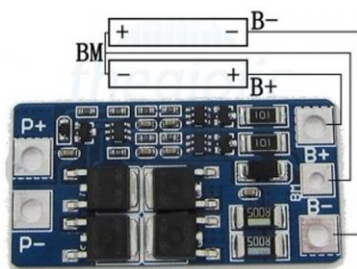
V_{pack} : Điện áp tổng của khối nguồn

V_{BT1} : Điện áp của cell pin thứ nhất

V_{BT2} : Điện áp của cell pin thứ hai

Như vậy, khối nguồn được thiết kế sẽ có điện áp dao động từ 6V đến 8.4V tùy thuộc vào mức dung lượng của pin.

- Hai cell pin được kết nối với mạch bảo vệ sạc xả BMS 2S 1P 10A, các chân chính của BMS gồm B+, B1 và B-. Trong đó, chân B+ được nối với cực dương của bộ pin, chân B1 được nối với điểm giữa hai cell pin và chân B- được nối với cực âm của bộ pin. Cách kết nối này giúp BMS có thể giám sát điện áp của từng cell pin trong quá trình sạc và xả.



P+P-input/output interface

Hình 3.11: Sơ đồ kết nối 2 cell pin với mạch bảo vệ BMS

- Ngõ ra của BMS được đưa qua công tắc nguồn SW1. Công tắc này có nhiệm vụ đóng hoặc ngắt nguồn cấp cho toàn bộ hệ thống. Khi công tắc ở trạng thái OFF, nguồn từ pin không được cấp đến các khối phía sau. Khi công tắc ở trạng thái ON, điện áp từ bộ pin sau BMS được đưa đến đầu vào của mạch hạ áp XL4015.
- Mạch XL4015 có nhiệm vụ chuyển đổi điện áp đầu vào từ bộ pin 2S xuống mức điện áp 5V ổn định. Đầu ra dương của XL4015 được nối với đường nguồn VDD 5V, còn đầu ra âm được nối với GND chung của hệ thống.
- Nguồn 5V sau mạch hạ áp được phân phối đến các khối chức năng trong hệ thống, bao gồm Raspberry Pi, vi điều khiển STM32F407, các cảm biến siêu âm, module giao tiếp CAN MCP2515 và mạch điều khiển động cơ. Việc sử dụng chung một đường GND giúp các khối xử lý, cảm biến và module truyền thông có cùng mức điện áp tham chiếu, từ đó đảm bảo tín hiệu giao tiếp giữa các khối được ổn định.

Nguyên lý hoạt động:

- Ở trạng thái ban đầu, khi hệ thống chưa được khởi động, công tắc SW1 ở trạng thái hở, do đó nguồn điện từ bộ pin chưa được cấp đến mạch hạ áp XL4015 và

các khối phía sau. Khi người dùng gạt công tắc sang trạng thái đóng, dòng điện sẽ đi từ bộ pin Lithium-ion 2S, qua mạch bảo vệ BMS, tiếp tục qua công tắc SW1, rồi đến đầu vào của mạch hạ áp XL4015.

- Trong suốt quá trình hệ thống hoạt động, mạch BMS đảm nhiệm vai trò giám sát và bảo vệ bộ pin liên tục. Thông qua các điểm đo B+, B1 và B-, BMS theo dõi riêng điện áp của từng cell pin trong bộ pin 2S. Nếu điện áp của một cell giảm xuống dưới ngưỡng an toàn (báo hiệu nguy cơ xả sâu), hoặc nếu dòng điện tải vượt quá giới hạn cho phép, hoặc xảy ra hiện tượng ngắn mạch, BMS sẽ tự động ngắt kết nối nguồn để bảo vệ pin và các linh kiện phía sau khỏi hư hỏng.
- Sau khi qua công tắc SW1, điện áp từ bộ pin (dao động trong khoảng 6V–8,4V tùy theo mức dung lượng còn lại) được đưa đến mạch hạ áp XL4015. Mạch này hoạt động theo nguyên lý nguồn xung hạ áp (buck converter), có nhiệm vụ chuyển đổi và ổn định điện áp đầu vào xuống mức 5V tại đầu ra. Công suất đầu ra của mạch được xác định theo công thức:

$$P_{out} = V_{out} \times I_{out}$$

Trong đó:

P_{out} : Công suất đầu ra của mạch hạ áp

V_{out} : Điện áp đầu ra của mạch hạ áp (5.1V)

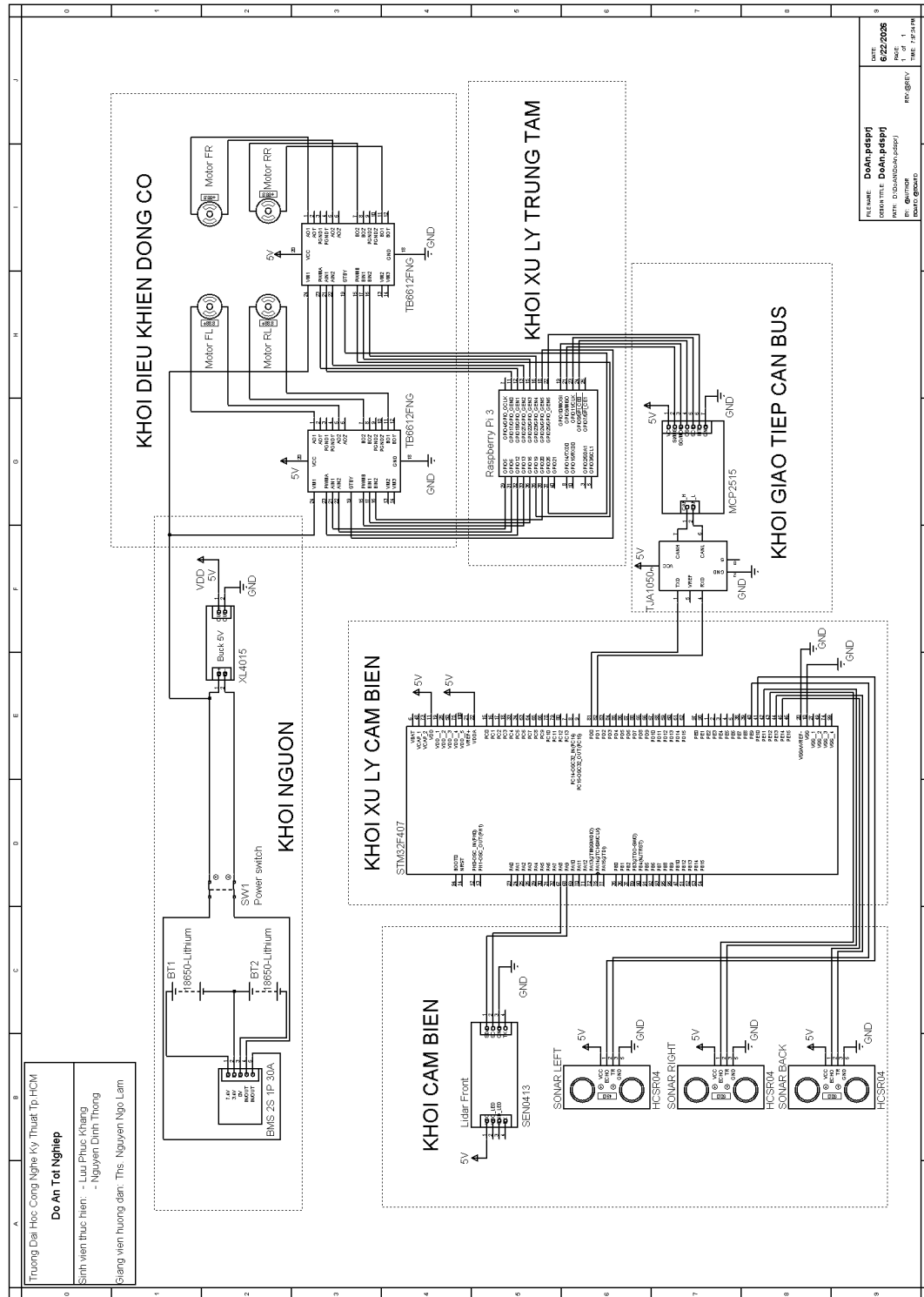
I_{out} : Tổng dòng điện tiêu thụ của các tải

- Nguồn 5V ổn định sau mạch hạ áp được phân phối đến toàn bộ các khối chức năng trong hệ thống, bao gồm Raspberry Pi, vi điều khiển STM32F407, các cảm biến siêu âm, module giao tiếp CAN MCP2515 và mạch điều khiển động cơ. Nhờ thiết kế sử dụng chung một đường GND, các khối xử lý, cảm biến và truyền thông trong hệ thống có cùng mức điện áp tham chiếu, giúp tín hiệu giao tiếp giữa các khối được ổn định và hạn chế sai số do nhiễu điện áp.
- Như vậy, quy trình hoạt động của khối nguồn có thể tóm tắt qua ba giai đoạn chính: (1) cấp và bảo vệ nguồn từ bộ pin thông qua BMS, (2) đóng/ngắt nguồn

tổng thông qua công tắc SW1, và (3) hạ áp, ổn định điện áp đầu ra thông qua mạch XL4015 để cấp cho các khối chức năng của hệ thống.

3.2.8 Sơ đồ nguyên lý toàn hệ thống

Sau khi xác định chức năng của từng khối trong hệ thống, nhóm tiến hành xây dựng sơ đồ nguyên lý tổng thể như sau:



Hình 3.12: Sơ đồ nguyên lý của toàn hệ thống

CHƯƠNG 4 THIẾT KẾ HỆ THỐNG PHẦN MỀM

4.1 Tổng quan về kiến trúc phần mềm hệ thống

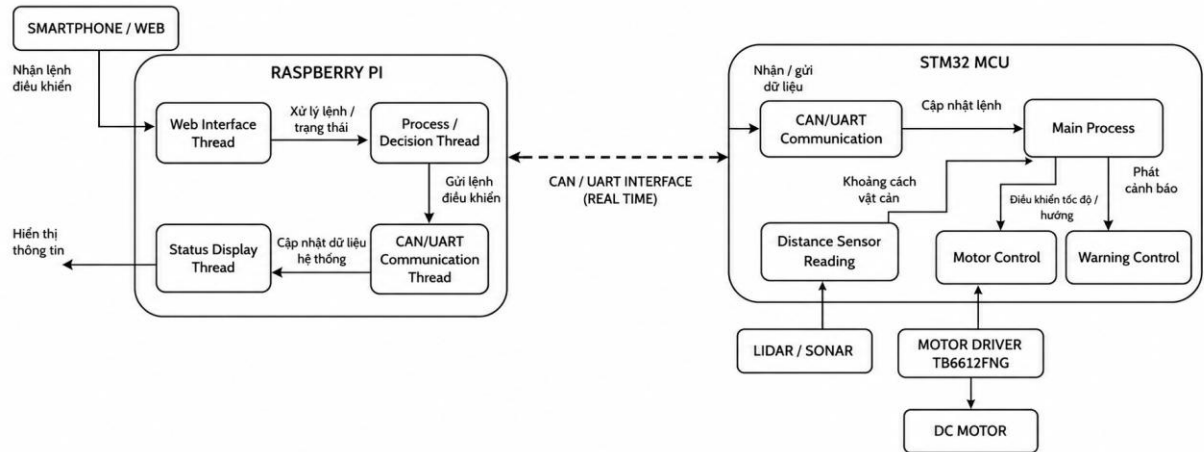
Kiến trúc phần mềm của hệ thống được xây dựng theo hướng phân chia nhiệm vụ giữa Raspberry Pi và vi điều khiển STM32F407:

- Trong đó, Raspberry Pi sẽ đóng vai trò là trung tâm xử lý, thực hiện việc quản lý trạng thái của xe, nhận lệnh điều khiển từ người dùng vào trao đổi dữ liệu với các khối khác trong hệ thống.
- STM32F407 sẽ được sử dụng cho các tác vụ gần với phần cứng hơn như đọc dữ liệu từ các cảm biến xung quanh xe và xử lý các tín hiệu đó sao cho đáp ứng được khả năng phản hồi nhanh trong các trường hợp phát hiện nguy cơ va chạm.

Ưu điểm của cách thiết kế kiến trúc như vậy chính là: Raspberry Pi xử lý tốt các tác vụ liên quan đến giao diện và mạng, nhưng không phù hợp cho các nhiệm vụ về an toàn yêu cầu về độ phản hồi cực thấp. Ngược lại, STM32F407 dù tài nguyên nhỏ nhưng rất phù hợp để xử lý các tác vụ thời gian thực.

4.1.1 Sơ đồ kiến trúc phần mềm hệ thống

Sơ đồ kiến trúc phần mềm của hệ thống được trình bày bên dưới, thể hiện mối quan hệ giữa các khối kiến trúc phần mềm chính của hệ thống. Đồng thời mô tả luồng hoạt động, cách 2 tiến trình nhiệm vụ liên kết và phối hợp tạo nên 1 hệ thống an toàn hoàn chỉnh.



Hình 4.0: Sơ đồ nguyên lý của toàn hệ thống

Luồng hoạt động của hệ thống:

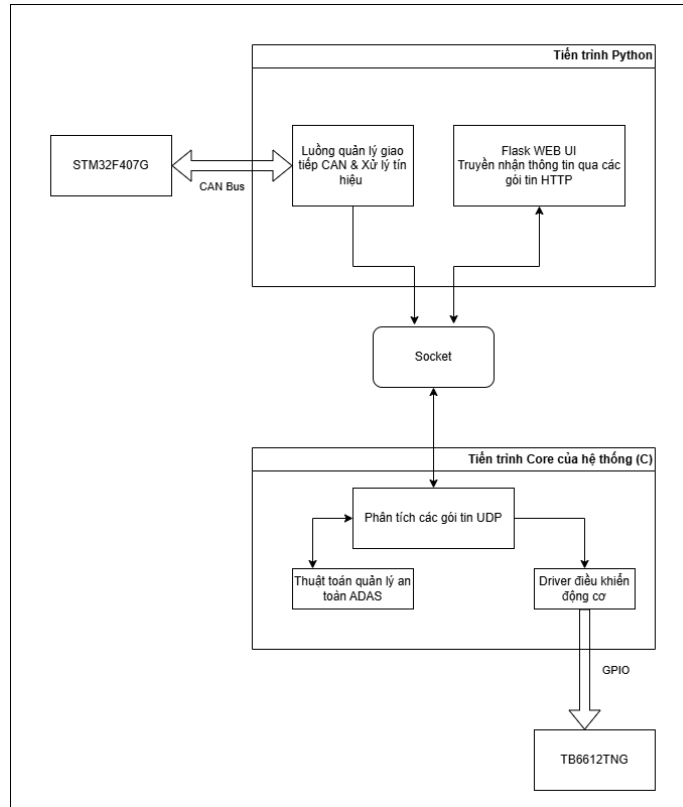
- Khi khởi động, Raspberry Pi khởi chạy giao diện Web và chuẩn bị kênh truyền với STM32F407. Đồng thời, STM32F407 khởi tạo các chân điều khiển và giao tiếp cảm biến.
- Trong quá trình hoạt động: STM32F407 sẽ liên tục đọc cảm biến và gửi dữ liệu lên Raspberry Pi. Raspberry Pi cập nhật thông tin lên giao diện để người dùng theo dõi. Khi người dùng điều khiển, Raspberry Pi gửi lệnh xuống driver điều khiển động cơ TB6612FNG, sau đó sẽ điều khiển motor dựa trên lệnh và trạng thái an toàn từ cảm biến.

Cách tổ chức này giúp phần mềm có các nhiệm vụ rõ ràng: Raspberry Pi phụ trách giao diện và điều khiển, STM32F407 đảm nhận việc quản lý và xử lý tín hiệu từ cảm biến đảm bảo phát hiện nguy hiểm theo thời gian thực. Kiến trúc này có ưu điểm là dễ phát triển, dễ mở rộng thêm chức năng trong tương lai.

4.2 Lập trình phần mềm trên Raspberry Pi 3B

Hệ thống Raspberry Pi được xem như bộ não quản lý hoạt động của toàn bộ phương tiện, với nhiệm vụ nhận dữ liệu cảm biến từ STM32 thông qua CAN bus và

xử lý các thuật toán an toàn ADAS, từ đó xuất tín hiệu điều khiển động cơ phù hợp. Đồng thời máy tính này cũng cung cấp giao diện Web giao diện giúp người dùng theo dõi các thông số cảm biến và trực tiếp điều khiển phương tiện.

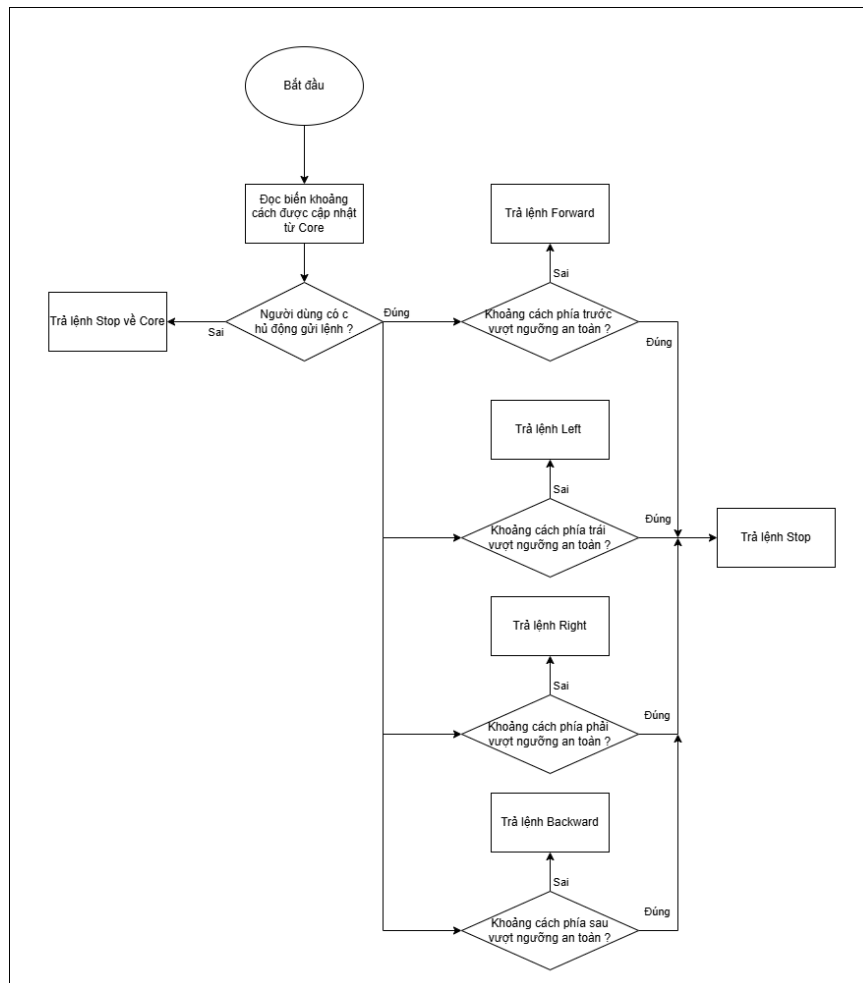


Hình 4.1: Tổng quan hệ thống phần mềm trên Raspberry Pi 3B

Để quản lý điều khiển hệ thống, một tiến trình C được sử dụng để quyết định trạng thái hoạt động của phương tiện. Trong đó, tiến trình thiết lập một cổng UDP Socket để nhận toàn bộ các thao tác lệnh từ chương trình quản lý sử dụng Python, đồng thời tham gia kiểm tra các lệnh để đảm bảo hệ thống hoạt động ổn định và đưa ra các lệnh an toàn xuống lớp Motor.

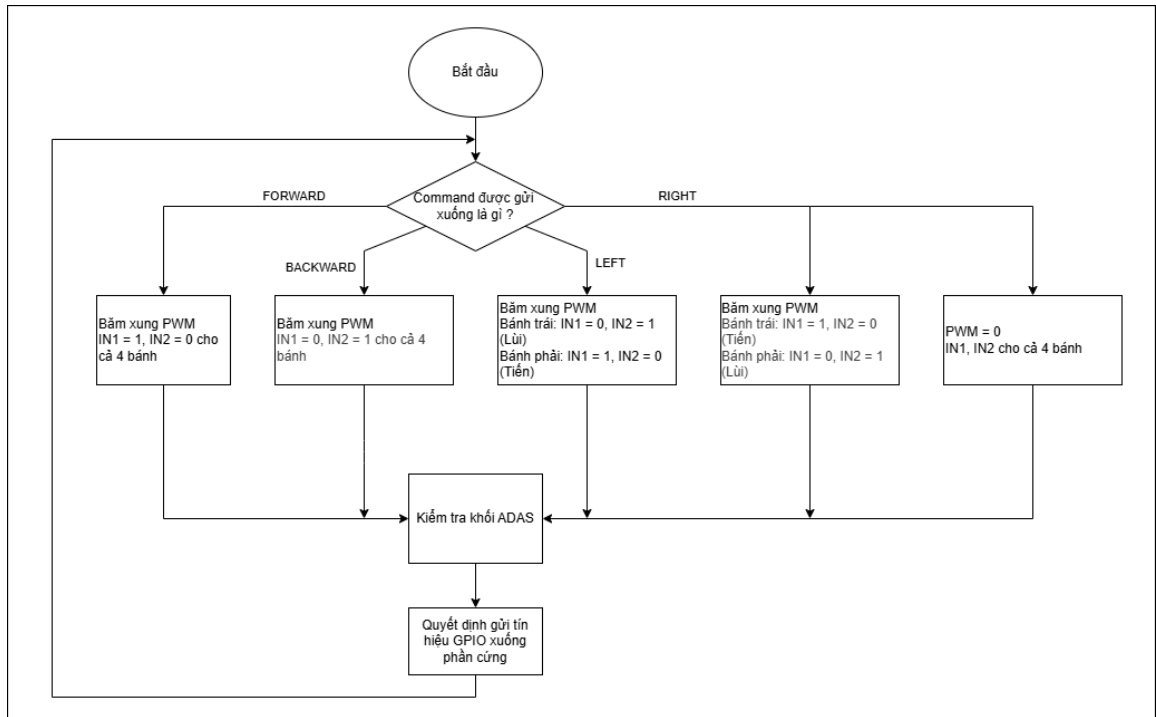
Mặt khác, khối thuật toán lọc ADAS đóng vai trò như một lớp lọc vật lý để kiểm tra lệnh của người dùng và cơ chế chống va chạm. Trong trường hợp vượt

ngưỡng an toàn, hệ thống có thể chủ động can thiệp vào lệnh của người dùng và tự đưa ra quyết định trong để tránh va chạm.



Hình 4.2: Lưu đồ thuật toán xử lý điều khiển người dùng

Ngoài ra, khối Driver điều khiển lại đóng vai trò quyết định khả năng di chuyển của phương tiện. Bằng việc hoạt động điều khiển module TB6612FNG thông qua các chân GPIO, việc quyết định hướng di chuyển của phương tiện được phân giải theo thuật toán sau:



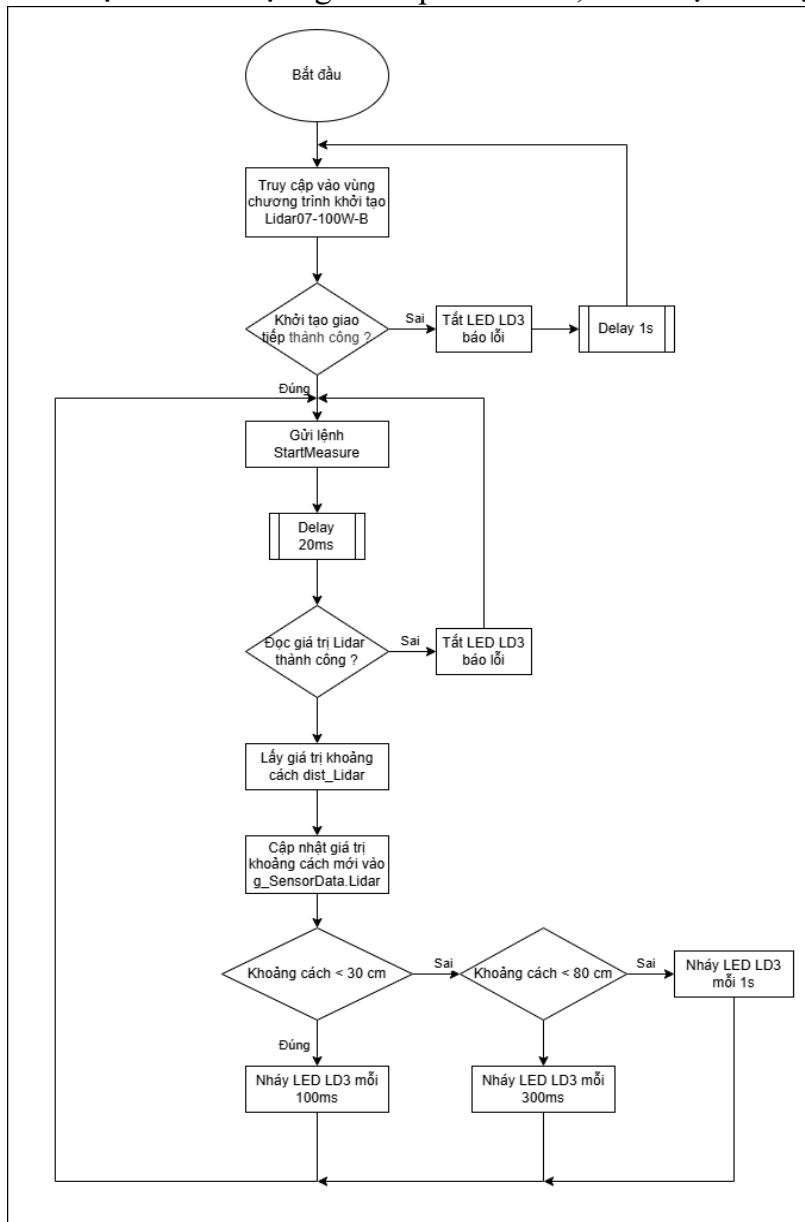
Hình 4.3: Lưu đồ giải thuật điều khiển TB6612FNG

Theo đó khi nhận được tín hiệu điều khiển từ Core, hệ thống thay đổi trạng thái các GPIO tương ứng sau khi kiểm tra thông tin khoảng cách trong khối ADAS. Chi tiết hơn, khi ở module TB6612FNG điều khiển 2 bánh xe bên trái và bên phải của phương tiện được trang bị 2 pin điều khiển xung PWM là PWMA-PWMB. Khi điều khiển, hệ thống dựa vào giá trị được người dùng set trong thanh kéo ở lớp giao diện để quyết định giá trị xung sẽ cũng cấp cho 2 Pin này. Ngoài ra, với các pin IN1-A, IN1-B, IN2-A, IN2-B được kích hoạt các mức trạng thái tùy thuộc vào hướng xoay mong muốn để điều khiển phương tiện.

4.3 Lập trình phần mềm trên STM32F407G-DISC

Khi khởi động hệ thống, STM32F407G-DISC tiến hành khởi tạo HAL và cấu hình các Clock từ chương trình được phần mềm STMCubeMX tự động tạo theo cấu hình thủ công của người dùng. Sau đó, hệ thống thực hiện khởi tạo kernel FreeRTOS để cấu hình các task trong chương trình điều khiển chính do nhóm thực hiện và bắt đầu chạy OS kernel, từ đó bắt đầu vòng lặp của các tiến trình.

Như đã trình bày trước đó, hệ thống được thiết kế để chạy đồng thời ba tiến trình: Task chịu trách nhiệm giao tiếp với Lidar, Task đọc dữ liệu cảm biến Ultra sonic và

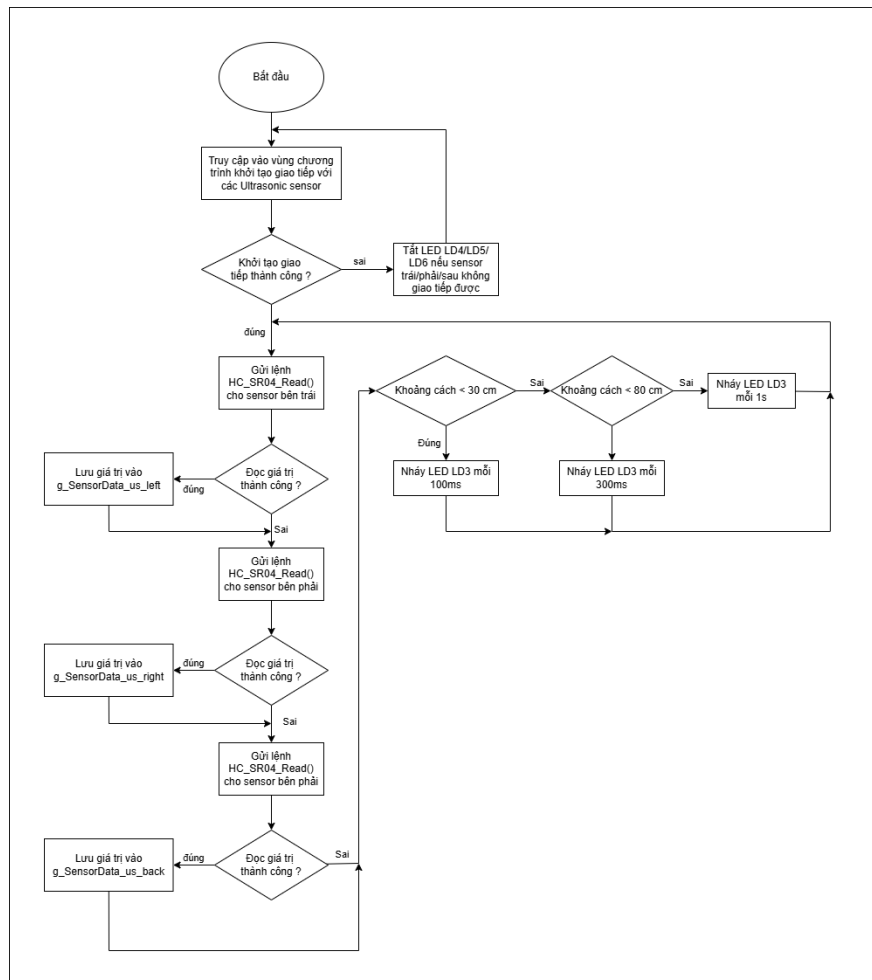


Hình 4.4: Lưu đồ giải thuật Task Lidar

Task đóng gói tất cả dữ liệu để gửi lên tầng trên thông qua CAN bus định kì. Khi Task Lidar được khởi tạo thành công, hệ thống thực hiện vòng lặp đọc và cập nhật dữ liệu liên tục vào buffer dữ liệu sensor. Trong đó, khi chương trình khởi tạo giao thức kết nối với Lidar07 thành công, hệ thống sẽ gửi lệnh StartMeasure để Lidar07 gửi tín hiệu khoảng cách đã tính toán về. Hệ thống khi này phân giải dữ liệu để kiểm tra tính hợp lệ, nếu thành công hệ thống lập tức cập nhật giá trị mới

vào g_Sensor_Lidar. Chỉ khi đảm bảo hệ thống đã nhận được giá trị đã cập nhật, hệ thống sẽ tiếp tục thay đổi trạng thái LED LD3 tích hợp trên board để người dùng chủ động biết được đại khái trạng thái khoảng cách đã đo được. Nguyên nhân hệ thống được lập trình chức năng nháy LED này vì STM32F407G-DISC không có công giao

tiếp trực tiếp UART để giao tiếp với các công cụ lập trình như các dòng board ESP hay ARDUINO. Để người dùng biết được trạng thái hoạt động của hệ thống giao tiếp Lidar này có hoạt động chính xác mà không cần thủ công xây dựng cơ chế giao tiếp UART với board, cơ chế nháy LED này được đánh giá là phù hợp để người dùng nhanh chóng nhận biết được tình hình hệ thống và chủ động gỡ lỗi nếu tồn tại.



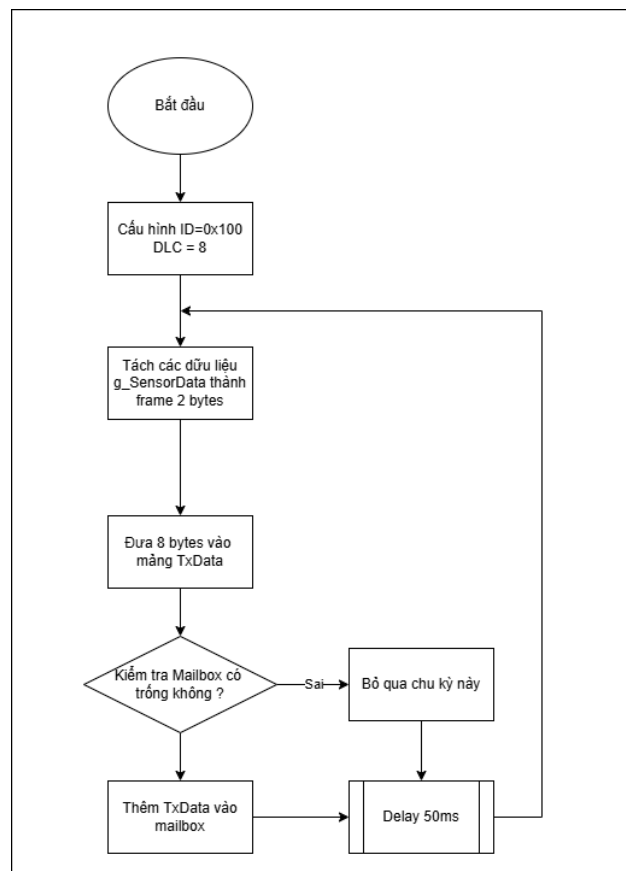
Hình 4.5: Lưu đồ giả thuật Task US

Khác biệt với Task Lidar, Task Radar đồng thời đảm nhận quản lý cả ba sensor ultra sonic phía bên trái, bên phải và đằng sau phương tiện. Do các sensor này có cơ chế giao tiếp đơn giản và tần số hoạt động không quá phức tạp nên task có thể đảm nhận mà không gây ảnh hưởng đến thời gian timeout cho mỗi lần đọc các cảm biến trong thời gian ngắn, đảm bảo hệ thống hoạt động hiệu quả. Theo đó, Sau khi đã khởi

tạo cơ chế giao tiếp thành công, Hệ thống sử dụng lệnh HC_SR04_Read() để yêu cầu dữ liệu từ sensor. Sau khi nhận được tín hiệu, Hệ thống kiểm tra tính hợp lệ và lưu dữ liệu đã cập nhật vào g_SensorData ở các phía. Đồng thời khi đã cập nhật dữ liệu thành công hệ thống cũng thực hiện thay đổi trạng thái các LED LD4, LD5 và LD6 để người dùng quan sát được trạng thái đọc dữ liệu ở cách cánh bên trái, bên phải và phía sau.

4.4 Lập trình giao tiếp CAN giữa Raspberry Pi và STM32

Ở phía của STM32, hệ thống đảm nhận tổng hợp các dữ liệu khoảng cách thu thập được từ các task quản lý giao tiếp với các sensor và gán chúng vào frame dữ liệu CAN trước khi truyền đến Raspberry Pi.

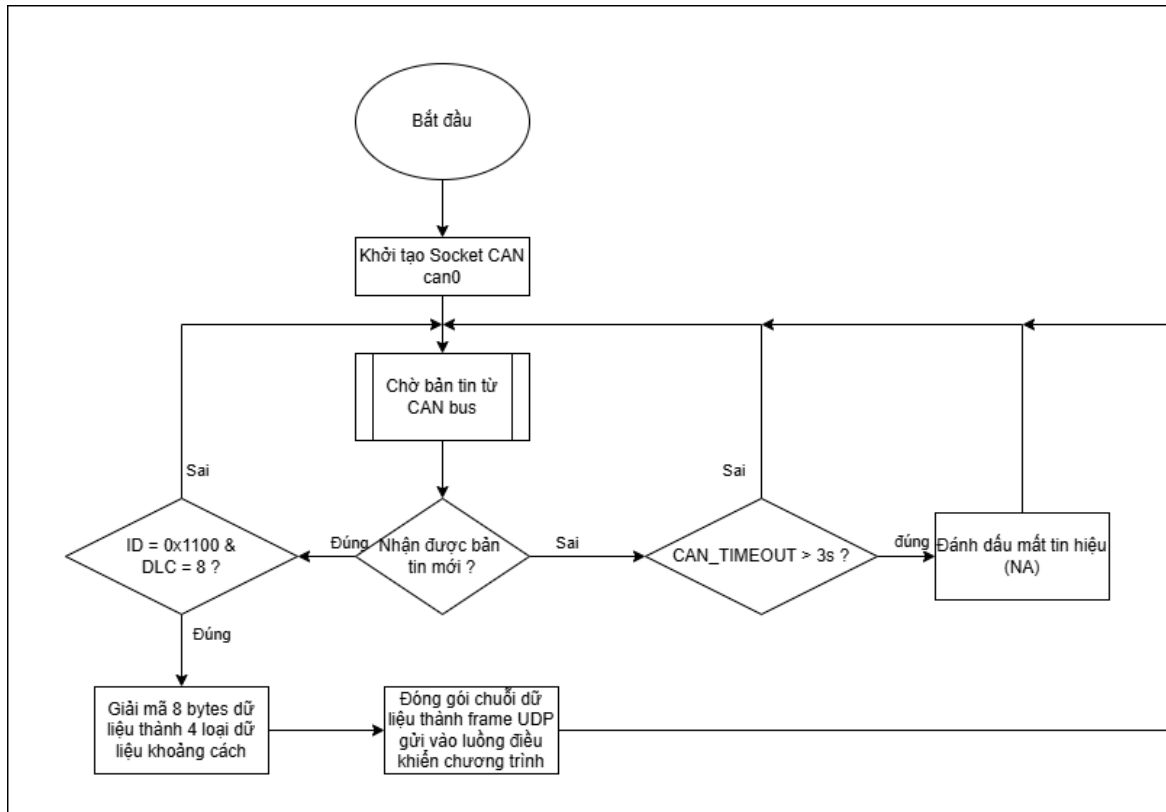


Hình 4.6: Lưu đồ giải thuật giao tiếp CAN trên STM323

Trong Task CAN quản lý công việc đóng gói và truyền tin, hệ thống thực hiện tách dữ liệu cảm biến khoảng cách ở các phía thành mỗi mảng kích thước 2 bytes. Do

1 frame truyền của CAN classic có kích thước 8 bytes, việc tách này đảm bảo 4 loại dữ liệu được gán khớp vào 8 bytes này. Sau khi đã có được các dữ liệu đóng gói vào mảng TxData làm payload của CAN frame. Hệ thống kiểm tra xem mailbox hiện tại có đang trống không. Do đặc tính mailbox chỉ được phép lưu trữ 1 CAN frame trong một lần truyền, nếu không kiểm tra thì mailbox sẽ bị ghi đè dẫn đến mất thông tin ở lần truyền gần nhất. Khi mailbox đã trống, hệ thống tiến hành ghi vào và hoàn tất một chu kì ghi dữ liệu vào CAN frame trước khi được truyền đến Raspberry.

Mặt khác, để hiểu được thông tin được truyền đến từ STM32, bản Raspberry Pi cũng phải được thiết lập bộ điều khiển để giải mã thông tin trên CAN bus được truyền gián tiếp qua MCP2515.



Hình 4.7: Lưu đồ giải thuật giao tiếp CAN trên Raspberry

Khi hoạt động, Raspberry sử dụng driver giao tiếp với MCP2515 được khởi tạo sẵn bởi Linux kernel để quản lý đường truyền CAN. Đồng thời, để đảm bảo đường truyền

luôn được kích hoạt, hệ thống được cung cấp một script bash `can_watchdog` để kiểm tra trạng thái của mạng. Khi phát hiện trạng thái STOPPED hay BUS-OFF, script sẽ tự khởi động lại mạng giao tiếp thông qua command:

```
ip link set dev can0 down  
ip link set dev can0 type can bitrate 500000 restart-ms 100  
ip link set dev can0 up
```

Trong quá trình hoạt động chính của mạng CAN, hệ thống tạo một luồng chạy ngầm độc lập để liên tục lắng nghe CAN bus. Khi nhận tín hiệu, hệ điều hành sẽ giải mã 8 bytes dữ liệu từ frame và chia cho 10 để chuyển dữ liệu từ hệ mm sang cm và đóng gói thành chuỗi gửi lên mạng thông qua giao thức UDP cho chương trình chính.

CHƯƠNG 5 KẾT QUẢ VÀ ĐÁNH GIÁ HỆ THỐNG

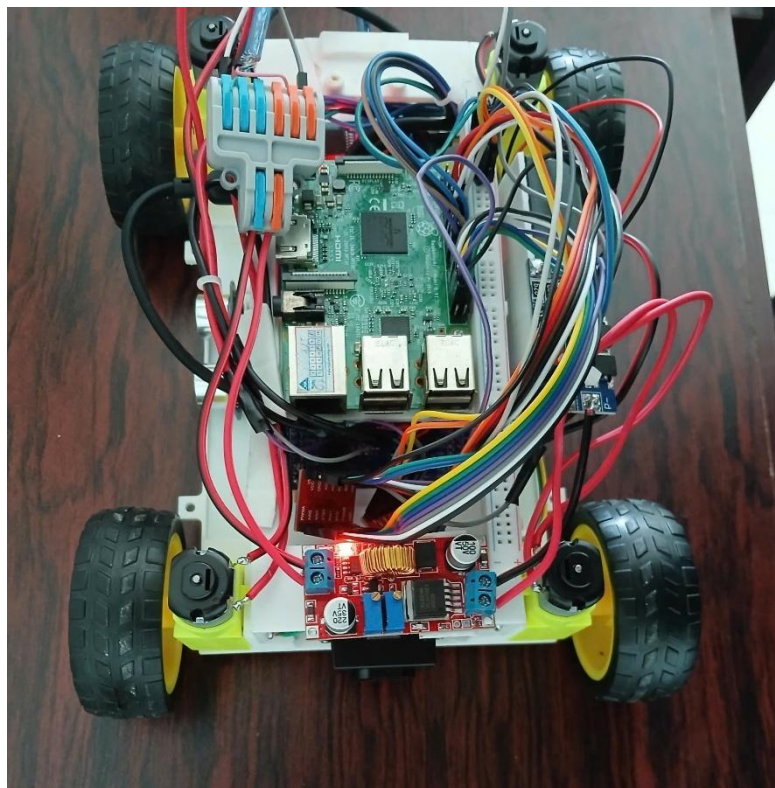
5.1 KẾT QUẢ THI CÔNG HỆ THỐNG

5.1.1 Kết quả xây dựng phần cứng

Ở cuối giai đoạn phát triển sản phẩm, phần cứng đã được triển khai thành công với một khung xe được thiết kế có khả năng kết nối các cảm biến ở bốn phía, bốn động cơ kết nối trực tiếp với các bánh dẫn động ở các góc mô hình cùng với các bộ điều khiển, module, IC được đặt phía trên.

Kết quả thực tế cho thấy phương tiện được xây dựng rõ ràng, phù hợp với tiêu chí đặt ra từ giai đoạn ý tưởng và triển khai đồ án.

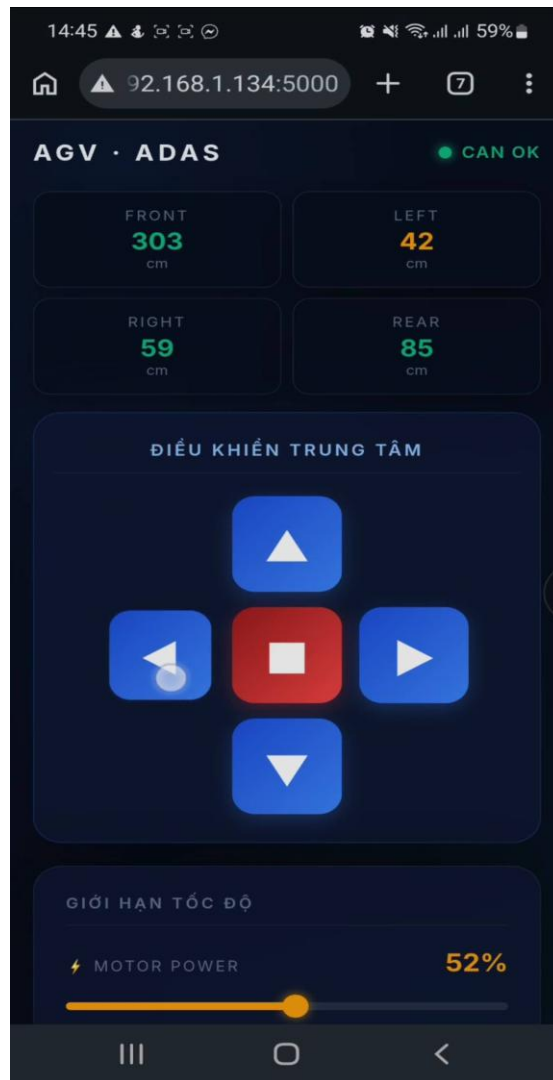
Thành phần	Khả năng hoạt động
STM32F407G-DISC	Hoạt động tốt
Raspberry Pi 3B	Hoạt động tốt
2 module TB6612FNG	Hoạt động ổn định
4 động cơ DC	Đáp ứng khả năng truyền động đến bánh xe
TJA1050	Đảm bảo giao tiếp CAN cho STM32 tốt
MCP2515	Hoạt động ổn định trong giao tiếp CAN hỗ trợ Raspberry
Nguồn	Hoạt động ổn định trong công tác cung cấp điện áp 5V cho hệ thống
Cảm biến Lidar07-100W-B	Hoạt động tốt
3 Cảm biến HC-SR04	Đáp ứng điều kiện hoạt động yêu cầu
Khung xe	Đáp ứng khả năng chịu đựng khối lượng các linh kiện được sử dụng



Hình 5.1: Kết quả thi công phần cứng

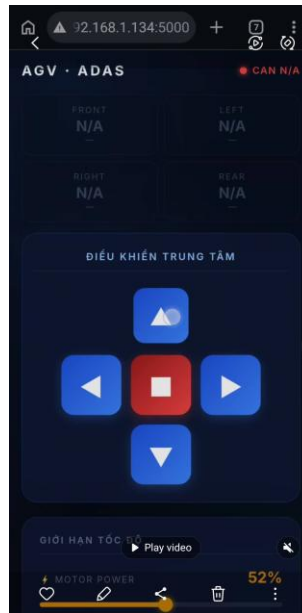
5.1.2 Kết quả xây dựng giao diện người dùng

Cùng với việc xây dựng phần cứng, một web interface để giao tiếp với người dùng cũng được xây dựng dựa trên việc sử dụng html, css và quản lý bởi code python sử dụng thư viện Flask. Giao diện được thiết kế với phong nền tối phù hợp điều khiển trong không gian thiếu ánh sáng, giao diện đơn giản chỉ gồm 5 nút nhấn để điều khiển tịnh tiến và xoay phương tiện, một thanh trượt để thay đổi công suất động cơ và một bảng hiển thị thông số các khoảng cách.

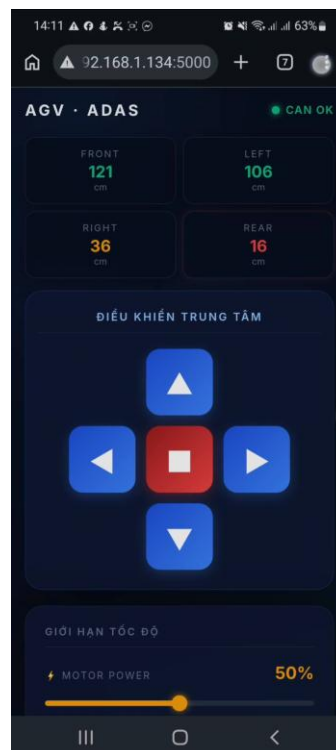


Hình 5.2: Giao diện người dùng trên điện thoại di động

Ngoài ra, hệ thống giao diện cũng đáp ứng được khả năng cập nhật trạng thái khi các dữ liệu khoảng cách thu thập cảm biến được cập nhật và thay đổi hiển thị liên tục. Đồng thời, hệ thống cũng đáp ứng khả năng thông báo người dùng trong tình trạng 1 hoặc nhiều cảm biến mất liên lạc độc lập với các luồng khác, cũng như hiển thị cảnh báo khi CAN bus bị ngắt bởi lý do ngoại lực hoặc phần mềm gặp lỗi không mong muốn.



Hình 5.3: Giao diện người dùng hiển thị CAN bị ngắt



Hình 5.4: Giao diện người dùng hiển thị khoảng cách phía sau vượt ngưỡng

5.2 ĐÁNH GIÁ ĐỘ ỔN ĐỊNH CỦA HỆ THỐNG THEO THỜI GIAN THỰC

Khi phương tiện được điều khiển, hệ thống biểu hiện tốt trong khoảng thời gian dài điều khiển, các cảm biến vẫn được cập nhật liên tục và chính xác các thông tin khoảng các khi phương tiện tiến dần vào vật cản hoặc di chuyển trong không gian hẹp. Ngoài ra, hệ thống ADAS được kích hoạt đúng lúc khi phương tiện tiến quá gần đến vật cản, đảm bảo khả năng tránh va chạm đã được đề ra. Tuy nhiên, hệ thống hoạt động tương đối nhạy dẫn đến một vài hành vi kích hoạt hệ thống ADAS đột ngột.

CHƯƠNG 6 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận chung quá trình xây dựng đề tài

Trong khuôn khổ đề tài, nhóm đã hoàn tất công tác nghiên cứu, xây dựng và tích hợp hệ thống Cảnh báo va chạm trên phương tiện điều khiển. Hệ thống được xây dựng dựa trên sự kết hợp giữa phần mềm quản lý hoạt động của phương tiện điều khiển và phần mềm quản lý hoạt động giao tiếp với môi trường.

Về mặt phần cứng, nhóm đã nghiên cứu và xây dựng thành công mô hình mô phỏng phương tiện điều khiển bao gồm các module chính bao gồm STM32F407G-DISC, Raspberry Pi 3B, TJA1050, MCP2515... Các thành phần được tích hợp và giao tiếp thành công với nhau, đảm bảo hệ thống được vận hành có sự phối hợp giữa các bộ phận.

Về giao diện người dùng, nhóm thực hiện xây dựng thành công giao diện Web bằng html, css dưới nền điều khiển bằng Python sử dụng thư viện Flask và WebSocket để người dùng có thể điều khiển phương tiện và giám sát các trạng thái của hệ thống tại thời gian điều khiển.

Về module điều khiển động cơ, nhóm cũng hoàn tất công tác tích hợp các động cơ vào IC TB6612FNG và xây dựng một chương trình tích hợp hệ thống ADAS cơ bản để quản lý và giao tiếp với các IC này nhằm mục đích điều khiển động cơ. Bên cạnh đó, hệ thống cũng đáp ứng nhu cầu chủ động ghi đè lên lệnh được gửi từ người dùng thông qua Web interface bằng lệnh của hệ thống ADAS để chủ động tránh khả năng va chạm với vật thể.

Kết quả thực nghiệm thông qua điều khiển trực tiếp phương tiện và quan sát các thông tin hệ thống cho thấy hệ thống đáp ứng các thiết kế được đề ra. Các chức năng cơ bản như Thu thập thông tin cảm biến, Truyền thông tin qua mạng giao tiếp CAN, Đẩy dữ liệu khoảng cách lên lớp giao diện người dùng, Thu thập lệnh điều khiển từ người

dùng, Điều khiển động cơ dưới sự can thiệp của module ADAS đều hoạt động ổn định.

Kết luận chung, đồ án đã đảm bảo mục tiêu nghiên cứu, xây dựng và tích hợp hệ thống Cảnh báo va chạm trên phương tiện điều khiển, vừa đáp ứng khả năng điều khiển phương tiện không dây vừa đảm bảo tính an toàn được đề ra trong hệ thống.

6.2 Hạn chế của đề tài

Trong quá trình thực nghiệm, hệ thống đã hoàn thành các chức năng chính và đảm bảo khả năng vận hành, tuy nhiên hệ thống vẫn tồn tại một số hạn chế:

- Hệ thống bị ảnh hưởng trực tiếp bởi phần cứng. Với độ phức tạp trong các kết nối trong mô hình hiện tại, hệ thống không tránh được rủi ro mất kết nối trong quá trình vận chuyển, điều khiển.
- Các tín hiệu bị nhiễu dẫn đến thông tin truyền đến trung tâm khiến hệ thống ADAS kích hoạt đột ngột.
- Hệ thống ADAS chỉ dừng lại ở mức 1 hoặc 2, chưa được tích hợp các hệ thống trí tuệ nhân tạo để tăng khả năng chủ động của hệ thống trong vận hành.
- Độ phức tạp phần cứng hiện tại của mô hình dẫn đến khó khăn trong việc tích hợp thêm module định vị GPS để hỗ trợ xác định vị trí của phương tiện điều khiển
- Hệ thống được xây dựng với tiêu chí chỉ mô phỏng hoạt động của một phương tiện tham gia giao thông thực tế, việc đánh giá hệ thống được dựa hoàn toàn vào thử nghiệm trực tiếp và quan sát các thông tin quản lý nên các đánh giá chỉ dừng lại ở mức thử nghiệm, chưa phải triển khai vận hành trên quy mô thực tế.

6.3 Định hướng phát triển hệ thống

Trong tương lai, mô hình hệ thống có thể được tiếp tục công tác nghiên cứu và xây dựng, tích hợp với nhiều hướng khác nhau:

- Hệ thống được tích hợp thêm các cơ chế thu thập thông tin khoảng cách, môi trường, địa hình... để tăng khả năng giao tiếp với môi trường của phương tiện điều khiển
- Bổ sung các chức năng cho hệ thống ADAS để tăng khả năng tự vận hành của hệ thống mà không cần sự can thiệp, điều khiển trực tiếp của người dùng.
- Triển khai giao tiếp giữa người dùng với hệ thống thông qua nhiều giao thức khác nhau, tăng cường các thông tin điều khiển hệ thống nhằm đảm bảo khả năng điều khiển và quản lý phương tiện.
- Hoàn thiện và thay đổi thiết kế phần cứng mô hình nhằm đáp ứng tốt hơn khả năng thay thế, tích hợp các module phần cứng và phần mềm.

PHỤ LỤC

TÀI LIỆU THAM KHẢO

- [1] Khoa Điện - Điện tử, Trường Đại học Công nghệ kỹ thuật Tp.HCM, “Giáo trình và tài liệu giảng dạy trong chương trình đào tạo: Vi xử lý, Truyền số liệu, Hệ thống nhúng, Cơ sở IOT” Biên soạn nội bộ, TP. Hồ Chí Minh, 2025.
- [2] Khoa Điện - Điện tử, Trường Đại học Công nghệ kỹ thuật Tp.HCM, “Khóa luận tốt nghiệp ngành Công nghệ kỹ thuật máy tính: Thiết kế hệ thống cảnh báo va chạm ô tô” Biên soạn nội bộ, TP. Hồ Chí Minh, 2025.
- [3] National Highway Traffic Safety Administration, “Driver Assistance Technologies” NHTSA, 2026.
- [4] European Commission, “Advanced Driver Assistance Systems” European Commission Mobility and Transport, 2026.
- [5] STMicroelectronics, “STM32F405xx/STM32F407xx Datasheet”
Performance Line Datasheet, STMicroelectronics, 2015.
- [6] Raspberry Pi Ltd., “Raspberry Pi 3 Model B+ Product Brief”, “Raspberry Pi Computer Hardware Documentation,” Raspberry Pi Documentation, 2025.
- [7] STMicroelectronics, “Description of STM32F4 HAL and Low-Layer Drivers” User Manual UM1725, STMicroelectronics.
- [8] Microchip Technology Inc., “MCP2515 Stand-Alone CAN Controller with SPI Interface” Microchip Datasheet.
- [9] Texas Instruments, “Introduction to the Controller Area Network (CAN)” Application Report SLOA101, 2018.